Code Export

Source Code PDF Report

Generated	2026-04-21 22:57
Root directory	C:\Users\LENOVO\Documents\Java_run\EmployeeManagementSystem\EmployeeManagementSystem
Files found	97
Total lines	8,998

Table of Contents

1. Parent: README.md (24 lines)
2. Current: .mvn\wrapper\maven-wrapper.properties (4 lines)
3. Current: pom.xml (131 lines)
4. Current: src\main\java\com\example\EmployeeManagementSystem\code\CodeToPdf.java (347 lines)
5. Current: src\main\java\com\example\EmployeeManagementSystem\config\BasicAuthConfig.java (52 lines)
6. Current: src\main\java\com\example\EmployeeManagementSystem\config\JWTauth\JWTAuthConfig.java (68 lines)
7. Current: src\main\java\com\example\EmployeeManagementSystem\config\Seassion\SessionAuthConfig.java (32 lines)
8. Current: src\main\java\com\example\EmployeeManagementSystem\config\swaggerConfig.java (38 lines)
9. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\ApiKeyController.java (106 lines)
10. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\AuthController.java (110 lines)
11. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\DeliveryController.java (33 lines)
12. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\EmployeeController.java (65 lines)
13. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\GlobalExceptionHandler.java (120 lines)
14. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\LeaveRequestController.java (60 lines)
15. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\LoginController.java (17 lines)
16. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\OauthController.java (168 lines)
17. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\RestaurantController.java (121 lines)
18. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\SchedulerController.java (50 lines)
19. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\SessionAuthController.java (86 lines)
20. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\SubscriptionController.java (73 lines)
21. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\TotpController.java (101 lines)
22. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\VendorController.java (67 lines)
23. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AccessTokenRequest.java (18 lines)
24. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\ActionDTO.java (34 lines)
25. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AuthRequest.java (28 lines)
26. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AuthResponse.java (28 lines)

27. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\EmployeeDTO.java (50 lines)
28. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\LeaveRequestDTO.java (48 lines)
29. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\LeaveResponseDTO.java (73 lines)
30. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\RestaurantDTO.java (42 lines)
31. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\RestaurantRequest.java (46 lines)
32. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\SubscriptionDTO.java (103 lines)
33. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\SubscriptionRequest.java (72 lines)
34. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\VendorDTO.java (26 lines)
35. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\VendorRequest.java (36 lines)
36. Current: src\main\java\com\example\EmployeeManagementSystem\EmployeeManagementSystemApplication.java (16 lines)
37. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\ApiKey.java (79 lines)
38. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Delivery.java (93 lines)
39. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Employee.java (173 lines)
40. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\LeaveRequest.java (105 lines)
41. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\RefreshToken.java (62 lines)
42. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Restaurant.java (58 lines)
43. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Subscription.java (105 lines)
44. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Vendor.java (108 lines)
45. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\DeliveryStatus.java (9 lines)
46. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\LeaveStatus.java (9 lines)
47. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\LeaveType.java (8 lines)
48. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\MealSlot.java (8 lines)
49. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\Permission.java (19 lines)
50. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\Role.java (36 lines)
51. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\ScheduleType.java (7 lines)
52. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\Status.java (8 lines)
53. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\SubscriptionStatus.java (8 lines)
54. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\DuplicateRequestException.java (8 lines)
55. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\EmployeeNotFound.java (10 lines)
56. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidEndDateException.java (8 lines)
57. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidManagerException.java

(8 lines)

58. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidStartDateException.java (8 lines)

59. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\LeaveRequestNotFoundException.java (8 lines)

60. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\OverlappingLeaveException.java (8 lines)

61. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\RestaurantNotFoundException.java (8 lines)

62. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\SubscriptionAlreadyExists.java (8 lines)

63. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\UnauthorizedAccessException.java (7 lines)

64. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\VendorNotFoundException.java (7 lines)

65. Current: src\main\java\com\example\EmployeeManagementSystem\Filter\ApiKeyFilter.java (45 lines)

66. Current: src\main\java\com\example\EmployeeManagementSystem\Filter\JwtAuthFilter.java (73 lines)

67. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\ApiKeyRepository.java (36 lines)

68. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\DeliveryRepository.java (23 lines)

69. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\EmployeeRepo.java (16 lines)

70. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveRequestRepo.java (69 lines)

71. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\RefreshTokenRepository.java (11 lines)

72. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\RestaurantRepository.java (31 lines)

73. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\SubscriptionRepository.java (32 lines)

74. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\VendorRepo.java (13 lines)

75. Current: src\main\java\com\example\EmployeeManagementSystem\Scheduler\DeliveryScheduler.java (30 lines)

76. Current: src\main\java\com\example\EmployeeManagementSystem\Scheduler\LeaveStatusScheduler.java (107 lines)

77. Current: src\main\java\com\example\EmployeeManagementSystem\security\ApiKeyAuthenticationProvider.java (85 lines)

78. Current: src\main\java\com\example\EmployeeManagementSystem\security\ApiKeyAuthenticationToken.java (84 lines)

79. Current: src\main\java\com\example\EmployeeManagementSystem\Service\CombinedUserDetailService.java (41 lines)

80. Current: src\main\java\com\example\EmployeeManagementSystem\Service\DeliveryService.java (166 lines)

81. Current: src\main\java\com\example\EmployeeManagementSystem\Service\DeliveryTimeService.java (74 lines)

82. Current: src\main\java\com\example\EmployeeManagementSystem\Service\EmployeeService.java (134 lines)

83. Current: src\main\java\com\example\EmployeeManagementSystem\Service\LeaveRequestService.java (225 lines)

84. Current: src\main\java\com\example\EmployeeManagementSystem\Service\MealSlotService.java (22 lines)

85. Current: src\main\java\com\example\EmployeeManagementSystem\Service\RefreshTokenService.java (43 lines)

86. Current: src\main\java\com\example\EmployeeManagementSystem\Service\RestaurantService.java (157 lines)

87. Current: src\main\java\com\example\EmployeeManagementSystem\Service\SubscriptionService.java (225 lines)

88. Current: src\main\java\com\example\EmployeeManagementSystem\Service\TotpService.java (40 lines)

89. Current: src\main\java\com\example\EmployeeManagementSystem\Util\JWTUtil.java (59 lines)

90. Current: src\main\resources\application.yaml (66 lines)

91. Current: src\main\resources\static\dashboard.html (660 lines)

92. Current: src\main\resources\static\employee-dashboard.html (654 lines)

93. Current: src\main\resources\static\google.html (627 lines)

94. Current: src\main\resources\static\login.html (586 lines)

95. Current: src\main\resources\static\manager-dashboard.html (604 lines)

96. Current: src\main\resources\static\vendor-dashboard.html (470 lines)

97. Current: src\test\java\com\example\EmployeeManagementSystem\EmployeeManagementSystemApplicationTests.java (14 lines)

1 Parent: README.md

24 lines

```
Employee Leave & Meal Subscription Management System
 Overview

This project is a Spring Boot-based backend application that manages employee operations along with an integrated meal subscription system.

It allows employees to:

Manage their profiles
Apply for leaves
 Subscribe to meals
  Features
  Employee Management
 Add, update, and delete employees
 View employee details
 Secure authentication (JWT / Basic Auth)
  Leave Management
 Apply for leave
 Approve/reject leave requests
 Track leave history
  Meal Subscription
 Subscribe to meals
 Manage daily/weekly meal plans
 Link meals with employees

```

2 Current: .mvn\wrapper\maven-wrapper.properties

4 lines

```
wrapperVersion=3.3.4
distributionType=only-script
distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.14/apache-maven-3.9.14-bin.zip

```

3 Current: pom.xml

131 lines

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.0.5</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.example</groupId>
    <artifactId>EmployeeLeaveManagementSystem</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name/>
    <description/>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
    </scm>
    <properties>
        <java.version>21</java.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springdoc</groupId>
            <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
            <version>3.0.2</version>
        </dependency>
        <dependency>
            <groupId>dev.samstevens.totp</groupId>
            <artifactId>totp-spring-boot-starter</artifactId>
            <version>1.7.1</version>
        </dependency>
        <dependency>
            <groupId>org.springframework.session</groupId>
            <artifactId>spring-session-core</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
        </dependency>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>com.itextpdf</groupId>
```

```
            <artifactId>layout</artifactId>
            <version>7.2.5</version>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-api</artifactId>
            <version>0.13.0</version>
            <scope>compile</scope>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-impl</artifactId>
             <version>0.13.0</version>
             <scope>runtime</scope>
         </dependency>
         <dependency>
             <groupId>io.jsonwebtoken</groupId>
             <artifactId>jjwt-jackson</artifactId>
             <version>0.13.0</version>
             <scope>runtime</scope>
         </dependency>
         <dependency>
             <groupId>org.springframework.boot</groupId>
             <artifactId>spring-boot-starter-security-oauth2-client</artifactId>
         </dependency>

         <dependency>
             <groupId>org.springframework.boot</groupId>
             <artifactId>spring-boot-starter-security-oauth2-client-test</artifactId>
             <scope>test</scope>
         </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>

</project>

```

```java
package com.example.EmployeeManagementSystem.code;

import com.itextpdf.kernel.colors.ColorConstants;
import com.itextpdf.kernel.colors.DeviceRgb;
import com.itextpdf.kernel.font.PdfFont;
import com.itextpdf.kernel.font.PdfFontFactory;
import com.itextpdf.kernel.geom.PageSize;
import com.itextpdf.kernel.pdf.PdfDocument;
import com.itextpdf.kernel.pdf.PdfWriter;
import com.itextpdf.layout.Document;
import com.itextpdf.layout.element.AreaBreak;
import com.itextpdf.layout.element.Cell;
import com.itextpdf.layout.element.Paragraph;
import com.itextpdf.layout.element.Table;
import com.itextpdf.layout.element.Text;
import com.itextpdf.layout.properties.AreaBreakType;
import com.itextpdf.layout.properties.UnitValue;
import com.itextpdf.io.font.constants.StandardFonts;

import java.io.IOException;
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.*;

/**
 * CodeToPdf - Extracts all source code files from the current, parent,
 * and immediate child directories into a formatted PDF document.
 *
 * Dependencies (add to your pom.xml or build.gradle):
 *   com.itextpdf:itext7-core:7.2.5  (or itext7 bundle)
 *
 * Compile:
 *   javac -cp itext7-core-7.2.5.jar:. CodeToPdf.java
 *
 * Run:
 *   java -cp itext7-core-7.2.5.jar:. CodeToPdf
 *
 * Or with Maven:
 *   mvn compile exec:java -Dexec.mainClass=CodeToPdf
 */
public class CodeToPdf {

    //  Configuration

    /** Output PDF filename */
    private static final String OUTPUT_FILE = "code_export.pdf";

    /** Maximum file size to include (skip very large generated files) */
    private static final long MAX_FILE_SIZE_BYTES = 500_000; // 500 KB

    /** Source code file extensions to include */
    private static final Set<String> CODE_EXTENSIONS = new LinkedHashSet<>(Arrays.asList(
            // JVM languages
            ".java", ".kt", ".groovy", ".scala", ".clj",
            // Web
            ".js", ".ts", ".jsx", ".tsx", ".html", ".css", ".scss", ".less",
            // Python / Ruby / PHP
            ".py", ".rb", ".php",
            // Systems / low-level
            ".c", ".cpp", ".cc", ".h", ".hpp", ".rs", ".go", ".swift",
            // Data / config
            ".xml", ".json", ".yaml", ".yml", ".toml", ".properties", ".env",
            // Shell / scripts
            ".sh", ".bash", ".zsh", ".bat", ".ps1",
            // Build
            ".gradle", ".cmake", "Makefile", "Dockerfile",
            // Other
            ".sql", ".md", ".txt"
    ));
```

```java

    /** Directories to skip entirely */
    private static final Set<String> SKIP_DIRS = new HashSet<>(Arrays.asList(
            ".git", ".svn", ".hg", "node_modules", "target", "build",
            "out", ".idea", ".vscode", "__pycache__", ".gradle", "dist", "vendor"
    ));

    //  Colors
    private static final DeviceRgb COLOR_HEADER_BG    = new DeviceRgb(0x18, 0x5F, 0xA5); // blue
    private static final DeviceRgb COLOR_HEADER_TEXT  = new DeviceRgb(0xFF, 0xFF, 0xFF);
    private static final DeviceRgb COLOR_FILE_TITLE   = new DeviceRgb(0x18, 0x5F, 0xA5);
    private static final DeviceRgb COLOR_CODE_BG      = new DeviceRgb(0xF7, 0xF7, 0xF7);
    private static final DeviceRgb COLOR_LINE_NUM     = new DeviceRgb(0x99, 0x99, 0x99);
    private static final DeviceRgb COLOR_TOC_HEADER   = new DeviceRgb(0x18, 0x5F, 0xA5);

    //  Font sizes
    private static final float FONT_TITLE   = 20f;
    private static final float FONT_SECTION = 13f;
    private static final float FONT_FILE    = 11f;
    private static final float FONT_CODE    =  7.5f;
    private static final float FONT_TOC     = 10f;

    //  State
    private final List<FileEntry> fileEntries = new ArrayList<>();
    private int totalLines = 0;

    //

     public static void main(String[] args) throws Exception {
         new CodeToPdf().run();
     }

     private void run() throws Exception {
         Path currentDir = Paths.get("").toAbsolutePath();
         Path parentDir  = currentDir.getParent();

         System.out.println("=== Code to PDF Extractor ===");
         System.out.println("Current directory : " + currentDir);
         System.out.println("Parent  directory : " + (parentDir != null ? parentDir : "(none)"));
         System.out.println();

         // 1. Collect files

         // Parent directory (top-level files only, not recursive)
         if (parentDir != null && Files.exists(parentDir)) {
             collectFiles(parentDir, false, "Parent");
         }

         // Current directory (recursive — includes all child subdirectories)
         collectFiles(currentDir, true, "Current");

         System.out.printf("%nFound %d code file(s) with %,d total lines.%n",
                 fileEntries.size(), totalLines);

         if (fileEntries.isEmpty()) {
             System.out.println("No code files found. Exiting.");
             return;
         }

         // 2. Generate PDF
         generatePdf(currentDir);

         System.out.println("PDF saved to: " + Paths.get(OUTPUT_FILE).toAbsolutePath());
     }

     //  File Collection

     private void collectFiles(Path root, boolean recursive, String scope) throws IOException {
         if (!Files.exists(root)) return;

         Files.walkFileTree(root, new SimpleFileVisitor<>() {
             @Override
             public FileVisitResult preVisitDirectory(Path dir, BasicFileAttributes attrs) {
                 String name = dir.getFileName() == null ? "" : dir.getFileName().toString();
                 // Never recurse into skip-dirs; always allow the root itself
                 if (!dir.equals(root) && SKIP_DIRS.contains(name)) {
```

```
                    return FileVisitResult.SKIP_SUBTREE;
                }
                // If not recursive, skip any subdirectory of root
                if (!recursive && !dir.equals(root)) {
                    return FileVisitResult.SKIP_SUBTREE;
                }
                return FileVisitResult.CONTINUE;
            }

            @Override
            public FileVisitResult visitFile(Path file, BasicFileAttributes attrs) throws IOException {
                if (isCodeFile(file) && attrs.size() <= MAX_FILE_SIZE_BYTES) {
                    String content = Files.readString(file);
                    int lines = content.split("\n", -1).length;
                    String relativePath = scope + ": " + root.relativize(file);
                    fileEntries.add(new FileEntry(relativePath, file, content, lines));
                    totalLines += lines;
                    System.out.println("  + " + relativePath + "  (" + lines + " lines)");
                }
                return FileVisitResult.CONTINUE;
            }
        });
    }

    private boolean isCodeFile(Path file) {
        String name = file.getFileName().toString();
        // Check for exact filename match (e.g. "Makefile", "Dockerfile")
        if (CODE_EXTENSIONS.contains(name)) return true;
        // Check extension
        int dot = name.lastIndexOf('.');
        if (dot >= 0) {
            return CODE_EXTENSIONS.contains(name.substring(dot));
        }
        return false;
    }

    //  PDF Generation

    private void generatePdf(Path currentDir) throws IOException {
        PdfWriter writer   = new PdfWriter(OUTPUT_FILE);
        PdfDocument pdfDoc = new PdfDocument(writer);
        Document doc       = new Document(pdfDoc, PageSize.A4);
        doc.setMargins(40, 40, 40, 40);

        PdfFont sansFont  = PdfFontFactory.createFont(StandardFonts.HELVETICA);
        PdfFont boldFont  = PdfFontFactory.createFont(StandardFonts.HELVETICA_BOLD);
        PdfFont monoFont  = PdfFontFactory.createFont(StandardFonts.COURIER);

        // Cover page
        addCoverPage(doc, sansFont, boldFont, currentDir);
        doc.add(new AreaBreak(AreaBreakType.NEXT_PAGE));

        // Table of contents
        addTableOfContents(doc, sansFont, boldFont);
        doc.add(new AreaBreak(AreaBreakType.NEXT_PAGE));

        // Code sections
        for (int i = 0; i < fileEntries.size(); i++) {
            FileEntry entry = fileEntries.get(i);
            addCodeSection(doc, entry, i + 1, sansFont, boldFont, monoFont);
            if (i < fileEntries.size() - 1) {
                doc.add(new AreaBreak(AreaBreakType.NEXT_PAGE));
            }
        }

        doc.close();
    }

    private void addCoverPage(Document doc, PdfFont sansFont, PdfFont boldFont, Path dir)
            throws IOException {

        doc.add(new Paragraph("\n\n\n"));

        // Big title
        doc.add(new Paragraph("Code Export")
                .setFont(boldFont).setFontSize(FONT_TITLE + 10)
```

```
                .setFontColor(COLOR_FILE_TITLE)
                .setMarginBottom(6));

        doc.add(new Paragraph("Source Code PDF Report")
                .setFont(sansFont).setFontSize(FONT_SECTION)
                .setFontColor(ColorConstants.DARK_GRAY)
                .setMarginBottom(24));

        // Stats table
        Table stats = new Table(UnitValue.createPercentArray(new float[]{40, 60}))
                .setWidth(UnitValue.createPercentValue(80));
        addStatRow(stats, "Generated",    LocalDateTime.now()
                .format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm")), sansFont, boldFont);
        addStatRow(stats, "Root directory", dir.toString(), sansFont, boldFont);
        addStatRow(stats, "Files found",  String.valueOf(fileEntries.size()), sansFont, boldFont);
        addStatRow(stats, "Total lines",  String.format("%,d", totalLines), sansFont, boldFont);
        doc.add(stats);
    }

    private void addStatRow(Table t, String label, String value,
                            PdfFont sans, PdfFont bold) {
        t.addCell(new Cell().add(new Paragraph(label)
                        .setFont(bold).setFontSize(10)
                        .setFontColor(ColorConstants.DARK_GRAY))
                .setBorder(null).setPaddingBottom(6));
        t.addCell(new Cell().add(new Paragraph(value)
                        .setFont(sans).setFontSize(10))
                .setBorder(null).setPaddingBottom(6));
    }

    private void addTableOfContents(Document doc, PdfFont sansFont, PdfFont boldFont)
            throws IOException {
        doc.add(new Paragraph("Table of Contents")
                .setFont(boldFont).setFontSize(FONT_SECTION + 2)
                .setFontColor(COLOR_TOC_HEADER)
                .setMarginBottom(12));

        for (int i = 0; i < fileEntries.size(); i++) {
            FileEntry entry = fileEntries.get(i);
            String label = String.format("%d.  %s  (%,d lines)",
                    i + 1, entry.relativePath, entry.lineCount);
            doc.add(new Paragraph(label)
                    .setFont(sansFont).setFontSize(FONT_TOC)
                    .setMarginBottom(3));
        }
    }

    private void addCodeSection(Document doc, FileEntry entry, int index,
                                PdfFont sansFont, PdfFont boldFont, PdfFont monoFont)
            throws IOException {

        // Section header bar
        Table header = new Table(UnitValue.createPercentArray(new float[]{5, 75, 20}))
                .setWidth(UnitValue.createPercentValue(100))
                .setMarginBottom(8);

        header.addCell(new Cell().add(new Paragraph(String.valueOf(index))
                        .setFont(boldFont).setFontSize(FONT_FILE)
                        .setFontColor(COLOR_HEADER_TEXT))
                .setBackgroundColor(COLOR_HEADER_BG)
                .setBorder(null).setPadding(6));

        header.addCell(new Cell().add(new Paragraph(entry.relativePath)
                        .setFont(boldFont).setFontSize(FONT_FILE)
                        .setFontColor(COLOR_HEADER_TEXT))
                .setBackgroundColor(COLOR_HEADER_BG)
                .setBorder(null).setPadding(6));

        header.addCell(new Cell().add(new Paragraph(entry.lineCount + " lines")
                        .setFont(sansFont).setFontSize(FONT_FILE - 1)
                        .setFontColor(COLOR_HEADER_TEXT))
                .setBackgroundColor(COLOR_HEADER_BG)
                .setBorder(null).setPadding(6));

        doc.add(header);

```

```
        // Code block with line numbers
        String[] lines = entry.content.split("\n", -1);
        int padWidth = String.valueOf(lines.length).length();

        for (int ln = 0; ln < lines.length; ln++) {
            String lineNum = String.format("%" + padWidth + "d", ln + 1);
            String codeLine = lines[ln]
                    .replace("\t", "    ")   // expand tabs
                    .replace("\r", "");       // strip CR

            // Truncate very long lines
            if (codeLine.length() > 200) {
                codeLine = codeLine.substring(0, 197) + "...";
            }

            Text numText  = new Text(lineNum + "  ").setFont(monoFont)
                    .setFontSize(FONT_CODE).setFontColor(COLOR_LINE_NUM);
            Text codeText = new Text(codeLine).setFont(monoFont)
                    .setFontSize(FONT_CODE).setFontColor(ColorConstants.BLACK);

            Paragraph p = new Paragraph().add(numText).add(codeText)
                    .setBackgroundColor(ln % 2 == 0 ? COLOR_CODE_BG : ColorConstants.WHITE)
                    .setMargin(0).setPaddingLeft(4).setPaddingRight(4)
                    .setMultipliedLeading(1.3f)
                    .setFixedLeading(FONT_CODE + 2.5f);

            doc.add(p);
        }
    }

    //  Data class

    private static class FileEntry {
        final String relativePath;
        final Path   absolutePath;
        final String content;
        final int    lineCount;

        FileEntry(String relativePath, Path absolutePath, String content, int lineCount) {
            this.relativePath = relativePath;
            this.absolutePath = absolutePath;
            this.content      = content;
            this.lineCount    = lineCount;
        }
    }
}
```

5 Current: src\main\java\com\example\EmployeeManagementSystem\config\BasicAuthConfig.java

52 lines

```
package com.example.EmployeeManagementSystem.config;

import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.annotation.Order;
import org.springframework.http.HttpMethod;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.ProviderManager;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;


@Configuration
@EnableMethodSecurity
public class BasicAuthConfig {
//    @Bean
//    @Order(3)
//   public SecurityFilterChain basicAuth(HttpSecurity security){
//      security
//               .csrf(csrf-> csrf.disable())
//               .authorizeHttpRequests(
//                       auth->auth
//
.requestMatchers("/employee/register","/employee/register/manager").permitAll()
//                               .requestMatchers(HttpMethod.POST,"/vendors").permitAll()
//                               .anyRequest().authenticated()
//               )
//               .httpBasic(Customizer.withDefaults());
//       return security.build();
//   }

    @Bean
     public PasswordEncoder passwordEncoder(){
         return new BCryptPasswordEncoder();
    }

    @Bean
     public AuthenticationManager authenticationManager(@Qualifier("combinedUserDetailService")
UserDetailsService userDetailsService,
                                                        PasswordEncoder passwordEncoder){
        DaoAuthenticationProvider employeeAuthenticationProvider=new
DaoAuthenticationProvider(userDetailsService);
        employeeAuthenticationProvider.setPasswordEncoder(passwordEncoder);
        return new ProviderManager(employeeAuthenticationProvider);
    }

}

```

6 Current: src\main\java\com\example\EmployeeManagementSystem\config\JWTauth\JWTAuthConfig.java

68 lines

```java
// src/main/java/com/example/EmployeeManagementSystem/config/JWTAuth/JWTAuthConfig.java
package com.example.EmployeeManagementSystem.config.JWTauth;

import com.example.EmployeeManagementSystem.Filter.ApiKeyFilter;
import com.example.EmployeeManagementSystem.Filter.JwtAuthFilter;
import com.example.EmployeeManagementSystem.security.ApiKeyAuthenticationProvider;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Primary;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.ProviderManager;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.client.RestTemplate;

import java.util.List;

@Configuration
@EnableWebSecurity
public class JWTAuthConfig {

    private final JwtAuthFilter jwtAuthFilter;
    private final ApiKeyAuthenticationProvider apiKeyAuthProvider;

    public JWTAuthConfig(JwtAuthFilter jwtAuthFilter,
                         ApiKeyAuthenticationProvider apiKeyAuthProvider) {
        this.jwtAuthFilter = jwtAuthFilter;
        this.apiKeyAuthProvider = apiKeyAuthProvider;
    }

    @Bean
    public RestTemplate getRestTemplate(){
        return new RestTemplate();
    }

    @Bean
    public ApiKeyFilter apiKeyFilter() {
        return new ApiKeyFilter();
    }

    // DON'T create a new authenticationManager bean - use existing or rename
    // Instead, just add the provider to existing auth manager
    // Remove or comment out this bean if BasicAuthConfig already provides it

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
                .csrf(csrf -> csrf.disable())
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers("/Authenticate","/Authenticate/refresh","/session/**").permitAll()
                        .requestMatchers("/oauth2/**", "/login/oauth2/**").permitAll()
                        .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()
                        .requestMatchers("/google.html").permitAll()

.requestMatchers("/employee/register","/vendors/register","/employee/register/manager").permitAll()
                        .requestMatchers("/login.html", "/employee-dashboard.html", "/manager-
dashboard.html", "/vendor-dashboard.html").permitAll()
                        .requestMatchers("/auth/google/callback").permitAll()
                        .requestMatchers("/api-keys/generate").authenticated()  // Requires auth
                        .anyRequest().authenticated()
                )
                // Add API Key filter BEFORE JWT filter
                .addFilterBefore(apiKeyFilter(), UsernamePasswordAuthenticationFilter.class)
                .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

7 Current: src\main\java\com\example\EmployeeManagementSystem\config\Seassion\SessionAuthConfig.java

```java
package com.example.EmployeeManagementSystem.config.Seassion;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.annotation.Order;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.context.HttpSessionSecurityContextRepository;

 @Configuration
 public class SessionAuthConfig {

     @Bean
     @Order(1)
     public SecurityFilterChain sessionFilterChain(HttpSecurity http) throws Exception {
         http
                 .securityMatcher("/session/**", "/web/**")
                 .csrf(csrf -> csrf.disable())
                 .authorizeHttpRequests(auth -> auth
                         .requestMatchers("/session/login", "/session/logout").permitAll()
                         .anyRequest().authenticated()
                 )
                 .sessionManagement(session -> session
                         .maximumSessions(1)           // one session per user
                         .maxSessionsPreventsLogin(false) // new login kicks old session
                 )
                 .securityContext(ctx -> ctx
                         .securityContextRepository(new HttpSessionSecurityContextRepository())
                 );
         return http.build();
     }
 }
```

8 Current: src\main\java\com\example\EmployeeManagementSystem\config\swaggerConfig.java

38 lines

```
package com.example.EmployeeManagementSystem.config;

import io.swagger.v3.oas.models.Components;
import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.security.SecurityRequirement;
import io.swagger.v3.oas.models.security.SecurityScheme;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

 @Configuration
 public class swaggerConfig {
     @Bean
     public OpenAPI openAPI() {
         return new OpenAPI()
                 .components(new Components()
                         .addSecuritySchemes("bearerAuth",
                                 new SecurityScheme()
                                         .type(SecurityScheme.Type.HTTP)
                                         .scheme("bearer")
                                         .bearerFormat("JWT")
                         )
                 )
                 .addSecurityItem(new SecurityRequirement().addList("bearerAuth"));
     }
 //    @Bean
 //    public OpenAPI openAPI() {
 //        return new OpenAPI()
 //                .components(new Components()
 //                        .addSecuritySchemes("basicAuth",
 //                                new SecurityScheme()
 //                                        .type(SecurityScheme.Type.HTTP)
 //                                        .scheme("basic")
 //                        )
 //                )
 //                .addSecurityItem(new SecurityRequirement().addList("basicAuth"));
 //    }
 }

```

9 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\ApiKeyController.java

106 lines

```java
// src/main/java/com/example/EmployeeManagementSystem/Controller/ApiKeyController.java
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.Entity.ApiKey;
import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.Vendor;
import com.example.EmployeeManagementSystem.Repository.ApiKeyRepository;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.VendorRepo;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;

import java.time.Instant;
import java.time.temporal.ChronoUnit;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api-keys")
public class ApiKeyController {

    private final ApiKeyRepository apiKeyRepository;
    private final EmployeeRepo employeeRepo;
    private final VendorRepo vendorRepo;

    public ApiKeyController(ApiKeyRepository apiKeyRepository,
                            EmployeeRepo employeeRepo,
                            VendorRepo vendorRepo) {
        this.apiKeyRepository = apiKeyRepository;
        this.employeeRepo = employeeRepo;
        this.vendorRepo = vendorRepo;
    }

    @PostMapping("/generate")
    @PreAuthorize("hasAnyRole('EMPLOYEE', 'MANAGER', 'VENDOR')")
    public ResponseEntity<Map<String, Object>> generateApiKey(
            @RequestBody Map<String, String> request,
            Authentication authentication) {

        String keyName = request.get("keyName");
        String permissions = request.get("permissions");
        Integer expiryDays = request.get("expiryDays") != null ?
                Integer.parseInt(request.get("expiryDays")) : 30;

        ApiKey apiKey = new ApiKey();
        apiKey.setKeyName(keyName);
        apiKey.setPermissions(permissions);

        // Find the user
        Employee employee = employeeRepo.findByEmail(authentication.getName()).orElse(null);
        if (employee != null) {
            apiKey.setEmployee(employee);
        } else {
            Vendor vendor = vendorRepo.findByEmail(authentication.getName()).orElse(null);
            apiKey.setVendor(vendor);
        }

        apiKey.setExpiresAt(Instant.now().plus(expiryDays, ChronoUnit.DAYS));

        ApiKey saved = apiKeyRepository.save(apiKey);

        Map<String, Object> response = new HashMap<>();
        response.put("apiKey", saved.getApiKey());
        response.put("message", "API key generated successfully. Save this key - it won't be shown again!");
        response.put("expiresAt", saved.getExpiresAt());

        return ResponseEntity.ok(response);
    }
```

```java

    // In ApiKeyController.java - update these methods:

    @GetMapping
    @PreAuthorize("hasAnyRole('EMPLOYEE', 'MANAGER', 'VENDOR')")
    public ResponseEntity<List<ApiKey>> getMyApiKeys(Authentication authentication) {
        Employee employee = employeeRepo.findByEmail(authentication.getName()).orElse(null);
        if (employee != null) {
            // FIX: Use corrected method name
            return ResponseEntity.ok(apiKeyRepository.findByEmployee_EmployeeIdAndActiveTrue(employee.getEmployeeId()));
        }

        Vendor vendor = vendorRepo.findByEmail(authentication.getName()).orElse(null);
        if (vendor != null) {
            // FIX: Use corrected method name
            return ResponseEntity.ok(apiKeyRepository.findByVendor_IdAndActiveTrue(vendor.getId()));
        }

        return ResponseEntity.badRequest().build();
    }

    @DeleteMapping("/{id}")
    @PreAuthorize("hasAnyRole('EMPLOYEE', 'MANAGER', 'VENDOR')")
    public ResponseEntity<Map<String, String>> revokeApiKey(@PathVariable Long id) {
        ApiKey apiKey = apiKeyRepository.findById(id).orElse(null);
        if (apiKey != null) {
            apiKey.setActive(false);
            apiKeyRepository.save(apiKey);
         }

         Map<String, String> response = new HashMap<>();
         response.put("message", "API key revoked successfully");
         return ResponseEntity.ok(response);
     }
}
```

10 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\AuthController.java

110 lines

```java
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.DTO.AccessTokenRequest;
import com.example.EmployeeManagementSystem.DTO.AuthRequest;
import com.example.EmployeeManagementSystem.DTO.AuthResponse;
import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.RefreshToken;
import com.example.EmployeeManagementSystem.Entity.Vendor;
import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
import com.example.EmployeeManagementSystem.Service.RefreshTokenService;
import com.example.EmployeeManagementSystem.Util.JWTUtil;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.time.Instant;

@RestController
@RequestMapping("/Authenticate")
public class AuthController {

    private final AuthenticationManager authenticationManager;
    private final JWTUtil jwtUtil;
    private final RefreshTokenService refreshTokenService;
    private final RefreshTokenRepository refreshTokenRepository;

    public AuthController(AuthenticationManager authenticationManager, JWTUtil jwtUtil, RefreshTokenService refreshTokenService, RefreshTokenRepository refreshTokenRepository) {
        this.authenticationManager = authenticationManager;
        this.jwtUtil = jwtUtil;
        this.refreshTokenService = refreshTokenService;
        this.refreshTokenRepository = refreshTokenRepository;
    }

    @PostMapping
    public AuthResponse authenticateToken(@RequestBody AuthRequest authRequest) {
        try {
            System.out.println("1. Trying to authenticate user: " + authRequest.getUsername());
            System.out.println("2. Password length: " + authRequest.getPassword().length());

            Authentication auth = authenticationManager.authenticate(
                    new UsernamePasswordAuthenticationToken(
                            authRequest.getUsername(),
                            authRequest.getPassword()
                    )
            );

            System.out.println("3. Authentication successful! User: " + auth.getName());
            System.out.println("4. Authorities: " + auth.getAuthorities());

            UserDetails user = (UserDetails) auth.getPrincipal();

            String accessToken = jwtUtil.generateToken(user.getUsername());

            RefreshToken refreshToken;

            if (user instanceof Employee employee) {
                refreshToken = refreshTokenService.createForEmployee(employee);
            } else if (user instanceof Vendor vendor) {
                refreshToken = refreshTokenService.createForVendor(vendor);
            } else {
                throw new RuntimeException("Unknown user type");
            }

            refreshTokenRepository.save(refreshToken);

```

```
            String token = jwtUtil.generateToken(authRequest.getUsername());
            System.out.println("5. Token generated: " + token);

            return new AuthResponse(accessToken, refreshToken.getToken());
        } catch (Exception e) {
            System.out.println("ERROR: " + e.getClass().getSimpleName());
            System.out.println("Message: " + e.getMessage());
            e.printStackTrace();  // This will show full stack trace
            throw new RuntimeException(e);
        }


    }

    @PostMapping("/refresh")
    public AuthResponse refresh(@RequestBody AccessTokenRequest request) {
        RefreshToken storedToken = refreshTokenRepository.findByToken(request.getRefreshToken())
                .orElseThrow(() -> new RuntimeException("Invalid refresh token"));

        if (storedToken.getExpiryTime().isBefore(Instant.now())) {
            refreshTokenRepository.delete(storedToken);
            throw new RuntimeException("Refresh token expired");
        }

        String username;

        if (storedToken.getEmployee() != null) {
            username = storedToken.getEmployee().getUsername();
        } else if (storedToken.getVendor() != null) {
             username = storedToken.getVendor().getUsername();
         } else {
             throw new RuntimeException("Refresh token has no owner");
         }

         String newAccessToken = jwtUtil.generateToken(username);

         return new AuthResponse(newAccessToken, storedToken.getToken());
     }
}

```

11 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\DeliveryController.java

33 lines

```java
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.Entity.Delivery;
import com.example.EmployeeManagementSystem.Enum.DeliveryStatus;
import com.example.EmployeeManagementSystem.Service.DeliveryService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

 @RestController
 @RequestMapping("/delivery")
 public class DeliveryController {
     private final DeliveryService deliveryService;

     public DeliveryController(DeliveryService deliveryService) {
         this.deliveryService = deliveryService;
     }

     @PutMapping("/{id}/status")
     public ResponseEntity<Delivery> updateDeliveryStatus(
             @PathVariable Long id,
             @RequestParam DeliveryStatus status) {
         return ResponseEntity.ok(deliveryService.updateDeliveryStatus(id, status));
     }

     @GetMapping("/subscription/{subscriptionId}")
     public ResponseEntity<List<Delivery>> getDeliveriesBySubscription(
             @PathVariable Long subscriptionId) {
         return ResponseEntity.ok(deliveryService.getDeliveriesBySubscription(subscriptionId));
     }
 }

```

12 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\EmployeeController.java

65 lines

```
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.DTO.EmployeeDTO;
import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Service.EmployeeService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;
@RestController
@RequestMapping("/employee")
public class EmployeeController {
    private final EmployeeService employeeService;

    public EmployeeController(EmployeeService employeeService) {
        this.employeeService = employeeService;
    }

    @GetMapping
    public ResponseEntity<?> getAllEmployees(){
        return ResponseEntity.ok(employeeService.getAllEmployees());
    }

    @GetMapping("/me")
    @PreAuthorize("hasAnyRole('MANAGER','EMPLOYEE')")
    public ResponseEntity<Employee> myAccount(Authentication authentication){
        return ResponseEntity.ok(employeeService.getAccount(authentication));
    }

    @PostMapping("/register")
    public ResponseEntity<Employee> addEmployee(@RequestBody EmployeeDTO employee){
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(employeeService.createEmployee(employee));

    }

    @PostMapping("/register/manager")
    public ResponseEntity<Employee> addManager(@RequestBody EmployeeDTO employee){
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(employeeService.createManager(employee));

    }



    @PutMapping("/{id}")
    public ResponseEntity<Employee> updateEmployee(@PathVariable long id,@RequestBody EmployeeDTO employeeDTO){
        return ResponseEntity.ok(employeeService.updateEmployee(id,employeeDTO));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<?> deleteEmployee(@PathVariable long id){
        employeeService.deleteEmployee(id);
        return ResponseEntity.ok("Employee with id: "+id+" is deleted");
    }

    //should be done by manager
    @PutMapping("/inactive")
    @PreAuthorize("hasRole('MANAGER')")
    public ResponseEntity<String> inactivateEmployee(@RequestParam long employeeId) {
        return employeeService.inactivateUser(employeeId);
    }
}

```

13 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\GlobalExceptionHandler.java

```java
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.Exception.*;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.context.request.WebRequest;

 import java.time.Instant;
 import java.util.LinkedHashMap;
 import java.util.Map;

 @RestControllerAdvice
 public class GlobalExceptionHandler {

     private Map<String, Object> buildError(HttpStatus status, String message, WebRequest request) {
         Map<String, Object> body = new LinkedHashMap<>();
         body.put("timestamp", Instant.now());
         body.put("status", status.value());
         body.put("error", status.getReasonPhrase());
         body.put("message", message);
         body.put("path", request.getDescription(false).replace("uri=", ""));
         return body;
     }

     @ExceptionHandler(EmployeeNotFound.class)
     public ResponseEntity<Map<String, Object>> handleEmployeeNotFound(
             EmployeeNotFound ex, WebRequest request) {
         return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND, ex.getMessage(), request),
                 HttpStatus.NOT_FOUND);
     }

     @ExceptionHandler(InvalidStartDateException.class)
     public ResponseEntity<Map<String, Object>> handleInvalidStartDate(
             InvalidStartDateException ex, WebRequest request) {
         return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
                 HttpStatus.BAD_REQUEST);
     }

     @ExceptionHandler(InvalidEndDateException.class)
     public ResponseEntity<Map<String, Object>> handleInvalidEndDate(
             InvalidEndDateException ex, WebRequest request) {
         return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
                 HttpStatus.BAD_REQUEST);
     }

     @ExceptionHandler(OverlappingLeaveException.class)
     public ResponseEntity<Map<String, Object>> handleOverlappingLeave(
             OverlappingLeaveException ex, WebRequest request) {
         return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
                 HttpStatus.BAD_REQUEST);
     }

     @ExceptionHandler(DuplicateRequestException.class)
     public ResponseEntity<Map<String, Object>> handleDuplicateRequest(
             DuplicateRequestException ex, WebRequest request) {
         return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
                 HttpStatus.BAD_REQUEST);
     }

     @ExceptionHandler(LeaveRequestNotFoundException.class)
     public ResponseEntity<Map<String, Object>> handleLeaveNotFound(
             LeaveRequestNotFoundException ex, WebRequest request) {
         return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND, ex.getMessage(), request),
                 HttpStatus.NOT_FOUND);
     }

     @ExceptionHandler(InvalidManagerException.class)
     public ResponseEntity<Map<String, Object>> handleInvalidManager(
```

```
            InvalidManagerException ex, WebRequest request) {
        return new ResponseEntity<>(buildError(HttpStatus.FORBIDDEN, ex.getMessage(), request),
                HttpStatus.FORBIDDEN);
    }

    @ExceptionHandler(SubscriptionAlreadyExists.class)
    public ResponseEntity<Map<String, Object>> handleSubscriptionExists(
            SubscriptionAlreadyExists ex, WebRequest request) {
        return new ResponseEntity<>(buildError(HttpStatus.CONFLICT, ex.getMessage(), request),
                HttpStatus.CONFLICT);
    }


    @ExceptionHandler(VendorNotFoundException.class)
    public ResponseEntity<Map<String, Object>> handleVendorNotFound(
            VendorNotFoundException ex, WebRequest request) {
        return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND, ex.getMessage(), request),
                HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(RestaurantNotFoundException.class)
    public ResponseEntity<Map<String, Object>> handleRestaurantNotFound(
            RestaurantNotFoundException ex, WebRequest request) {
        return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND, ex.getMessage(), request),
                HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(UnauthorizedAccessException.class)
     public ResponseEntity<Map<String, Object>> handleUnauthorized(
             UnauthorizedAccessException ex, WebRequest request) {
         return new ResponseEntity<>(buildError(HttpStatus.FORBIDDEN, ex.getMessage(), request),
                 HttpStatus.FORBIDDEN);
     }

     @ExceptionHandler(IllegalArgumentException.class)
     public ResponseEntity<Map<String, Object>> handleIllegalArgument(
             IllegalArgumentException ex, WebRequest request) {
         return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
                 HttpStatus.BAD_REQUEST);
     }

     @ExceptionHandler(DataIntegrityViolationException.class)
     public ResponseEntity<Map<String, Object>> handleDataIntegrity(
             DataIntegrityViolationException ex, WebRequest request) {
         return new ResponseEntity<>(
                 buildError(HttpStatus.CONFLICT, "Cannot delete: related records exist", request),
                 HttpStatus.CONFLICT);
     }
}
```

14 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\LeaveRequestController.java

```java
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.DTO.ActionDTO;
import com.example.EmployeeManagementSystem.DTO.LeaveRequestDTO;
import com.example.EmployeeManagementSystem.DTO.LeaveResponseDTO;
import com.example.EmployeeManagementSystem.Service.LeaveRequestService;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("leave_requests")
public class LeaveRequestController {
    private final LeaveRequestService leaveRequestService;

    public LeaveRequestController(LeaveRequestService leaveRequestService) {
        this.leaveRequestService = leaveRequestService;
    }

    @GetMapping
    @PreAuthorize("hasRole('MANAGER')")
    public ResponseEntity<List<LeaveResponseDTO>> getAllLeaveRequest() {
        return ResponseEntity.ok(leaveRequestService.getAllTheLeaveRequest());
    }

    @PostMapping
    @PreAuthorize("hasAnyRole('MANAGER','EMPLOYEE')")
    public ResponseEntity<?> addLeaveRequest(Authentication authentication,@RequestBody LeaveRequestDTO dto){
        return leaveRequestService.createRequest(authentication,dto);
    }

    @GetMapping("/pending")
    @PreAuthorize("hasRole('MANAGER')")
    public ResponseEntity<List<LeaveResponseDTO>> getAllTheLeaveRequests() {
        return ResponseEntity.ok(leaveRequestService.getAllThePendingLeaveRequests());
    }

    @PutMapping("/approval")
    @PreAuthorize("hasRole('MANAGER')")
    public ResponseEntity<?> updateLeaveRequestStatus(@RequestBody ActionDTO actionDTO,Authentication authentication){
      return leaveRequestService.updateLeaveRequestStatus(actionDTO,authentication);
    }

    @PutMapping("/cancel/{leaveId}")
    @PreAuthorize("hasAuthority('LEAVE_CANCEL')")
    public ResponseEntity<?> cancelLeaveRequest(Authentication authentication,@PathVariable long leaveId){
      return leaveRequestService.cancelLeaveRequest(authentication, leaveId);
    }

    @GetMapping("/employee")
    @PreAuthorize("hasAnyRole('MANAGER','EMPLOYEE')")
    public ResponseEntity<?> getEmployeeLeaves(Authentication authentication) {
        return leaveRequestService.getLeaveRequestsByEmployee(authentication);
    }

}

```

15 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\LoginController.java

17 lines

```
package com.example.EmployeeManagementSystem.Controller;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

@RestController
 public class LoginController {

     @GetMapping("/login")
     public ResponseEntity<?> login() {
         return ResponseEntity.ok().body(Map.of("message", "Please use POST to /api/auth/login with credentials"));
     }
 }

```

16 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\OauthController.java

168 lines

```java
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.DTO.AuthResponse;
import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.RefreshToken;
import com.example.EmployeeManagementSystem.Entity.Vendor;
import com.example.EmployeeManagementSystem.Enum.Role;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
import com.example.EmployeeManagementSystem.Repository.VendorRepo;
import com.example.EmployeeManagementSystem.Service.CombinedUserDetailService;
import com.example.EmployeeManagementSystem.Service.RefreshTokenService;
import com.example.EmployeeManagementSystem.Util.JWTUtil;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.http.*;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.util.LinkedMultiValueMap;
import org.springframework.util.MultiValueMap;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.client.RestTemplate;

import java.util.Collections;
import java.util.Map;
import java.util.Optional;
import java.util.UUID;

@RestController
@RequestMapping("/auth/google")
public class OauthController {
    private final EmployeeRepo employeeRepo;
    private final RefreshTokenService refreshTokenService;
    private final RefreshTokenRepository refreshTokenRepository;
    public OauthController(EmployeeRepo employeeRepo, RefreshTokenService refreshTokenService, RefreshTokenRepository refreshTokenRepository) {
        this.employeeRepo = employeeRepo;
        this.refreshTokenService = refreshTokenService;
        this.refreshTokenRepository = refreshTokenRepository;
    }

    @Value("${spring.security.oauth2.client.registration.google.client-id}")
    private String client_id;
    @Value("${spring.security.oauth2.client.registration.google.client-secret}")
    private String client_secret;

    @Autowired
    private VendorRepo vendorRepo;

    @Autowired
    private RestTemplate restTemplate;

    @Autowired
    CombinedUserDetailService combinedUserDetailService;

    @Autowired
    private PasswordEncoder passwordEncoder;

    @Autowired
    private JWTUtil jwtUtil;


    @GetMapping("/callback")
    public ResponseEntity<?> handelGoogleCallback(@RequestParam String code,
                                                  @RequestParam(required = false) String role,
                                                  @RequestParam(required = false, defaultValue = "Asia/Kolkata") String timezone){
```

```
        try {
            System.out.println("CODE RECEIVED: " + code);
            System.out.println("STATE RECEIVED: " + role);
            // Parse role from state parameter (sent from frontend)
            Role userRole = Role.EMPLOYEE; // default role
            if(role != null && !role.isEmpty()) {
                try {
                    userRole = Role.valueOf(role.toUpperCase());
                } catch (IllegalArgumentException e) {
                    System.out.println("Invalid role: " + role);
                }
            }
            System.out.println("USER ROLE: " + userRole);

            String tokenEndpoint="https://oauth2.googleapis.com/token";
            MultiValueMap<String,String> params=new LinkedMultiValueMap<>();
            params.add("code",code);
            params.add("client_id",client_id);
            params.add("client_secret",client_secret);
            params.add("redirect_uri","http://localhost:8080/google.html");
            params.add("grant_type","authorization_code");

            HttpHeaders headers=new HttpHeaders();
            headers.setContentType(MediaType.APPLICATION_FORM_URLENCODED);
            HttpEntity<MultiValueMap<String,String>> request=new HttpEntity<>(params,headers);
            ResponseEntity<Map> tokenResponse=restTemplate.postForEntity(tokenEndpoint,request,Map.class);
            String idToken=(String) tokenResponse.getBody().get("id_token");
            String UserInfoUrl="https://oauth2.googleapis.com/tokeninfo?id_token="+idToken;
            ResponseEntity<Map> userInfoResponse=restTemplate.getForEntity(UserInfoUrl, Map.class);

             if(userInfoResponse.getStatusCode()== HttpStatus.OK){
                Map<String,Object> userInfo=userInfoResponse.getBody();
                String email=(String) userInfo.get("email");
                UserDetails userDetails=null;

                try{
                    userDetails=combinedUserDetailService.loadUserByUsername(email);
                } catch (UsernameNotFoundException e) {
                    if (userRole == Role.VENDOR) {

                        Vendor vendor = new Vendor();
                        vendor.setEmail(email);
                        vendor.setName((String) userInfo.get("name"));
                        vendor.setPassword(passwordEncoder.encode(UUID.randomUUID().toString()));

                        vendorRepo.save(vendor);

                    } else if (userRole == Role.MANAGER) {

                        Employee manager = new Employee();
                        manager.setEmail(email);
                        manager.setName((String) userInfo.get("name"));
                        manager.setPassword(passwordEncoder.encode(UUID.randomUUID().toString()));
                        manager.setRole(Role.MANAGER);
                        manager.setTimezone(timezone);
                        manager.setDept("UNASSIGNED");
                        employeeRepo.save(manager);

                    } else { // EMPLOYEE

                        Employee employee = new Employee();
                        employee.setEmail(email);
                        employee.setName((String) userInfo.get("name"));
                        employee.setPassword(passwordEncoder.encode(UUID.randomUUID().toString()));
                        employee.setRole(Role.EMPLOYEE);
                        employee.setDept("UNASSIGNED");
                        employee.setTimezone(timezone);
                        employeeRepo.save(employee);
                    }

                    userDetails = combinedUserDetailService.loadUserByUsername(email);
                }
                String accessToken= jwtUtil.generateToken(email);
                RefreshToken refreshToken;

                if (userDetails instanceof Employee employee) {
```

```
                    refreshToken = refreshTokenService.createForEmployee(employee);
                } else if (userDetails instanceof Vendor vendor) {
                    refreshToken = refreshTokenService.createForVendor(vendor);
                } else {
                    throw new RuntimeException("Unknown user type");
                }

                refreshTokenRepository.save(refreshToken);

                AuthResponse response = new AuthResponse(accessToken, refreshToken.getToken());

                return ResponseEntity.ok(response);
            }
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).build();

        } catch (Exception e) {
            e.printStackTrace();
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
        }

    }
}

```

17 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\RestaurantController.java

121 lines

```java
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.DTO.RestaurantDTO;
import com.example.EmployeeManagementSystem.DTO.RestaurantRequest;
import com.example.EmployeeManagementSystem.Enum.MealSlot;
import com.example.EmployeeManagementSystem.Service.RestaurantService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;

import java.util.List;

/**
 * Restaurant management endpoints.
 *
 *  VENDOR-ONLY (write operations)
 * POST   /restaurants                        — Create a restaurant (vendor supplies vendorId in body)
 * PUT    /restaurants/{id}                   — Update restaurant details (vendor ownership verified)
 * PUT    /restaurants/{id}/deactivate        — Soft-delete a restaurant  (vendor ownership verified)
 * GET    /restaurants/vendor/{vendorId}      — List MY restaurants (vendor views their own)
 *
 *  EMPLOYEE / MANAGER (read-only browse)
 * GET    /restaurants                        — Browse all active restaurants
 * GET    /restaurants/{id}                   — Get a specific restaurant
 * GET    /restaurants/by-slot?slot=LUNCH     — Find restaurants that serve a given meal slot
 *
 * Employees use the browse endpoints to pick a restaurant when creating a subscription.
 * Employees and managers cannot create, update, or deactivate restaurants.
 */
@RestController
@RequestMapping("/restaurants")
public class RestaurantController {

    private final RestaurantService restaurantService;

    public RestaurantController(RestaurantService restaurantService) {
        this.restaurantService = restaurantService;
    }

    //  VENDOR-ONLY write operations

    /**
     * Create a new restaurant.
     * Request body must include vendorId — the service validates the vendor exists.
     */
    @PostMapping
    @PreAuthorize("hasRole('VENDOR')")
    public ResponseEntity<RestaurantDTO> createRestaurant(@RequestBody RestaurantRequest
request,Authentication authentication) {
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(restaurantService.createRestaurant(request,authentication));
    }

    /**
     * Update a restaurant's name, address, description, or supported meal slots.
     * vendorId in the request body is used to verify ownership before any change is made.
     */
    @PutMapping("/{id}")
    @PreAuthorize("hasRole('VENDOR')")
    public ResponseEntity<RestaurantDTO> updateRestaurant(@PathVariable Long id,
                                                          @RequestBody RestaurantRequest request,
                                                          Authentication authentication) {
        return ResponseEntity.ok(restaurantService.updateRestaurant(id, request,authentication));
    }

    /**
     * Soft-deactivate a restaurant. Active subscriptions remain in the DB but
     * new deliveries will stop since the restaurant is inactive.
     * Pass vendorId as a query param so ownership can be verified.
```

```
     */
    @PutMapping("/{id}/deactivate")
    @PreAuthorize("hasRole('VENDOR')")
    public ResponseEntity<String> deactivateRestaurant(@PathVariable Long id,
                                                       Authentication authentication) {
        restaurantService.deactivateRestaurant(id, authentication);
        return ResponseEntity.ok("Restaurant " + id + " has been deactivated.");
    }

    /**
     * List all active restaurants owned by a specific vendor.
     * Only the vendor themselves should call this to see their portfolio.
     */
    @GetMapping("/vendor")
    @PreAuthorize("hasRole('VENDOR')")
    public ResponseEntity<List<RestaurantDTO>> getRestaurantsByVendor(Authentication authentication) {
        return ResponseEntity.ok(restaurantService.getRestaurantsByVendor(authentication));
    }

    //  EMPLOYEE / MANAGER read-only browse

    /**
     * Browse all active restaurants.
     * Employees use this to discover available restaurants before subscribing.
     */
    @GetMapping
    @PreAuthorize("hasAuthority('RESTAURANTS_READ')")
    public ResponseEntity<List<RestaurantDTO>> getAllActiveRestaurants() {
        return ResponseEntity.ok(restaurantService.getAllActiveRestaurants());
    }

    /**
     * Get a specific restaurant by id.
     */
    @GetMapping("/{id}")
    @PreAuthorize("hasAuthority('RESTAURANTS_READ')")
    public ResponseEntity<RestaurantDTO> getRestaurant(@PathVariable Long id) {
        return ResponseEntity.ok(restaurantService.getRestaurant(id));
    }

    /**
     * Find all active restaurants that serve a specific meal slot.
     * Example: GET /restaurants/by-slot?slot=LUNCH
     * Employees call this when building their subscription to see valid options for each slot.
     */
    @GetMapping("/by-slot")
    @PreAuthorize("hasAnyRole('VENDOR','EMPLOYEE','MANAGER')")
    public ResponseEntity<List<RestaurantDTO>> getRestaurantsByMealSlot(@RequestParam MealSlot slot) {
        return ResponseEntity.ok(restaurantService.getRestaurantsByMealSlot(slot));
    }
}
```

18 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\SchedulerController.java

50 lines

```java
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.Scheduler.LeaveStatusScheduler;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Map;

 /**
  * Controller for Scheduler Operations
  */
 @RestController
 @RequestMapping("/scheduler")
 public class SchedulerController {

     private static final Logger log = LoggerFactory.getLogger(SchedulerController.class);

     private final LeaveStatusScheduler leaveStatusScheduler;

     public SchedulerController(LeaveStatusScheduler leaveStatusScheduler) {
         this.leaveStatusScheduler = leaveStatusScheduler;
     }

     /**
      * Manual trigger for leave scheduler
      */
     @PostMapping("/run")
     public ResponseEntity<?> runScheduler() {
         try {
             leaveStatusScheduler.runManually();

             return ResponseEntity.ok().body(
                     Map.of(
                             "status", "success",
                             "message", "Scheduler executed successfully"
                     )
             );

         } catch (Exception e) {
             return ResponseEntity.internalServerError().body(
                     Map.of(
                             "status", "error",
                             "message", e.getMessage()
                     )
             );
         }
     }
 }
```

19 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\SessionAuthController.java

8 li

```java
package com.example.EmployeeManagementSystem.Controller;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
import org.springframework.http.ResponseEntity;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContext;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.web.context.HttpSessionSecurityContextRepository;
import org.springframework.security.web.context.SecurityContextRepository;
import org.springframework.web.bind.annotation.*;

import java.util.Map;

@RestController
@RequestMapping("/session")
public class SessionAuthController {

    private final AuthenticationManager authenticationManager;
    private final SecurityContextRepository securityContextRepository = new HttpSessionSecurityContextRepository();

    public SessionAuthController(AuthenticationManager authenticationManager) {
        this.authenticationManager = authenticationManager;
    }

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody Map<String, String> body,
                                   HttpServletRequest request,
                                   HttpServletResponse response) {
        String username = body.get("username");
        String password = body.get("password");

        try {
            Authentication auth = authenticationManager.authenticate(
                    new UsernamePasswordAuthenticationToken(username, password)
            );

            // Create and set security context
            SecurityContext context = SecurityContextHolder.createEmptyContext();
            context.setAuthentication(auth);
            SecurityContextHolder.setContext(context);

            // CRITICAL: Save the context to the HTTP session
            HttpSession session = request.getSession(true);
            securityContextRepository.saveContext(context, request, response);

            // Alternative: Manual session attribute (redundant but safe)
            session.setAttribute(HttpSessionSecurityContextRepository.SPRING_SECURITY_CONTEXT_KEY, context);

            return ResponseEntity.ok(Map.of(
                    "message", "Login successful",
                    "username", auth.getName(),
                    "sessionId", session.getId(),
                    "roles", auth.getAuthorities().toString()
            ));

        } catch (Exception e) {
            return ResponseEntity.status(401).body(Map.of("error", "Invalid credentials: " + e.getMessage()));
        }
    }

    @PostMapping("/logout")
    public ResponseEntity<?> logout(HttpServletRequest request) {
        HttpSession session = request.getSession(false);
        if (session != null) {
            session.invalidate();
```

```
        }
        SecurityContextHolder.clearContext();
        return ResponseEntity.ok(Map.of("message", "Logged out successfully"));
    }

    @GetMapping("/me")
    public ResponseEntity<?> me(Authentication authentication) {
        if (authentication == null || !authentication.isAuthenticated()) {
            return ResponseEntity.status(401).body(Map.of("error", "Not logged in"));
        }
        return ResponseEntity.ok(Map.of(
                "username", authentication.getName(),
                "roles", authentication.getAuthorities().toString(),
                "authenticated", true
        ));
    }
}
```

20 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\SubscriptionController.java

7 li

```java
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.DTO.SubscriptionDTO;
import com.example.EmployeeManagementSystem.DTO.SubscriptionRequest;
import com.example.EmployeeManagementSystem.Entity.Subscription;
import com.example.EmployeeManagementSystem.Service.SubscriptionService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/subscription")
public class SubscriptionController {
    private final SubscriptionService subscriptionService;

    public SubscriptionController(SubscriptionService subscriptionService) {
        this.subscriptionService = subscriptionService;
    }

    //Authentication makes sure that whoever is logged can add subscription and cannot add other subscription
    @PostMapping
    @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER')")
    public ResponseEntity<?> addSubscription(@RequestBody SubscriptionRequest request,Authentication authentication) {
        subscriptionService.addSubscription(request,authentication);
        return ResponseEntity.status(HttpStatus.CREATED)
                .body("Added new subscription for user: " + authentication.getName());
    }

    @GetMapping
    @PreAuthorize("hasAuthority('SUBSCRIPTION_READ')")
    public List<SubscriptionDTO> getAllSubscriptions() {
        return subscriptionService.getAllSubscriptions();
    }


    @PutMapping("/{id}")
    @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER')")
    public ResponseEntity<Subscription> updateSubscription(
            @PathVariable Long id,
            @RequestBody SubscriptionRequest request) {
        return ResponseEntity.ok(subscriptionService.updateSubscription(id, request));
    }

    @PutMapping("/{id}/pause")
    @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER')")
    public ResponseEntity<Subscription> pauseSubscription(@PathVariable Long id) {
        return ResponseEntity.ok(subscriptionService.pauseSubscription(id));
    }

    @PutMapping("/{id}/resume")
    @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER')")
    public ResponseEntity<Subscription> resumeSubscription(@PathVariable Long id) {
        return ResponseEntity.ok(subscriptionService.resumeSubscription(id));
    }

    @PutMapping("/{id}/expire")
    @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER')")
    public ResponseEntity<Subscription> expireSubscription(@PathVariable Long id) {
        return ResponseEntity.ok(subscriptionService.expireSubscription(id));
    }

    @GetMapping("/user")
    @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER')")
    public ResponseEntity<List<SubscriptionDTO>> getSubscriptionOfUser(Authentication authentication){
        return ResponseEntity.ok(subscriptionService.getSubscriptionOfUser(authentication));
```

```
    }

}

```

21 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\TotpController.java

101 lines

```java
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Service.TotpService;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;

import java.util.Map;

@RestController
@RequestMapping("/totp")
public class TotpController {

    private final TotpService totpService;
    private final EmployeeRepo employeeRepo;

    public TotpController(TotpService totpService, EmployeeRepo employeeRepo) {
        this.totpService = totpService;
        this.employeeRepo = employeeRepo;
    }

    /**
     * Step 1 — Employee calls this to get their QR code URI.
     * They scan it with Google Authenticator / Authy.
     */
    @PostMapping("/setup")
    public ResponseEntity<?> setup(Authentication authentication) {
        Employee employee = employeeRepo.findByName(authentication.getName())
                .orElseThrow(() -> new RuntimeException("Employee not found"));

        String secret = totpService.generateSecret();
        employee.setTotpSecret(secret);
        employee.setTotpEnabled(false); // not enabled until verified
        employeeRepo.save(employee);

        String qrUri = totpService.getQrCodeUri(secret, employee.getName());

        return ResponseEntity.ok(Map.of(
                "secret", secret,          // show this as backup code
                "qrCodeUri", qrUri         // embed this in a QR image on frontend
        ));
    }

    /**
     * Step 2 — Employee scans QR, types their first code to confirm setup.
     * Only after this is totpEnabled = true.
     */
    @PostMapping("/verify-setup")
    public ResponseEntity<?> verifySetup(@RequestBody Map<String, String> body,
                                         Authentication authentication) {
        Employee employee = employeeRepo.findByName(authentication.getName())
                .orElseThrow(() -> new RuntimeException("Employee not found"));

        String code = body.get("code");
        if (!totpService.verifyCode(employee.getTotpSecret(), code)) {
            return ResponseEntity.badRequest().body(Map.of("error", "Invalid code. Try again."));
        }

        employee.setTotpEnabled(true);
        employeeRepo.save(employee);

        return ResponseEntity.ok(Map.of("message", "TOTP 2FA enabled successfully"));
    }

    /**
     * Step 3 — On every login, if totpEnabled=true, call this after Basic Auth succeeds.
     * Returns 200 if code is valid (you can then issue a session or JWT).
     */
    @PostMapping("/validate")
```

```
    public ResponseEntity<?> validate(@RequestBody Map<String, String> body,
                                      Authentication authentication) {
        Employee employee = employeeRepo.findByName(authentication.getName())
                .orElseThrow(() -> new RuntimeException("Employee not found"));

        if (!employee.isTotpEnabled()) {
            return ResponseEntity.ok(Map.of("message", "TOTP not enabled for this user"));
        }

        String code = body.get("code");
        if (!totpService.verifyCode(employee.getTotpSecret(), code)) {
            return ResponseEntity.status(401).body(Map.of("error", "Invalid TOTP code"));
        }

        return ResponseEntity.ok(Map.of("message", "TOTP verified", "authenticated", true));
    }

    /** Disable TOTP for the logged-in employee */
    @DeleteMapping("/disable")
    public ResponseEntity<?> disable(Authentication authentication) {
        Employee employee = employeeRepo.findByName(authentication.getName())
                .orElseThrow(() -> new RuntimeException("Employee not found"));

        employee.setTotpSecret(null);
        employee.setTotpEnabled(false);
        employeeRepo.save(employee);

        return ResponseEntity.ok(Map.of("message", "TOTP disabled"));
     }
}
```

22 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\VendorController.java

67 lines

```java
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.DTO.VendorDTO;
import com.example.EmployeeManagementSystem.DTO.VendorRequest;
import com.example.EmployeeManagementSystem.Service.DeliveryService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;

import java.util.List;

/**
 * Vendor management endpoints.
 *
 * POST   /vendors              — Register a new vendor (public / admin)
 * GET    /vendors              — List all vendors (admin)
 * GET    /vendors/{id}         — Get a specific vendor
 * PUT    /vendors/{id}         — Update vendor details (vendor-only)
 * DELETE /vendors/{id}         — Remove vendor (admin)
 *
 * NOTE: Until Spring Security is wired in, role checks are done manually in the
 *       service layer. Employees and managers have NO access to /vendors/** write operations
 *       — the service will throw UnauthorizedAccessException if called with wrong role context.
 */
@RestController
@RequestMapping("/vendors")
public class VendorController {

    private final DeliveryService.VendorService vendorService;

    public VendorController(DeliveryService.VendorService vendorService) {
        this.vendorService = vendorService;
    }

    @PostMapping("/register")
    public ResponseEntity<VendorDTO> registerVendor(@RequestBody VendorRequest request) {
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(vendorService.registerVendor(request));
    }

    @GetMapping
    @PreAuthorize("hasRole('VENDOR')")
    public ResponseEntity<List<VendorDTO>> getAllVendors() {
        return ResponseEntity.ok(vendorService.getAllVendors());
    }

    @GetMapping("/me")
    @PreAuthorize("hasRole('VENDOR')")
    public ResponseEntity<VendorDTO> getVendor(Authentication authentication) {
        return ResponseEntity.ok(vendorService.getVendor(authentication));
    }

    @PutMapping("/update")
    @PreAuthorize("hasRole('VENDOR')")
    public ResponseEntity<VendorDTO> updateVendor(@RequestBody VendorRequest request,Authentication authentication) {
        return ResponseEntity.ok(vendorService.updateVendor(request,authentication));
    }

    @DeleteMapping
    @PreAuthorize("hasRole('VENDOR')")
    public ResponseEntity<String> deleteVendor(Authentication authentication) {
        vendorService.deleteVendor(authentication);
        return ResponseEntity.ok("Vendor with email " + authentication.getName() + " has been removed.");
    }
}
```

23 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AccessTokenRequest.java

18 lines

```java
package com.example.EmployeeManagementSystem.DTO;

public class AccessTokenRequest {
    private String refreshToken;

    public AccessTokenRequest(String refreshToken) {
        this.refreshToken = refreshToken;
    }

     public void setRefreshToken(String refreshToken) {
         this.refreshToken = refreshToken;
     }

     public String getRefreshToken() {
         return refreshToken;
     }
 }

```

```java
package com.example.EmployeeManagementSystem.DTO;


public class ActionDTO {
    private long leaveRequestId;
    private String action;
    private String remarks;

    public long getLeaveRequestId() {
        return leaveRequestId;
     }

     public void setLeaveRequestId(long leaveRequestId) {
        this.leaveRequestId = leaveRequestId;
     }


     public String getAction() {
        return action;
     }

     public void setAction(String action) {
        this.action = action;
     }

     public String getRemarks() {
        return remarks;
     }

     public void setRemarks(String remarks) {
        this.remarks = remarks;
     }
 }

```

25 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AuthRequest.java

28 lines

```
package com.example.EmployeeManagementSystem.DTO;


import com.fasterxml.jackson.annotation.JsonProperty;

public class AuthRequest {
    @JsonProperty("username")
    private String Username;
    @JsonProperty("password")
     private String Password;

     public String getPassword() {
         return Password;
     }

     public void setPassword(String password) {
         Password = password;
     }

     public String getUsername() {
         return Username;
     }

     public void setUsername(String username) {
         Username = username;
     }
 }

```

26 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AuthResponse.java

28 lines

```java
package com.example.EmployeeManagementSystem.DTO;

public class AuthResponse {
    private String accessToken;
    private String refreshToken;

    public AuthResponse(String accessToken, String token) {
        this.accessToken=accessToken;
        this.refreshToken=token;
     }

     public String getAccessToken() {
         return accessToken;
     }

     public void setAccessToken(String accessToken) {
         this.accessToken = accessToken;
     }

     public String getRefreshToken() {
         return refreshToken;
     }

     public void setRefreshToken(String refreshToken) {
         this.refreshToken = refreshToken;
     }
 }

```

27 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\EmployeeDTO.java

50 lines

```java
package com.example.EmployeeManagementSystem.DTO;


public class EmployeeDTO {
    private String name;
    private String email;
    private String dept;
    private String password;
    private String timezone;

     public String getTimezone() {
         return timezone;
     }

     public void setTimezone(String timezone) {
         this.timezone = timezone;
     }

     public String getName() {
         return name;
     }

     public void setName(String name) {
         this.name = name;
     }

     public String getEmail() {
         return email;
     }

     public void setEmail(String email) {
         this.email = email;
     }

     public String getDept() {
         return dept;
     }

     public void setDept(String dept) {
         this.dept = dept;
     }

     public String getPassword() {
         return password;
     }

     public void setPassword(String password) {
         this.password = password;
     }
 }
```

28 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\LeaveRequestDTO.java

48 lines

```java
package com.example.EmployeeManagementSystem.DTO;

import com.example.EmployeeManagementSystem.Enum.LeaveType;

import java.time.LocalDate;

// DTO used when employee applies for leave.
// Contains only input fields required from employee.
public class LeaveRequestDTO {
    private LeaveType leaveType;
    private LocalDate startDate;
    private LocalDate endDate;
    private String reason;


    public LeaveType getLeaveType() {
        return leaveType;
    }

    public void setLeaveType(LeaveType leaveType) {
        this.leaveType = leaveType;
    }

    public LocalDate getStartDate() {
        return startDate;
    }

    public void setStartDate(LocalDate startDate) {
        this.startDate = startDate;
    }

    public LocalDate getEndDate() {
        return endDate;
    }

    public void setEndDate(LocalDate endDate) {
        this.endDate = endDate;
    }

    public String getReason() {
        return reason;
    }

    public void setReason(String reason) {
        this.reason = reason;
    }
}

```

29 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\LeaveResponseDTO.java

73 lines

```java
package com.example.EmployeeManagementSystem.DTO;

import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
import com.example.EmployeeManagementSystem.Enum.LeaveType;

import java.time.LocalDate;

public class LeaveResponseDTO {
    private long leaveRequestId;
     private long employeeId;
     private LeaveType leaveType;
     private LocalDate startDate;
     private LocalDate endDate;
     private String reason;
     private LeaveStatus status;

     public long getLeaveRequestId() {
         return leaveRequestId;
     }

     public void setLeaveRequestId(long leaveRequestId) {
         this.leaveRequestId = leaveRequestId;
     }

     public long getEmployeeId() {
         return employeeId;
     }

     public void setEmployeeId(long employeeId) {
         this.employeeId = employeeId;
     }

     public LocalDate getStartDate() {
         return startDate;
     }

     public void setStartDate(LocalDate startDate) {
         this.startDate = startDate;
     }

     public LeaveType getLeaveType() {
         return leaveType;
     }

     public void setLeaveType(LeaveType leaveType) {
         this.leaveType = leaveType;
     }

     public LocalDate getEndDate() {
         return endDate;
     }

     public void setEndDate(LocalDate endDate) {
         this.endDate = endDate;
     }

     public String getReason() {
         return reason;
     }

     public void setReason(String reason) {
         this.reason = reason;
     }

     public LeaveStatus getStatus() {
         return status;
     }

     public void setStatus(LeaveStatus status) {
         this.status = status;
     }
```

```
}

```

30 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\RestaurantDTO.java

42 lines

```java
package com.example.EmployeeManagementSystem.DTO;

import com.example.EmployeeManagementSystem.Enum.MealSlot;

import java.util.Set;

public class RestaurantDTO {
    private Long id;
    private String name;
     private String address;
     private String description;
     private Set<MealSlot> supportedMealSlots;
     private boolean active;
     private Long vendorId;
     private String vendorName;

     public Long getId() { return id; }
     public void setId(Long id) { this.id = id; }

     public String getName() { return name; }
     public void setName(String name) { this.name = name; }

     public String getAddress() { return address; }
     public void setAddress(String address) { this.address = address; }

     public String getDescription() { return description; }
     public void setDescription(String description) { this.description = description; }

     public Set<MealSlot> getSupportedMealSlots() { return supportedMealSlots; }
     public void setSupportedMealSlots(Set<MealSlot> supportedMealSlots) {
         this.supportedMealSlots = supportedMealSlots;
     }

     public boolean isActive() { return active; }
     public void setActive(boolean active) { this.active = active; }

     public Long getVendorId() { return vendorId; }
     public void setVendorId(Long vendorId) { this.vendorId = vendorId; }

     public String getVendorName() { return vendorName; }
     public void setVendorName(String vendorName) { this.vendorName = vendorName; }
}
```

31 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\RestaurantRequest.java

46 lines

```java
package com.example.EmployeeManagementSystem.DTO;

import com.example.EmployeeManagementSystem.Enum.MealSlot;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotEmpty;
import jakarta.validation.constraints.NotNull;

import java.util.Set;

 public class RestaurantRequest {

     @NotBlank
     private String name;

     @NotBlank
     private String address;

     private String description;

     /**
      * Which meal slots this restaurant serves.
      * Must be at least one of: BREAKFAST, LUNCH, DINNER.
      */
     @NotNull
     @NotEmpty
     private Set<MealSlot> supportedMealSlots;

     /**
      * The vendor ID of the owner making this request.
      * Used to verify ownership on updates/deletes.
      */

     public String getName() { return name; }
     public void setName(String name) { this.name = name; }

     public String getAddress() { return address; }
     public void setAddress(String address) { this.address = address; }

     public String getDescription() { return description; }
     public void setDescription(String description) { this.description = description; }

     public Set<MealSlot> getSupportedMealSlots() { return supportedMealSlots; }
     public void setSupportedMealSlots(Set<MealSlot> supportedMealSlots) {
         this.supportedMealSlots = supportedMealSlots;
     }
 }
```

32 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\SubscriptionDTO.java

103 lines

```java
package com.example.EmployeeManagementSystem.DTO;

import com.example.EmployeeManagementSystem.Enum.MealSlot;
import com.example.EmployeeManagementSystem.Enum.ScheduleType;
import com.example.EmployeeManagementSystem.Enum.SubscriptionStatus;

import java.time.DayOfWeek;
import java.time.ZonedDateTime;

 public class SubscriptionDTO {
     private long subscriptionId;
     private long userId;
     private MealSlot mealSlot;
     private ScheduleType scheduleType;
     private DayOfWeek dayOfWeek;
     private SubscriptionStatus status;
     private ZonedDateTime nextDeliveryTime;

     private Long restaurantId;
     private String restaurantName;
     private String restaurantAddress;

     public void setRestaurantId(Long restaurantId) {
         this.restaurantId = restaurantId;
     }

     public void setRestaurantName(String restaurantName) {
         this.restaurantName = restaurantName;
     }

     public void setRestaurantAddress(String restaurantAddress) {
         this.restaurantAddress = restaurantAddress;
     }

     public Long getRestaurantId() {
         return restaurantId;
     }

     public String getRestaurantName() {
         return restaurantName;
     }

     public String getRestaurantAddress() {
         return restaurantAddress;
     }

     public long getSubscriptionId() {
         return subscriptionId;
     }

     public void setSubscriptionId(long subscriptionId) {
         this.subscriptionId = subscriptionId;
     }

     public long getUserId() {
         return userId;
     }

     public void setUserId(long userId) {
         this.userId = userId;
     }

     public MealSlot getMealSlot() {
         return mealSlot;
     }

     public void setMealSlot(MealSlot mealSlot) {
         this.mealSlot = mealSlot;
     }

     public ScheduleType getScheduleType() {
```

```
        return scheduleType;
    }

    public void setScheduleType(ScheduleType scheduleType) {
        this.scheduleType = scheduleType;
    }

    public DayOfWeek getDayOfWeek() {
        return dayOfWeek;
    }

    public void setDayOfWeek(DayOfWeek dayOfWeek) {
        this.dayOfWeek = dayOfWeek;
    }

    public SubscriptionStatus getStatus() {
        return status;
    }

    public void setStatus(SubscriptionStatus status) {
        this.status = status;
    }

    public ZonedDateTime getNextDeliveryTime() {
        return nextDeliveryTime;
    }

    public void setNextDeliveryTime(ZonedDateTime nextDeliveryTime) {
         this.nextDeliveryTime = nextDeliveryTime;
     }
}

```

33 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\SubscriptionRequest.java

72 lines

```java
package com.example.EmployeeManagementSystem.DTO;

import com.example.EmployeeManagementSystem.Enum.MealSlot;
import com.example.EmployeeManagementSystem.Enum.ScheduleType;
import jakarta.validation.constraints.NotEmpty;
import jakarta.validation.constraints.NotNull;

import java.time.DayOfWeek;
import java.util.List;

 public class SubscriptionRequest {
     private ScheduleType scheduleType;
     private DayOfWeek dayOfWeek;
     @NotNull
     @NotEmpty
     private List<MealSlot> mealSlots;
     @NotNull
     private List<SlotOrder> slotOrders;
     public static class SlotOrder {
         @NotNull
         private MealSlot mealSlot;

         @NotNull
         private Long restaurantId;

         public MealSlot getMealSlot() {
             return mealSlot;
         }

         public void setMealSlot(MealSlot mealSlot) {
             this.mealSlot = mealSlot;
         }

         public Long getRestaurantId() {
             return restaurantId;
         }

         public void setRestaurantId(Long restaurantId) {
             this.restaurantId = restaurantId;
         }
     }

     public ScheduleType getScheduleType() {
         return scheduleType;
     }

     public void setScheduleType(ScheduleType scheduleType) {
         this.scheduleType = scheduleType;
     }

     public List<MealSlot> getMealSlots() {
         return mealSlots;
     }

     public void setMealSlots(List<MealSlot> mealSlots) {
         this.mealSlots = mealSlots;
     }

     public DayOfWeek getDayOfWeek() {
         return dayOfWeek;
     }

     public void setDayOfWeek(DayOfWeek dayOfWeek) {
         this.dayOfWeek = dayOfWeek;
     }



     public List<SlotOrder> getSlotOrders() { return slotOrders; }
     public void setSlotOrders(List<SlotOrder> slotOrders) { this.slotOrders = slotOrders; }
 }
```

34 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\VendorDTO.java

26 lines

```java
package com.example.EmployeeManagementSystem.DTO;

import java.time.LocalDate;

public class VendorDTO {
    private Long id;
    private String name;
    private String email;
    private String phone;
     private LocalDate registeredAt;

     public Long getId() { return id; }
     public void setId(Long id) { this.id = id; }

     public String getName() { return name; }
     public void setName(String name) { this.name = name; }

     public String getEmail() { return email; }
     public void setEmail(String email) { this.email = email; }

     public String getPhone() { return phone; }
     public void setPhone(String phone) { this.phone = phone; }

     public LocalDate getRegisteredAt() { return registeredAt; }
     public void setRegisteredAt(LocalDate registeredAt) { this.registeredAt = registeredAt; }
 }
```

35 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\VendorRequest.java

36 lines

```java
package com.example.EmployeeManagementSystem.DTO;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;

public class VendorRequest {

    @NotBlank
    private String name;

     @Email
     @NotBlank
     private String email;

     @NotBlank
     private String phone;

     private String password;

     public String getName() { return name; }
     public void setName(String name) { this.name = name; }

     public String getEmail() { return email; }
     public void setEmail(String email) { this.email = email; }

     public String getPhone() { return phone; }
     public void setPhone(String phone) { this.phone = phone; }

     public String getPassword() {
         return password;
     }

     public void setPassword(String password) {
         this.password = password;
     }
}
```

36 Current: src\main\java\com\example\EmployeeManagementSystem\EmployeeManagementSystemApplication

```
package com.example.EmployeeManagementSystem;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableScheduling;

@SpringBootApplication
@EnableScheduling
public class EmployeeManagementSystemApplication {

    public static void main(String[] args) {
        SpringApplication.run(EmployeeManagementSystemApplication.class, args);
    }

}

```

```java
// src/main/java/com/example/EmployeeManagementSystem/Entity/ApiKey.java
package com.example.EmployeeManagementSystem.Entity;

import jakarta.persistence.*;
import java.time.Instant;
import java.util.UUID;

@Entity
@Table(name = "api_keys")
 public class ApiKey {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String apiKey;

    @Column(nullable = false)
    private String keyName;

    @ManyToOne
    @JoinColumn(name = "employee_id")
    private Employee employee;

    @ManyToOne
    @JoinColumn(name = "vendor_id")
    private Vendor vendor;

    @Column(nullable = false)
    private String permissions; // Comma-separated permissions

    @Column(nullable = false)
    private Instant createdAt;

    private Instant expiresAt;

    private Instant lastUsedAt;

    @Column(nullable = false)
    private boolean active = true;

    @PrePersist
    public void prePersist() {
        this.apiKey = "ems_" + UUID.randomUUID().toString().replace("-", "");
        this.createdAt = Instant.now();
    }

    // Getters and setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getApiKey() { return apiKey; }
    public void setApiKey(String apiKey) { this.apiKey = apiKey; }

    public String getKeyName() { return keyName; }
    public void setKeyName(String keyName) { this.keyName = keyName; }

    public Employee getEmployee() { return employee; }
    public void setEmployee(Employee employee) { this.employee = employee; }

    public Vendor getVendor() { return vendor; }
    public void setVendor(Vendor vendor) { this.vendor = vendor; }

    public String getPermissions() { return permissions; }
    public void setPermissions(String permissions) { this.permissions = permissions; }

    public Instant getCreatedAt() { return createdAt; }
    public void setCreatedAt(Instant createdAt) { this.createdAt = createdAt; }

    public Instant getExpiresAt() { return expiresAt; }
```

```
    public void setExpiresAt(Instant expiresAt) { this.expiresAt = expiresAt; }

    public Instant getLastUsedAt() { return lastUsedAt; }
    public void setLastUsedAt(Instant lastUsedAt) { this.lastUsedAt = lastUsedAt; }

    public boolean isActive() { return active; }
    public void setActive(boolean active) { this.active = active; }
}
```

```java
package com.example.EmployeeManagementSystem.Entity;

import com.example.EmployeeManagementSystem.Enum.DeliveryStatus;
import com.example.EmployeeManagementSystem.Enum.MealSlot;
import jakarta.persistence.*;

import java.time.Instant;

@Entity
 @Table(name = "delivery", uniqueConstraints = { @UniqueConstraint(name = "uq_subscription_scheduled_time",
         columnNames = {"subscription_id", "scheduledDeliveryTime"})})
 public class Delivery {
     @Id
     @GeneratedValue(strategy = GenerationType.IDENTITY)
     private Long id;

     @ManyToOne
     @JoinColumn(name = "subscription_id")
     private Subscription subscription;
     private MealSlot mealSlot;
     private Instant scheduledDeliveryTime;
     private Instant actualDeliveryTime;

     @Enumerated(EnumType.STRING)
     private DeliveryStatus status;

     private Instant createdAt;

     @PrePersist
     public void prePersist() {
         createdAt = Instant.now();
         if (status == null) {
             status = DeliveryStatus.SCHEDULED;
         }
     }

     public MealSlot getMealSlot() {
         return mealSlot;
     }

     public void setMealSlot(MealSlot mealSlot) {
         this.mealSlot = mealSlot;
     }

     public Subscription getSubscription() {
         return subscription;
     }

     public void setSubscription(Subscription subscription) {
         this.subscription = subscription;
     }

     public Long getId() {
         return id;
     }

     public void setId(Long id) {
         this.id = id;
     }

     public Instant getScheduledDeliveryTime() {
         return scheduledDeliveryTime;
     }

     public void setScheduledDeliveryTime(Instant scheduledDeliveryTime) {
         this.scheduledDeliveryTime = scheduledDeliveryTime;
     }

     public Instant getActualDeliveryTime() {
         return actualDeliveryTime;
     }
```

```

    public void setActualDeliveryTime(Instant actualDeliveryTime) {
        this.actualDeliveryTime = actualDeliveryTime;
    }

    public DeliveryStatus getStatus() {
        return status;
    }

    public void setStatus(DeliveryStatus status) {
        this.status = status;
    }

    public Instant getCreatedAt() {
        return createdAt;
    }

    public void setCreatedAt(Instant createdAt) {
        this.createdAt = createdAt;
    }
}

```

```java
package com.example.EmployeeManagementSystem.Entity;

import com.example.EmployeeManagementSystem.Enum.Role;
import com.example.EmployeeManagementSystem.Enum.Status;
import jakarta.persistence.*;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;

import java.time.LocalDate;
import java.util.Collection;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;

@Entity
public class Employee implements UserDetails {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long employeeId;
    @Column(nullable = true,unique = false)
    private String name;
    @Column(unique = true, nullable = false)
    private String email;
    @Column(nullable = true)
    private String dept;
    @Enumerated(EnumType.STRING)
    private Status status;
    private String password;
    private LocalDate joined_at;
    @Enumerated(EnumType.STRING)
    private Role role;
    @Column(nullable = true)
    private String totpSecret;
    @Column(nullable = false)
    private boolean totpEnabled = false;
    /** IANA timezone id, e.g. "Asia/Kolkata", "America/New_York". Defaults to UTC. */
    @Column(nullable = false)
    private String timezone = "UTC";

    public String getTimezone() {
        return timezone;
    }

    public void setTimezone(String timezone) {
        this.timezone = timezone;
    }

    public Role getRole() {
        return role;
    }

    public void setRole(Role role) {
        this.role = role;
    }

    public long getEmployeeId() {
        return employeeId;
    }

    public void setEmployeeId(long employeeId) {
        this.employeeId = employeeId;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
```

```
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getDept() {
        return dept;
    }

    public void setDept(String dept) {
        this.dept = dept;
    }

    public Status getStatus() {
        return status;
    }

    public void setStatus(Status status) {
        this.status = status;
    }

    public LocalDate getJoined_at() {
        return joined_at;
    }

    public void setJoined_at(LocalDate joined_at) {
        this.joined_at = joined_at;
    }

    @PrePersist
    public void initialSetup() {
        this.status = Status.ACTIVE;
        this.joined_at = LocalDate.now();
        if (this.timezone == null || this.timezone.isBlank()) {
            this.timezone = "UTC";
        }
    }

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        Set<SimpleGrantedAuthority> authorities=new HashSet<>();
        authorities.add(new SimpleGrantedAuthority("ROLE_"+role.name()));
        authorities.addAll(role.getPermissions().stream()
                .map(permissions -> new SimpleGrantedAuthority(permissions.name()))
                .collect(Collectors.toSet()));
        return authorities;
    }

    public String getPassword() {
        return password;
    }

    @Override
    public String getUsername() {
        return email;
    }

    @Override
    public boolean isAccountNonExpired() {
        return UserDetails.super.isAccountNonExpired();
    }

    @Override
    public boolean isAccountNonLocked() {
        return UserDetails.super.isAccountNonLocked();
    }

    @Override
    public boolean isCredentialsNonExpired() {
        return UserDetails.super.isCredentialsNonExpired();
    }
```

```
    @Override
    public boolean isEnabled() {
        return UserDetails.super.isEnabled();
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getTotpSecret() {
        return totpSecret;
    }

    public boolean isTotpEnabled() {
        return totpEnabled;
    }

    public void setTotpSecret(String totpSecret) {
        this.totpSecret = totpSecret;
    }

    public void setTotpEnabled(boolean totpEnabled) {
        this.totpEnabled = totpEnabled;
    }
}
```

40 Current: src\main\java\com\example\EmployeeManagementSystem\Entity\LeaveRequest.java

105 lines

```java
package com.example.EmployeeManagementSystem.Entity;

import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
import com.example.EmployeeManagementSystem.Enum.LeaveType;
import jakarta.persistence.*;

import java.time.LocalDate;

@Entity
 public class LeaveRequest {
     @Id
     @GeneratedValue(strategy = GenerationType.IDENTITY)
     private long id;
     @Enumerated(EnumType.STRING)
     private LeaveType leaveType;
     private LocalDate startDate;
     private LocalDate endDate;
     private String reason;
     @Enumerated(EnumType.STRING)
     private LeaveStatus status;
     private String Manager;
     private String remarks;
     @ManyToOne
     @JoinColumn(name = "employee_id")
     private Employee employee;


     public long getId() {
         return id;
     }

     public void setId(long id) {
         this.id = id;
     }

     public LeaveType getLeaveType() {
         return leaveType;
     }

     public void setLeaveType(LeaveType leaveType) {
         this.leaveType = leaveType;
     }

     public LocalDate getStartDate() {
         return startDate;
     }

     public void setStartDate(LocalDate startDate) {
         this.startDate = startDate;
     }

     public LocalDate getEndDate() {
         return endDate;
     }

     public void setEndDate(LocalDate endDate) {
         this.endDate = endDate;
     }

     public LeaveStatus getStatus() {
         return status;
     }

     public void setStatus(LeaveStatus status) {
         this.status = status;
     }

     public String getManager() {
         return Manager;
     }

```

```
    public void setManager(String manager) {
        Manager = manager;
    }

    public String getRemarks() {
        return remarks;
    }

    public void setRemarks(String remarks) {
        this.remarks = remarks;
    }

    public Employee getEmployee() {
        return employee;
    }

    public void setEmployee(Employee employee) {
        this.employee = employee;
    }

    public String getReason() {
        return reason;
    }

    public void setReason(String reason) {
        this.reason = reason;
    }

     @PrePersist
     public void init(){
         this.status=LeaveStatus.PENDING;
     }
}

```

41 Current: src\main\java\com\example\EmployeeManagementSystem\Entity\RefreshToken.java

62 lines

```java
package com.example.EmployeeManagementSystem.Entity;

import jakarta.persistence.*;

import java.time.Instant;
@Entity
public class RefreshToken {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
     private Long id;

     @Column(nullable = false, unique = true)
     private String token;

     @Column(nullable = false)
     private Instant expiryTime;

     @ManyToOne
     @JoinColumn(name = "employee_id")
     private Employee employee;

     @ManyToOne
     @JoinColumn(name = "vendor_id")
     private Vendor vendor;

     public Long getId() {
         return id;
     }

     public String getToken() {
         return token;
     }

     public void setToken(String token) {
         this.token = token;
     }

     public Instant getExpiryTime() {
         return expiryTime;
     }

     public void setExpiryTime(Instant expiryTime) {
         this.expiryTime = expiryTime;
     }

     public Employee getEmployee() {
         return employee;
     }

     public void setEmployee(Employee employee) {
         this.employee = employee;
     }

     public Vendor getVendor() {
         return vendor;
     }

     public void setVendor(Vendor vendor) {
         this.vendor = vendor;
     }
 }

```

42 Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Restaurant.java

58 lines

```java
package com.example.EmployeeManagementSystem.Entity;

import com.example.EmployeeManagementSystem.Enum.MealSlot;
import jakarta.persistence.*;
import java.util.Set;


@Entity
public class Restaurant {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false)
    private String address;

    private String description;

    @ElementCollection(targetClass = MealSlot.class, fetch = FetchType.EAGER)
    @CollectionTable(name = "restaurant_meal_slots",
            joinColumns = @JoinColumn(name = "restaurant_id"))
    @Enumerated(EnumType.STRING)
    @Column(name = "meal_slot")
    private Set<MealSlot> supportedMealSlots;

    private boolean active = true;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "vendor_id", nullable = false)
    private Vendor vendor;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getAddress() { return address; }
    public void setAddress(String address) { this.address = address; }

    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }

    public Set<MealSlot> getSupportedMealSlots() { return supportedMealSlots; }
    public void setSupportedMealSlots(Set<MealSlot> supportedMealSlots) {
        this.supportedMealSlots = supportedMealSlots;
    }

    public boolean isActive() { return active; }
    public void setActive(boolean active) { this.active = active; }

    public Vendor getVendor() { return vendor; }
    public void setVendor(Vendor vendor) { this.vendor = vendor; }
}
```

```java
package com.example.EmployeeManagementSystem.Entity;

import com.example.EmployeeManagementSystem.Enum.MealSlot;
import com.example.EmployeeManagementSystem.Enum.ScheduleType;
import com.example.EmployeeManagementSystem.Enum.SubscriptionStatus;
import jakarta.persistence.*;

import java.time.DayOfWeek;
import java.time.Instant;
import java.time.LocalDateTime;
import java.time.LocalTime;
@Entity
public class Subscription {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long id;
    @Enumerated(EnumType.STRING)
    private MealSlot slot;
    private ScheduleType scheduleType;
    private DayOfWeek dayOfWeek;
    @Enumerated(EnumType.STRING)
    private SubscriptionStatus status;
    private Instant nextDeliveryTime;
    private LocalTime created_at;

    @ManyToOne
    @JoinColumn(name="employee_id")
    private Employee employee;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "restaurant_id", nullable = false)
    private Restaurant restaurant;

    @PrePersist
    public void init(){
        this.created_at=LocalTime.now();
    }

    public long getId() {
        return id;
    }

    public void setId(long id) {
        this.id = id;
    }

    public MealSlot getSlot() {
        return slot;
    }

    public void setSlot(MealSlot slot) {
        this.slot = slot;
    }

    public ScheduleType getScheduleType() {
        return scheduleType;
    }

    public void setScheduleType(ScheduleType scheduleType) {
        this.scheduleType = scheduleType;
    }

    public DayOfWeek getDayOfWeek() {
        return dayOfWeek;
    }

    public void setDayOfWeek(DayOfWeek dayOfWeek) {
        this.dayOfWeek = dayOfWeek;
    }

    public SubscriptionStatus getStatus() {
```

```
        return status;
    }

    public void setStatus(SubscriptionStatus status) {
        this.status = status;
    }

    public LocalTime getCreated_at() {
        return created_at;
    }

    public void setCreated_at(LocalTime created_at) {
        this.created_at = created_at;
    }

    public Instant getNextDeliveryTime() {
        return nextDeliveryTime;
    }

    public void setNextDeliveryTime(Instant nextDeliveryTime) {
        this.nextDeliveryTime = nextDeliveryTime;
    }

    public Employee getEmployee() {
        return employee;
    }

    public void setEmployee(Employee employee) {
         this.employee = employee;
     }
     public Restaurant getRestaurant() { return restaurant; }
     public void setRestaurant(Restaurant restaurant) { this.restaurant = restaurant; }
}

```

```java
package com.example.EmployeeManagementSystem.Entity;

import com.example.EmployeeManagementSystem.Enum.Role;
import jakarta.persistence.*;
import org.jspecify.annotations.Nullable;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;

import java.time.LocalDate;
import java.util.Collection;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;

/**
 * Represents a restaurant vendor/owner.
 * Vendors can register and manage their restaurants.
 * They cannot access employee, leave, or subscription APIs.
 */
@Entity
@Table(name = "vendor")
public class Vendor implements UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(unique = true, nullable = false)
    private String email;

    @Column(nullable = false)
    private String phone;

    @Column(nullable = false)
    private String password;

    private LocalDate registeredAt;

    @PrePersist
    public void init() {
        this.registeredAt = LocalDate.now();
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getPhone() { return phone; }
    public void setPhone(String phone) { this.phone = phone; }

    public LocalDate getRegisteredAt() { return registeredAt; }
    public void setRegisteredAt(LocalDate registeredAt) { this.registeredAt = registeredAt; }


    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        Set<SimpleGrantedAuthority> authorities=new HashSet<>();
        authorities.add(new SimpleGrantedAuthority("ROLE_VENDOR"));
        authorities.addAll(Role.VENDOR.getPermissions().stream()
                .map(permissions -> new SimpleGrantedAuthority(permissions.name()))
                .collect(Collectors.toSet()));
```

```
        return authorities;
    }

    @Override
    public @Nullable String getPassword() {
        return password;
    }

    @Override
    public String getUsername() {
        return email;
    }

    @Override
    public boolean isAccountNonExpired() {
        return UserDetails.super.isAccountNonExpired();
    }

    @Override
    public boolean isAccountNonLocked() {
        return UserDetails.super.isAccountNonLocked();
    }

    @Override
    public boolean isCredentialsNonExpired() {
        return UserDetails.super.isCredentialsNonExpired();
    }

     @Override
     public boolean isEnabled() {
         return UserDetails.super.isEnabled();
     }

     public void setPassword(String password) {
         this.password = password;
     }
}
```

45 Current: src\main\java\com\example\EmployeeManagementSystem\Enum\DeliveryStatus.java

9 lines

```
package com.example.EmployeeManagementSystem.Enum;

public enum DeliveryStatus {
    SCHEDULED,
    IN_PROGRESS,
    DELIVERED,
    MISSED
}

```

46 Current: src\main\java\com\example\EmployeeManagementSystem\Enum\LeaveStatus.java

9 lines

```java
package com.example.EmployeeManagementSystem.Enum;

public enum LeaveStatus {
    PENDING,
    APPROVED,
    REJECTED,
    CANCELLED
}

```

47 Current: src\main\java\com\example\EmployeeManagementSystem\Enum\LeaveType.java

8 lines

```
package com.example.EmployeeManagementSystem.Enum;

public enum LeaveType {
    SICK,
    CASUAL,
    EARNED
}

```

```java
package com.example.EmployeeManagementSystem.Enum;

public enum MealSlot {
    BREAKFAST,
    LUNCH,
    DINNER
}

```

```
package com.example.EmployeeManagementSystem.Enum;

import java.util.Set;

public enum Permission {
    LEAVE_READ,
    LEAVE_WRITE,
    LEAVE_UPDATE,
    LEAVE_CANCEL,
     SUBSCRIPTION_WRITE,
     SUBSCRIPTION_READ,
     SUBSCRIPTION_UPDATE,
     VENDOR_UPDATE,
     VENDOR_DELETE,
     RESTAURANTS_ADD,
     RESTAURANTS_READ,
     RESTAURANTS_DELETE;
 }

```

```
package com.example.EmployeeManagementSystem.Enum;

import java.util.Collection;
import java.util.Set;

public enum Role {
    EMPLOYEE(Set.of(Permission.LEAVE_WRITE,
            Permission.LEAVE_CANCEL,
            Permission.SUBSCRIPTION_WRITE,
             Permission.SUBSCRIPTION_UPDATE,
             Permission.RESTAURANTS_READ,
             Permission.SUBSCRIPTION_READ)),
     MANAGER(Set.of(Permission.LEAVE_UPDATE,
             Permission.LEAVE_WRITE,
             Permission.SUBSCRIPTION_WRITE,
             Permission.RESTAURANTS_READ,
             Permission.SUBSCRIPTION_UPDATE,
             Permission.LEAVE_READ,
             Permission.SUBSCRIPTION_READ)),
     VENDOR(Set.of(
             Permission.VENDOR_DELETE,
             Permission.RESTAURANTS_ADD,
             Permission.RESTAURANTS_DELETE,
             Permission.RESTAURANTS_READ
     ));
     private final Set<Permission> permissions;

     Role(Set<Permission> permissions) {
         this.permissions = permissions;
     }

     public Set<Permission> getPermissions() {
      return permissions;
     }
 }

```

51 Current: src\main\java\com\example\EmployeeManagementSystem\Enum\ScheduleType.java

7 lines

```
package com.example.EmployeeManagementSystem.Enum;

public enum ScheduleType {
    DAILY,
    WEEKLY
}

```

```java
package com.example.EmployeeManagementSystem.Enum;

public enum Status {
    ACTIVE,
    INACTIVE,
    ON_LEAVE
}

```

53 Current: src\main\java\com\example\EmployeeManagementSystem\Enum\SubscriptionStatus.java

8 lines

```java
package com.example.EmployeeManagementSystem.Enum;

public enum SubscriptionStatus {
    ACTIVE,
    PAUSED,
    EXPIRED
}

```

54 Current: src\main\java\com\example\EmployeeManagementSystem\Exception\DuplicateRequestException.jav

```
package com.example.EmployeeManagementSystem.Exception;

public class DuplicateRequestException extends RuntimeException {
    public DuplicateRequestException(String message) {
        super(message);
    }
}

```

55 Current: src\main\java\com\example\EmployeeManagementSystem\Exception\EmployeeNotFound.java

10 lines

```
package com.example.EmployeeManagementSystem.Exception;

import org.springframework.security.core.userdetails.UsernameNotFoundException;

public class EmployeeNotFound extends UsernameNotFoundException {
    public EmployeeNotFound(String message) {
        super(message);
    }
}

```

56 Current: src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidEndDateException.java

```java
package com.example.EmployeeManagementSystem.Exception;

public class InvalidEndDateException extends RuntimeException{
    public InvalidEndDateException(String message) {
        super(message);
    }
}

```

57 Current: src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidManagerException.java

```java
package com.example.EmployeeManagementSystem.Exception;

public class InvalidManagerException extends RuntimeException{
    public InvalidManagerException(String message) {
        super(message);
    }
}

```

58 Current: src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidStartDateException.java

```java
package com.example.EmployeeManagementSystem.Exception;

public class InvalidStartDateException extends RuntimeException{
    public InvalidStartDateException(String message) {
        super(message);
    }
}

```

59 Current: src\main\java\com\example\EmployeeManagementSystem\Exception\LeaveRequestNotFoundExcept

```
package com.example.EmployeeManagementSystem.Exception;

public class LeaveRequestNotFoundException extends RuntimeException{
    public LeaveRequestNotFoundException(String message) {
        super(message);
    }
}

```

60 Current: src\main\java\com\example\EmployeeManagementSystem\Exception\OverlappingLeaveException.jav

```
package com.example.EmployeeManagementSystem.Exception;

public class OverlappingLeaveException extends RuntimeException{
    public OverlappingLeaveException(String message) {
        super(message);
    }
}

```

61 Current: src\main\java\com\example\EmployeeManagementSystem\Exception\RestaurantNotFoundException

```java
package com.example.EmployeeManagementSystem.Exception;

public class RestaurantNotFoundException extends RuntimeException {
    public RestaurantNotFoundException(String message) {
        super(message);
    }
}

```

62 Current: src\main\java\com\example\EmployeeManagementSystem\Exception\SubscriptionAlreadyExists.java

```java
package com.example.EmployeeManagementSystem.Exception;

public class SubscriptionAlreadyExists extends RuntimeException{
    public SubscriptionAlreadyExists(String message) {
        super(message);
    }
}

```

63 Current: src\main\java\com\example\EmployeeManagementSystem\Exception\UnauthorizedAccessException.

```
package com.example.EmployeeManagementSystem.Exception;

public class UnauthorizedAccessException extends RuntimeException {
    public UnauthorizedAccessException(String message) {
        super(message);
    }
}
```

64 Current: src\main\java\com\example\EmployeeManagementSystem\Exception\VendorNotFoundException.java

```java
package com.example.EmployeeManagementSystem.Exception;

public class VendorNotFoundException extends RuntimeException {
    public VendorNotFoundException(String message) {
        super(message);
    }
}
```

65 Current: src\main\java\com\example\EmployeeManagementSystem\Filter\ApiKeyFilter.java

45 lines

```java
// src/main/java/com/example/EmployeeManagementSystem/Filter/ApiKeyFilter.java
package com.example.EmployeeManagementSystem.Filter;

import com.example.EmployeeManagementSystem.security.ApiKeyAuthenticationToken;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

public class ApiKeyFilter extends OncePerRequestFilter {

    private static final String API_KEY_HEADER = "X-API-Key";

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOException {

        String apiKey = request.getHeader(API_KEY_HEADER);

        if (apiKey != null && !apiKey.isEmpty()) {
            ApiKeyAuthenticationToken authToken = new ApiKeyAuthenticationToken(apiKey);
            SecurityContextHolder.getContext().setAuthentication(authToken);
        }

        filterChain.doFilter(request, response);
    }

    @Override
    protected boolean shouldNotFilter(HttpServletRequest request) {
        String path = request.getRequestURI();
        return path.startsWith("/Authenticate") ||
                path.startsWith("/session/login") ||
                path.startsWith("/session/logout") ||
                path.startsWith("/oauth2") ||
                path.startsWith("/login/oauth2") ||
                path.startsWith("/swagger-ui") ||
                path.startsWith("/v3/api-docs") ||
                path.startsWith("/api-keys/generate"); // Allow API key generation without API key
    }
}
```

```java
package com.example.EmployeeManagementSystem.Filter;

import com.example.EmployeeManagementSystem.Util.JWTUtil;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
@Component
public class JwtAuthFilter extends OncePerRequestFilter {
    @Autowired
    @Qualifier("combinedUserDetailService")
    private UserDetailsService customUserDetailService;
    @Autowired
    private JWTUtil jwtUtil;


    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOException {
        String token=null;
        String username=null;

        String authHeader=request.getHeader("Authorization");
        try {

            if (authHeader != null && authHeader.startsWith("Bearer ")) {
                token = authHeader.substring(7);
                username = jwtUtil.extractUsernameFromToken(token);
            }

        } catch (io.jsonwebtoken.ExpiredJwtException e) {

            response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
            response.setContentType("application/json");

            response.getWriter().write("""
        {
          "error": "TOKEN_EXPIRED",
          "message": "JWT expired. Please login again to get a new token."
        }
        """);

            return;
        }
        if (username!=null && SecurityContextHolder.getContext().getAuthentication()==null){
             UserDetails userDetails=customUserDetailService.loadUserByUsername(username);
             if(userDetails!=null && jwtUtil.validateToken(username,userDetails,token)){
                 UsernamePasswordAuthenticationToken authenticationToken=new UsernamePasswordAuthenticationToken(
                         userDetails,null,userDetails.getAuthorities()
                 );
                 SecurityContextHolder.getContext().setAuthentication(authenticationToken);
             }
        }

    filterChain.doFilter(request,response);


     }
```

```

}

```

67 Current: src\main\java\com\example\EmployeeManagementSystem\Repository\ApiKeyRepository.java

36 lines

```java
// src/main/java/com/example/EmployeeManagementSystem/Repository/ApiKeyRepository.java
package com.example.EmployeeManagementSystem.Repository;

import com.example.EmployeeManagementSystem.Entity.ApiKey;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.transaction.annotation.Transactional;

 import java.time.Instant;
 import java.util.List;
 import java.util.Optional;

 public interface ApiKeyRepository extends JpaRepository<ApiKey, Long> {

     Optional<ApiKey> findByApiKeyAndActiveTrue(String apiKey);

     // FIX: Change from findByEmployeeIdAndActiveTrue to findByEmployee_EmployeeIdAndActiveTrue
     // Because Employee entity uses employeeId, not id
     List<ApiKey> findByEmployee_EmployeeIdAndActiveTrue(Long employeeId);

     // FIX: Change from findByVendorIdAndActiveTrue to findByVendor_IdAndActiveTrue
     // Because Vendor entity uses id as primary key
     List<ApiKey> findByVendor_IdAndActiveTrue(Long vendorId);

     @Modifying
     @Transactional
     @Query("UPDATE ApiKey a SET a.lastUsedAt = :now WHERE a.apiKey = :apiKey")
     void updateLastUsed(@Param("apiKey") String apiKey, @Param("now") Instant now);

     @Modifying
     @Transactional
     @Query("UPDATE ApiKey a SET a.active = false WHERE a.expiresAt < :now AND a.active = true")
     int expireOldKeys(@Param("now") Instant now);
 }
```

68 Current: src\main\java\com\example\EmployeeManagementSystem\Repository\DeliveryRepository.java

23 lines

```
package com.example.EmployeeManagementSystem.Repository;

import com.example.EmployeeManagementSystem.Entity.Delivery;
import com.example.EmployeeManagementSystem.Entity.Subscription;
import com.example.EmployeeManagementSystem.Enum.DeliveryStatus;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

 import java.time.Instant;
 import java.util.List;

 public interface DeliveryRepository extends JpaRepository<Delivery,Long> {
     List<Delivery> findByStatusAndScheduledDeliveryTimeLessThanEqual(
             DeliveryStatus status, Instant time);

     @Query("SELECT d FROM Delivery d WHERE d.subscription = :subscription " +
             "AND d.status != :status ORDER BY d.scheduledDeliveryTime DESC")
     List<Delivery> findRecentDeliveriesBySubscription(
             @Param("subscription") Subscription subscription,
             @Param("status") DeliveryStatus status);
 }

```

69 Current: src\main\java\com\example\EmployeeManagementSystem\Repository\EmployeeRepo.java

16 lines

```
package com.example.EmployeeManagementSystem.Repository;

import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Enum.Role;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface EmployeeRepo extends JpaRepository<Employee,Long> {
    Optional<Employee> findByEmail(String email);
    Optional<Employee> findByName(String name);
    long countByRole(Role role);
}

```

```java
package com.example.EmployeeManagementSystem.Repository;

import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
import com.example.EmployeeManagementSystem.Enum.LeaveType;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

 import java.time.LocalDate;
 import java.util.List;

 public interface LeaveRequestRepo extends JpaRepository<LeaveRequest, Long> {

     List<LeaveRequest> findByStatus(LeaveStatus status);

     /**
      * Find leave requests by start date and status
      * Used by scheduler to find leaves starting today
      */
     List<LeaveRequest> findByStartDateAndStatus(LocalDate startDate, LeaveStatus status);

     // Fix the checkDuplicate query (fix parameter names)
     @Query(value = "SELECT COUNT(*) " +
             "FROM leave_request l " +
             "WHERE l.employee_id = :employeeId " +
             "AND l.start_date = :startDate " +
             "AND l.end_date = :endDate " +
             "AND l.status=:status",nativeQuery = true)
     long checkDuplicate(@Param("employeeId") long employeeId,
                         @Param("startDate") LocalDate startDate,
                         @Param("endDate") LocalDate endDate,
                         @Param("status") String status);

     // Fix overlapping leave query (was commented in your code)
     @Query(value = "select COUNT(*) " +
             "from leave_request l " +
             "where l.employee_id=:employeeId " +
             "AND l.status IN ('APPROVED','PENDING') " +
             "AND l.start_date <= :endDate " +
             "AND l.end_date >= :startDate",nativeQuery = true)
     long countOverlappingLeave(
             @Param("employeeId") long employeeId,
             @Param("startDate") LocalDate startDate,
             @Param("endDate") LocalDate endDate
     );

     // Add missing methods
     List<LeaveRequest> findByEmployeeOrderByStartDateDesc(Employee employee);


     List<LeaveRequest> findByEndDateAndStatus(LocalDate endDate, LeaveStatus status);

     // Add for leave duration validation
     @Query("SELECT COALESCE(SUM(DATEDIFF(l.endDate, l.startDate) + 1), 0) " +
             "FROM LeaveRequest l WHERE l.employee = :employee " +
             "AND l.leaveType = :leaveType " +
             "AND YEAR(l.startDate) = :year " +
             "AND l.status IN ('APPROVED', 'PENDING')")
     long countDaysByEmployeeAndLeaveTypeAndYear(@Param("employee") Employee employee,
                                                 @Param("leaveType") LeaveType leaveType,
                                                 @Param("year") int year);

     @Query("SELECT lr FROM LeaveRequest lr JOIN FETCH lr.employee WHERE lr.status = :status")
     List<LeaveRequest> findApprovedLeavesWithEmployee(LeaveStatus status);

 }

```

71 Current: src\main\java\com\example\EmployeeManagementSystem\Repository\RefreshTokenRepository.java

```
package com.example.EmployeeManagementSystem.Repository;

import com.example.EmployeeManagementSystem.Entity.RefreshToken;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface RefreshTokenRepository extends JpaRepository<RefreshToken,Long> {
    Optional<RefreshToken> findByToken(String token);
 }

```

72 Current: src\main\java\com\example\EmployeeManagementSystem\Repository\RestaurantRepository.java

3
li

```
package com.example.EmployeeManagementSystem.Repository;

import com.example.EmployeeManagementSystem.Entity.Restaurant;
import com.example.EmployeeManagementSystem.Enum.MealSlot;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface RestaurantRepository extends JpaRepository<Restaurant, Long> {

    /** All active restaurants owned by a vendor. */
    List<Restaurant> findByVendor_EmailAndActiveTrue(String email);

    /** All active restaurants (for employees browsing available options). */
    List<Restaurant> findByActiveTrue();

    /**
     * Active restaurants that support a given meal slot.
     * Employees use this to discover which restaurants they can subscribe to for a given slot.
     */
    @Query("SELECT r FROM Restaurant r JOIN r.supportedMealSlots s " +
            "WHERE r.active = true AND s = :mealSlot")
    List<Restaurant> findActiveByMealSlot(@Param("mealSlot") MealSlot mealSlot);

    /** Check if a restaurant belongs to a vendor (ownership guard). */
    boolean existsByIdAndVendor_Email(Long restaurantId, String email);
}
```

73 Current: src\main\java\com\example\EmployeeManagementSystem\Repository\SubscriptionRepository.java

```java
package com.example.EmployeeManagementSystem.Repository;

import com.example.EmployeeManagementSystem.Entity.Subscription;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.time.Instant;
 import java.util.List;

 @Repository
 public interface SubscriptionRepository extends JpaRepository<Subscription,Long> {
     @Query(value = "select count(*) from subscription s where s.employee_id=:employeeId and s.slot=:slot and s.status=:status",nativeQuery = true)
     int checkSubscriptionExists(
             @Param("employeeId") long employeeId,
             @Param("slot") String slot,
             @Param("status") String status
     );

     List<Subscription> findByEmployee_employeeId(long id);

     @Query(value = "SELECT * FROM subscription s WHERE s.status = :status AND s.next_delivery_time IS NOT NULL AND s.next_delivery_time <= :currentTime", nativeQuery = true)
     List<Subscription> findDueSubscriptions(
             @Param("status") String status,
             @Param("currentTime") Instant currentTime  //  was LocalDateTime
     );

     List<Subscription> findByEmployee_name(String name);
     List<Subscription> findByEmployee_Email(String email);
 }

```

74 Current: src\main\java\com\example\EmployeeManagementSystem\Repository\VendorRepo.java

13 lines

```
package com.example.EmployeeManagementSystem.Repository;

import com.example.EmployeeManagementSystem.Entity.Vendor;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
 public interface VendorRepo extends JpaRepository<Vendor, Long> {
     Optional<Vendor> findByEmail(String email);
     boolean existsByEmail(String email);
 }
```

75 Current: src\main\java\com\example\EmployeeManagementSystem\Scheduler\DeliveryScheduler.java

30 lines

```java
package com.example.EmployeeManagementSystem.Scheduler;

import com.example.EmployeeManagementSystem.Service.DeliveryService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

@Component
 public class DeliveryScheduler {
     private static final Logger logger = LoggerFactory.getLogger(DeliveryScheduler.class);
     private final DeliveryService deliveryService;

     public DeliveryScheduler(DeliveryService deliveryService) {
         this.deliveryService = deliveryService;
     }

     // Runs every hour
     @Scheduled(cron = "0 0 * * * *")
     public void processDeliveries() {
         logger.info("Starting delivery processing scheduler...");
         try {
             deliveryService.processDueDeliveries();
             logger.info("Delivery processing completed successfully");
         } catch (Exception e) {
             logger.error("Error processing deliveries: ", e);
         }
     }
 }

```

76 Current: src\main\java\com\example\EmployeeManagementSystem\Scheduler\LeaveStatusScheduler.java

```java
package com.example.EmployeeManagementSystem.Scheduler;

import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
import com.example.EmployeeManagementSystem.Enum.Status;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDate;
import java.time.ZoneId;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

@Component
public class LeaveStatusScheduler {

    private static final Logger log = LoggerFactory.getLogger(LeaveStatusScheduler.class);

    private final LeaveRequestRepo leaveRequestRepo;
    private final EmployeeRepo employeeRepo;

    public LeaveStatusScheduler(LeaveRequestRepo leaveRequestRepo,
                                EmployeeRepo employeeRepo) {
        this.leaveRequestRepo = leaveRequestRepo;
        this.employeeRepo = employeeRepo;
    }

    /**
     * Runs every day at midnight UTC
     */
    @Scheduled(cron = "0 0 0 * * *", zone = "UTC")
    @Transactional
    public void updateEmployeeLeaveStatus() {

        log.info("=== Starting Timezone-Aware Leave Scheduler ===");

        List<LeaveRequest> leaves =
                leaveRequestRepo.findApprovedLeavesWithEmployee(LeaveStatus.APPROVED);

        Map<Long, List<LeaveRequest>> employeeLeavesMap = leaves.stream()
                .collect(Collectors.groupingBy(lr -> lr.getEmployee().getEmployeeId()));

        int onLeaveCount = 0;
        int activeCount = 0;

        for (Map.Entry<Long, List<LeaveRequest>> entry : employeeLeavesMap.entrySet()) {

            Employee employee = entry.getValue().get(0).getEmployee();

            ZoneId zone = ZoneId.of(
                    employee.getTimezone() != null ? employee.getTimezone() : "UTC"
            );

            LocalDate today = LocalDate.now(zone);

            boolean isOnLeaveToday = entry.getValue().stream().anyMatch(leave ->
                    !today.isBefore(leave.getStartDate()) &&
                            !today.isAfter(leave.getEndDate())
            );

            //  CASE 1: Should be ON_LEAVE
            if (isOnLeaveToday && employee.getStatus() != Status.ON_LEAVE) {

                employee.setStatus(Status.ON_LEAVE);
```

```
                employeeRepo.save(employee);
                onLeaveCount++;

                log.info("Employee {} ({}) marked ON_LEAVE | TZ: {}",
                        employee.getEmployeeId(),
                        employee.getName(),
                        zone);
            }

            //  CASE 2: Should be ACTIVE
            if (!isOnLeaveToday && employee.getStatus() == Status.ON_LEAVE) {

                employee.setStatus(Status.ACTIVE);
                employeeRepo.save(employee);
                activeCount++;

                log.info("Employee {} ({}) marked ACTIVE | TZ: {}",
                        employee.getEmployeeId(),
                        employee.getName(),
                        zone);
            }
        }

        log.info("=== Scheduler Completed ===");
        log.info("ON_LEAVE updated: {}", onLeaveCount);
        log.info("ACTIVE updated: {}", activeCount);
    }

    /**
     * Manual trigger (for testing)
     */
    public void runManually() {
        log.info("Manual scheduler trigger started...");
        updateEmployeeLeaveStatus();
    }
}
```

77 Current: src\main\java\com\example\EmployeeManagementSystem\security\ApiKeyAuthenticationProvider.ja

```java
// src/main/java/com/example/EmployeeManagementSystem/security/ApiKeyAuthenticationProvider.java
package com.example.EmployeeManagementSystem.security;

import com.example.EmployeeManagementSystem.Entity.ApiKey;
import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.Vendor;
import com.example.EmployeeManagementSystem.Repository.ApiKeyRepository;
import org.springframework.security.authentication.AuthenticationProvider;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.AuthenticationException;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;

import java.time.Instant;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collection;
import java.util.List;
import java.util.stream.Collectors;

@Component
public class ApiKeyAuthenticationProvider implements AuthenticationProvider {

    private final ApiKeyRepository apiKeyRepository;

    public ApiKeyAuthenticationProvider(ApiKeyRepository apiKeyRepository) {
        this.apiKeyRepository = apiKeyRepository;
    }

    @Override
    @Transactional
    public Authentication authenticate(Authentication authentication) throws AuthenticationException {
        String apiKey = null;

        // Extract API key from the authentication object
        if (authentication instanceof ApiKeyAuthenticationToken) {
            apiKey = ((ApiKeyAuthenticationToken) authentication).getApiKey();
        }

        if (apiKey == null || apiKey.isEmpty()) {
            throw new RuntimeException("Invalid API key");
        }

        ApiKey key = apiKeyRepository.findByApiKeyAndActiveTrue(apiKey)
                .orElseThrow(() -> new RuntimeException("Invalid or inactive API key"));

        // Check expiration
        if (key.getExpiresAt() != null && key.getExpiresAt().isBefore(Instant.now())) {
            throw new RuntimeException("API key has expired");
        }

        // Update last used timestamp
        apiKeyRepository.updateLastUsed(apiKey, Instant.now());

        // Build authorities from permissions
        List<GrantedAuthority> authorities = new ArrayList<>();

        if (key.getPermissions() != null && !key.getPermissions().isEmpty()) {
            authorities.addAll(Arrays.stream(key.getPermissions().split(","))
                    .map(String::trim)
                    .map(SimpleGrantedAuthority::new)
                    .collect(Collectors.toList()));
        }

        Object principal;
        if (key.getEmployee() != null) {
            principal = key.getEmployee();
            authorities.add(new SimpleGrantedAuthority("ROLE_" + key.getEmployee().getRole().name()));
        } else if (key.getVendor() != null) {
```

```
            principal = key.getVendor();
            authorities.add(new SimpleGrantedAuthority("ROLE_VENDOR"));
        } else {
            throw new RuntimeException("No user associated with API key");
        }

        return new ApiKeyAuthenticationToken(principal, apiKey, authorities);
    }

    @Override
    public boolean supports(Class<?> authentication) {
        return ApiKeyAuthenticationToken.class.isAssignableFrom(authentication);
    }
}
```

78 Current: src\main\java\com\example\EmployeeManagementSystem\security\ApiKeyAuthenticationToken.java

```java
// src/main/java/com/example/EmployeeManagementSystem/security/ApiKeyAuthenticationToken.java
package com.example.EmployeeManagementSystem.security;

import org.springframework.security.core.Authentication;
import org.springframework.security.core.GrantedAuthority;
import java.util.Collection;

public class ApiKeyAuthenticationToken implements Authentication {

    private static final long serialVersionUID = 1L;

    private final Object principal;
    private final Object credentials;
    private final String apiKey;
    private final Collection<? extends GrantedAuthority> authorities;
    private boolean authenticated;

    // Constructor for unauthenticated API key (before validation)
    public ApiKeyAuthenticationToken(String apiKey) {
        this.apiKey = apiKey;
        this.principal = null;
        this.credentials = null;
        this.authorities = null;
        this.authenticated = false;
    }

    // Constructor for authenticated API key (after validation)
    public ApiKeyAuthenticationToken(Object principal, Object credentials,
                                     Collection<? extends GrantedAuthority> authorities) {
        this.apiKey = null;
        this.principal = principal;
        this.credentials = credentials;
        this.authorities = authorities;
        this.authenticated = true;
    }

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return authorities;
    }

    @Override
    public Object getCredentials() {
        return credentials;
    }

    @Override
    public Object getDetails() {
        return null;
    }

    @Override
    public Object getPrincipal() {
        return principal;
    }

    @Override
    public boolean isAuthenticated() {
        return authenticated;
    }

    @Override
    public void setAuthenticated(boolean isAuthenticated) throws IllegalArgumentException {
        this.authenticated = isAuthenticated;
    }

    @Override
    public String getName() {
        if (principal instanceof org.springframework.security.core.userdetails.UserDetails) {
            return ((org.springframework.security.core.userdetails.UserDetails) principal).getUsername();
        }
```

```
        if (principal instanceof com.example.EmployeeManagementSystem.Entity.Employee) {
            return ((com.example.EmployeeManagementSystem.Entity.Employee) principal).getUsername();
        }
        if (principal instanceof com.example.EmployeeManagementSystem.Entity.Vendor) {
            return ((com.example.EmployeeManagementSystem.Entity.Vendor) principal).getUsername();
        }
        return principal != null ? principal.toString() : apiKey;
    }

    public String getApiKey() {
        return apiKey;
    }
}
```

79 Current: src\main\java\com\example\EmployeeManagementSystem\Service\CombinedUserDetailService.java

```java
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.Vendor;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.VendorRepo;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

import java.util.Optional;

@Service("combinedUserDetailService")
public class CombinedUserDetailService implements UserDetailsService {

    private final VendorRepo vendorRepository;
    private final EmployeeRepo employeeRepository;

    public CombinedUserDetailService(VendorRepo vendorRepository, EmployeeRepo employeeRepository) {
        this.vendorRepository = vendorRepository;
        this.employeeRepository = employeeRepository;
    }

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        Optional<Employee> employee = employeeRepository.findByEmail(username);
        if (employee.isPresent()) {
            return employee.get(); // must implement UserDetails
        }

        // Then try Vendor
        Optional<Vendor> vendor = vendorRepository.findByEmail(username);
        if (vendor.isPresent()) {
            return vendor.get();
        }

        throw new UsernameNotFoundException("User not found: " + username);
    }
}

```

80 Current: src\main\java\com\example\EmployeeManagementSystem\Service\DeliveryService.java

166 lines

```java
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.DTO.VendorDTO;
import com.example.EmployeeManagementSystem.DTO.VendorRequest;
import com.example.EmployeeManagementSystem.Entity.Delivery;
import com.example.EmployeeManagementSystem.Entity.Subscription;
import com.example.EmployeeManagementSystem.Entity.Vendor;
import com.example.EmployeeManagementSystem.Enum.DeliveryStatus;
import com.example.EmployeeManagementSystem.Enum.SubscriptionStatus;
import com.example.EmployeeManagementSystem.Exception.VendorNotFoundException;
import com.example.EmployeeManagementSystem.Repository.DeliveryRepository;
import com.example.EmployeeManagementSystem.Repository.SubscriptionRepository;
import com.example.EmployeeManagementSystem.Repository.VendorRepo;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.security.core.Authentication;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.Instant;
import java.util.List;
import java.util.stream.Collectors;

@Service
public class DeliveryService {
    private final SubscriptionRepository subscriptionRepository;
    private final DeliveryRepository deliveryRepository;
    private final DeliveryTimeService deliveryTimeService;
    private static final Logger logger = LoggerFactory.getLogger(DeliveryService.class);

    public DeliveryService(SubscriptionRepository subscriptionRepository,
                           DeliveryRepository deliveryRepository,
                           DeliveryTimeService deliveryTimeService) {
        this.subscriptionRepository = subscriptionRepository;
        this.deliveryRepository = deliveryRepository;
        this.deliveryTimeService = deliveryTimeService;
    }

    // AFTER
    @Transactional
    public void processDueDeliveries() {
        Instant now = Instant.now();
        List<Subscription> dueSubscriptions = subscriptionRepository
                .findDueSubscriptions(SubscriptionStatus.ACTIVE.name(), now);

        for (Subscription subscription : dueSubscriptions) {
            try {
                Delivery delivery = new Delivery();
                delivery.setSubscription(subscription);
                delivery.setMealSlot(subscription.getSlot());
                delivery.setScheduledDeliveryTime(subscription.getNextDeliveryTime());
                delivery.setStatus(DeliveryStatus.IN_PROGRESS);
                deliveryRepository.save(delivery);

                Instant nextDelivery = deliveryTimeService.getNextDeliveryTime(subscription);
                subscription.setNextDeliveryTime(nextDelivery);
                subscriptionRepository.save(subscription);

            } catch (DataIntegrityViolationException e) {
                // Duplicate delivery already exists — just advance the next delivery time
                logger.warn("Duplicate delivery skipped for subscription ID: {}", subscription.getId());
                Instant nextDelivery = deliveryTimeService.getNextDeliveryTime(subscription);
                subscription.setNextDeliveryTime(nextDelivery);
                subscriptionRepository.save(subscription);
            }
        }
    }

    @Transactional
```

```
    public Delivery updateDeliveryStatus(Long deliveryId, DeliveryStatus status) {
        Delivery delivery = deliveryRepository.findById(deliveryId)
                .orElseThrow(() -> new RuntimeException("Delivery not found"));

        delivery.setStatus(status);
        if (status == DeliveryStatus.DELIVERED) {
            delivery.setActualDeliveryTime(Instant.now());
        }

        return deliveryRepository.save(delivery);
    }

    public List<Delivery> getDeliveriesBySubscription(Long subscriptionId) {
        Subscription subscription = subscriptionRepository.findById(subscriptionId)
                .orElseThrow(() -> new RuntimeException("Subscription not found"));
        return deliveryRepository.findRecentDeliveriesBySubscription(subscription,
DeliveryStatus.SCHEDULED);
    }

    @Service
    public static class VendorService {

        private static final Logger log = LoggerFactory.getLogger(VendorService.class);
        private final VendorRepo vendorRepo;
        private final PasswordEncoder passwordEncoder;

        public VendorService(VendorRepo vendorRepo, PasswordEncoder passwordEncoder) {
            this.vendorRepo = vendorRepo;
            this.passwordEncoder = passwordEncoder;
        }

        public VendorDTO registerVendor(VendorRequest request) {
            if (vendorRepo.existsByEmail(request.getEmail())) {
                throw new IllegalArgumentException(
                        "A vendor with email " + request.getEmail() + " already exists");
            }
            Vendor vendor = new Vendor();
            vendor.setName(request.getName());
            vendor.setEmail(request.getEmail());
            vendor.setPhone(request.getPhone());
            vendor.setPassword(passwordEncoder.encode(request.getPassword()));
            Vendor saved = vendorRepo.save(vendor);
            log.info("Vendor registered: {} (id={})", saved.getName(), saved.getId());
            return toDTO(saved);
        }

        public VendorDTO getVendor(Authentication authentication) {
            return toDTO(vendorRepo.findByEmail(authentication.getName()).orElseThrow(
                    ()->new VendorNotFoundException("Cannot get others accoun")
            ));
        }

        public List<VendorDTO> getAllVendors() {
            return vendorRepo.findAll().stream()
                    .map(this::toDTO)
                    .collect(Collectors.toList());
        }

        public VendorDTO updateVendor(VendorRequest request,Authentication authentication) {
            Vendor vendor = vendorRepo.findByEmail(authentication.getName()).orElseThrow(
                    ()->new VendorNotFoundException("Cannot update others account")
            );
            if (request.getName() != null) vendor.setName(request.getName());
            if (request.getPhone() != null) vendor.setPhone(request.getPhone());
            if (request.getPassword() != null)
vendor.setPassword(passwordEncoder.encode(request.getPassword()));
            // Email update not allowed to avoid breaking restaurant ownership references
            return toDTO(vendorRepo.save(vendor));
        }

        public void deleteVendor(Authentication authentication) {
            Vendor vendor = vendorRepo.findByEmail(authentication.getName()).orElseThrow(
                    ()->new VendorNotFoundException("Cannot delete others account")
            );
            vendorRepo.delete(vendor);
            log.info("Vendor deleted: email={}", authentication.getName());
```

```
        }

        //  helpers

        private Vendor findOrThrow(Long id) {
            return vendorRepo.findById(id)
                    .orElseThrow(() -> new VendorNotFoundException("Vendor with id " + id + " not found"));
        }

        private VendorDTO toDTO(Vendor vendor) {
            VendorDTO dto = new VendorDTO();
            dto.setId(vendor.getId());
            dto.setName(vendor.getName());
            dto.setEmail(vendor.getEmail());
            dto.setPhone(vendor.getPhone());
            dto.setRegisteredAt(vendor.getRegisteredAt());
            return dto;
        }
    }
}
```

81 Current: src\main\java\com\example\EmployeeManagementSystem\Service\DeliveryTimeService.java

74 lines

```java
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.DTO.SubscriptionRequest;
import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.Subscription;
import com.example.EmployeeManagementSystem.Enum.MealSlot;
import com.example.EmployeeManagementSystem.Enum.ScheduleType;
import org.springframework.stereotype.Service;

 import java.time.*;

 @Service
 public class DeliveryTimeService {

     private final MealSlotService mealSlotService;

     public DeliveryTimeService(MealSlotService mealSlotService) {
         this.mealSlotService = mealSlotService;
     }
     public Instant calculateNextDelivery(SubscriptionRequest request, MealSlot slot, Employee employee) {
         ZoneId userZone=ZoneId.of(employee.getTimezone());
         if (request.getScheduleType() == ScheduleType.DAILY) {
             return calculateDailyNextDelivery(slot,userZone);
         } else {
             return calculateWeeklyNextDelivery(slot, request.getDayOfWeek(),userZone);
         }
     }

     private Instant calculateDailyNextDelivery(MealSlot slot, ZoneId userZone) {
         LocalTime slotTime = mealSlotService.getTime(slot);
         ZonedDateTime now = ZonedDateTime.now(userZone);
         ZonedDateTime deliveryTime;

         if (now.toLocalTime().isBefore(slotTime)) {
             deliveryTime=ZonedDateTime.of(now.toLocalDate(),slotTime,userZone);
         } else {
             deliveryTime=ZonedDateTime.of(now.toLocalDate().plusDays(1), slotTime,userZone);
         }
         return deliveryTime.toInstant();
     }

     private Instant calculateWeeklyNextDelivery(MealSlot slot, DayOfWeek dayOfWeek,ZoneId userZone) {
         LocalTime slotTime = mealSlotService.getTime(slot);
         ZonedDateTime now=ZonedDateTime.now(userZone);
         DayOfWeek today = now.getDayOfWeek();

         int daysToAdd = dayOfWeek.getValue() - today.getValue();

         if (daysToAdd == 0) {
             if (now.toLocalTime().isBefore(slotTime)) {
                 return ZonedDateTime.of(now.toLocalDate(), slotTime,userZone).toInstant();
             } else {
                 return ZonedDateTime.of(now.toLocalDate().plusDays(7), slotTime,userZone).toInstant();
             }
         }
         if (daysToAdd < 0) {
             daysToAdd += 7;
         }

         ZonedDateTime deliveryTime = ZonedDateTime.of(
                 now.toLocalDate().plusDays(daysToAdd),slotTime,userZone);
         return deliveryTime.toInstant();
     }

     public Instant getNextDeliveryTime(Subscription subscription) {
         ZoneId userZone=ZoneId.of(subscription.getEmployee().getTimezone());
         if (subscription.getScheduleType() == ScheduleType.DAILY) {
             return calculateDailyNextDelivery(subscription.getSlot(),userZone);
         } else {
             return calculateWeeklyNextDelivery(subscription.getSlot(),
subscription.getDayOfWeek(),userZone);
```

```
        }
    }
}

```

82 Current: src\main\java\com\example\EmployeeManagementSystem\Service\EmployeeService.java

134 lines

```java
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.DTO.EmployeeDTO;
import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
import com.example.EmployeeManagementSystem.Enum.Role;
import com.example.EmployeeManagementSystem.Enum.Status;
import com.example.EmployeeManagementSystem.Exception.EmployeeNotFound;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
import org.jspecify.annotations.Nullable;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

import java.time.ZoneId;
import java.util.List;

@Service
public class EmployeeService {
    private static final Logger log = LoggerFactory.getLogger(EmployeeService.class);
    private final EmployeeRepo employeeRepo;
    private final LeaveRequestRepo leaveRequestRepo;
    private final PasswordEncoder passwordEncoder;

    public EmployeeService(EmployeeRepo employeeRepo, LeaveRequestRepo leaveRequestRepo, PasswordEncoder passwordEncoder) {
        this.employeeRepo = employeeRepo;
        this.leaveRequestRepo=leaveRequestRepo;
        this.passwordEncoder = passwordEncoder;
    }

    public List<Employee> getAllEmployees() {
        return employeeRepo.findAll();
    }

    public Employee createEmployee(EmployeeDTO employeeDTO) {
        log.info("Creating new employee with email: {}", employeeDTO.getEmail());
        var employee = new Employee();
        if (employeeDTO.getName() != null)  employee.setName(employeeDTO.getName());
        if (employeeDTO.getEmail() != null) employee.setEmail(employeeDTO.getEmail());
        if (employeeDTO.getDept() != null)  employee.setDept(employeeDTO.getDept());
        if (employeeDTO.getTimezone() != null && isValidTimezone(employeeDTO.getTimezone())) {
            employee.setTimezone(employeeDTO.getTimezone());
        } else {
            employee.setTimezone("UTC");
        }
        employee.setRole(Role.EMPLOYEE);
        if(employeeDTO.getPassword()==null){
            throw new RuntimeException("Passowrd is required");
        }
        employee.setPassword(passwordEncoder.encode(employeeDTO.getPassword()));
        return employeeRepo.save(employee);
    }

    public Employee createManager(EmployeeDTO employeeDTO) {
        log.info("Creating new employee with email: {}", employeeDTO.getEmail());
        var employee = new Employee();
        if (employeeDTO.getName() != null)  employee.setName(employeeDTO.getName());
        if (employeeDTO.getEmail() != null) employee.setEmail(employeeDTO.getEmail());
        if (employeeDTO.getDept() != null)  employee.setDept(employeeDTO.getDept());
        if (employeeDTO.getTimezone() != null && isValidTimezone(employeeDTO.getTimezone())) {
            employee.setTimezone(employeeDTO.getTimezone());
        } else {
            employee.setTimezone("UTC");
        }
        employee.setRole(Role.MANAGER);
        if(employeeDTO.getPassword()==null){
```

```
            throw new RuntimeException("Passowrd is required");
        }
        employee.setPassword(passwordEncoder.encode(employeeDTO.getPassword()));
        return employeeRepo.save(employee);
    }

    public Employee updateEmployee(long id, EmployeeDTO employee) {
        var updateEmployee = employeeRepo.findById(id).orElseThrow(
                () -> new EmployeeNotFound("Employee with id:" + id + " not found")
        );

        if (employee.getName() != null)     updateEmployee.setName(employee.getName());
        if (employee.getEmail() != null)    updateEmployee.setEmail(employee.getEmail());
        if (employee.getDept() != null)     updateEmployee.setDept(employee.getDept());
        if (employee.getTimezone() != null && isValidTimezone(employee.getTimezone())) {
            updateEmployee.setTimezone(employee.getTimezone());
        }
        return employeeRepo.save(updateEmployee);
    }

    public void deleteEmployee(long id) {
        var employee = employeeRepo.findById(id).orElseThrow(
                () -> new EmployeeNotFound("Employee with id:" + id + " not found")
        );
        List<LeaveRequest> leaveRequests=leaveRequestRepo.findByEmployeeOrderByStartDateDesc(employee);
        for(LeaveRequest leaveRequest:leaveRequests){
            leaveRequestRepo.delete(leaveRequest);
        }
        employeeRepo.delete(employee);
    }

    public ResponseEntity<String> inactivateUser(long employeeId) {

        var employee = employeeRepo.findById(employeeId).orElseThrow(
                () -> new EmployeeNotFound("Employee with id:" + employeeId + " not found")
        );

        if (employee.getStatus() == Status.INACTIVE) {
            return ResponseEntity.badRequest()
                    .body("Status of the employee with id: " + employeeId + " is already set to inactive");
        }
        employee.setStatus(Status.INACTIVE);
        employeeRepo.save(employee);
        return ResponseEntity.ok("Status of the employee with id: " + employeeId + " is set to inactive");
    }

    /** Validate that the given string is a recognized IANA timezone id. */
    private boolean isValidTimezone(String tz) {
        try {
            ZoneId.of(tz);
            return true;
        } catch (Exception e) {
            return false;
        }
    }

    public Employee getAccount(Authentication authentication) {
        Employee employee=employeeRepo.findByEmail( authentication.getName()).orElseThrow(
                ()-> new EmployeeNotFound("Employee :"+authentication.getName()+" NOT FOUND")

        );
        return employee;
    }
}
```

83 Current: src\main\java\com\example\EmployeeManagementSystem\Service\LeaveRequestService.java

225 lines

```java
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.DTO.ActionDTO;
import com.example.EmployeeManagementSystem.DTO.AuthRequest;
import com.example.EmployeeManagementSystem.DTO.LeaveRequestDTO;
import com.example.EmployeeManagementSystem.DTO.LeaveResponseDTO;
import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
import com.example.EmployeeManagementSystem.Enum.LeaveType;
import com.example.EmployeeManagementSystem.Enum.Role;
import com.example.EmployeeManagementSystem.Enum.Status;
import com.example.EmployeeManagementSystem.Exception.*;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.stereotype.Service;

import java.time.LocalDate;
import java.time.ZoneId;
import java.time.temporal.ChronoUnit;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

@Service
public class LeaveRequestService {
    private final LeaveRequestRepo leaveRequestRepo;
    private final EmployeeRepo employeeRepo;
    private static final Logger log = LoggerFactory.getLogger(LeaveRequestService.class);
    public LeaveRequestService(LeaveRequestRepo leaveRequestRepo, EmployeeRepo employeeRepo) {
        this.leaveRequestRepo = leaveRequestRepo;
        this.employeeRepo = employeeRepo;
    }

    public List<LeaveResponseDTO> getAllTheLeaveRequest(){
        List<LeaveRequest> requestList=leaveRequestRepo.findAll();
        List<LeaveResponseDTO> dtos=new ArrayList<>();
        for(LeaveRequest request:requestList){
           dtos.add(convertToDTO(request));
        }
        return dtos;
    }


    public ResponseEntity<?> createRequest(Authentication authentication, LeaveRequestDTO requestDTO){
        var leaveRequest=new LeaveRequest();

        //Validate employee exists and is active
        var employee=employeeRepo.findByEmail(authentication.getName()).orElseThrow(()->
                new EmployeeNotFound("Employee with name: "+authentication.getName()+" is not found")
        );
        if (employee.getStatus() != Status.ACTIVE) {
            return ResponseEntity.badRequest()
                    .body(Map.of("error", "Only active employees can apply for leave"));
        }

        leaveRequest.setEmployee(employee);
        //Checking if the start date is before today
        String timezone= employee.getTimezone();
        ZoneId zoneId=ZoneId.of(timezone != null && !timezone.isBlank()
                ? timezone
                : "UTC");
        LocalDate today = LocalDate.now(zoneId);
        if (requestDTO.getStartDate().isBefore(today)) {
            throw new InvalidStartDateException("Start date cannot be before current date");
```

```
        }
        //Checking if the end date is before start date
        if (requestDTO.getEndDate().isBefore(requestDTO.getStartDate())) {
            throw new InvalidEndDateException("End date must be equal to or greater than start date");
        }
        //optional
        long daysRequested = ChronoUnit.DAYS.between(requestDTO.getStartDate(), requestDTO.getEndDate()) +
1;
        if (daysRequested > 30) {
            return ResponseEntity.badRequest()
                    .body(Map.of("error", "Leave cannot exceed 30 consecutive days"));
        }
        //optional
        if (requestDTO.getLeaveType() == LeaveType.SICK) {
            long sickDaysUsed = leaveRequestRepo.countDaysByEmployeeAndLeaveTypeAndYear(
                    employee, LeaveType.SICK, today.getYear());

            if (sickDaysUsed + daysRequested > 12) { // Assuming 12 sick days per year
                return ResponseEntity.badRequest()
                        .body(Map.of("error", "Sick leave limit exceeded for this year"));
            }
        }
        long duplicateCount= leaveRequestRepo.checkDuplicate(
                employee.getEmployeeId(),
                requestDTO.getStartDate(),
                requestDTO.getEndDate(),
                LeaveStatus.PENDING.name()
        );
        if(duplicateCount>0){
             throw new DuplicateRequestException("Duplicate leave request");
         }

         //checking if there is any overlapping leave request of that employee
         long count= leaveRequestRepo.countOverlappingLeave(
                 employee.getEmployeeId(),
                 requestDTO.getStartDate(),
                 requestDTO.getEndDate()
         );
          if(count>0){
              throw new OverlappingLeaveException("Overlapping leave exists");
          }
          leaveRequest.setStartDate(requestDTO.getStartDate());
          leaveRequest.setEndDate(requestDTO.getEndDate());

          if(requestDTO.getLeaveType()==null){
              return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                      .body("Leave type is required..");
          }
          leaveRequest.setLeaveType(requestDTO.getLeaveType());
          if(requestDTO.getReason()!=null){
              leaveRequest.setReason(requestDTO.getReason());
          }
          LeaveRequest savedRequest=leaveRequestRepo.save(leaveRequest);
          return ResponseEntity.status(HttpStatus.CREATED).body(convertToDTO(savedRequest));
    }

    //Only for manager
    public List<LeaveResponseDTO> getAllThePendingLeaveRequests(){
        List<LeaveRequest> requestList=leaveRequestRepo.findByStatus(LeaveStatus.PENDING);
        List<LeaveResponseDTO> responseDTOS=new ArrayList<>();
        for(LeaveRequest request:requestList){
            responseDTOS.add(convertToDTO(request));
        }
        return responseDTOS;
    }

    public ResponseEntity<?> updateLeaveRequestStatus(ActionDTO actionDTO,Authentication authentication){
        Employee manager=employeeRepo.findByEmail(authentication.getName()).orElseThrow(
                ()->new EmployeeNotFound("Manager with name: "+authentication.getName()+" Not Found")
        );
        if (actionDTO.getAction() == null) {
            return ResponseEntity.badRequest().body("Action is required");
        }
        log.info("Updating leave request status for id: {}", actionDTO.getLeaveRequestId());

        var leaveRequest=leaveRequestRepo.findById(actionDTO.getLeaveRequestId()).orElseThrow(
```

```
                ()-> new LeaveRequestNotFoundException("LeaveRequest with id:"+actionDTO.getLeaveRequestId()+" not found")
        );

        if (leaveRequest.getStatus() != LeaveStatus.PENDING) {
            return ResponseEntity.badRequest()
                    .body("Only PENDING requests can be approved or rejected. Current status: "
                            + leaveRequest.getStatus());
        }

         if(actionDTO.getAction().equalsIgnoreCase("approved")){
             leaveRequest.setStatus(LeaveStatus.APPROVED);
         }
         else if(actionDTO.getAction().equalsIgnoreCase("rejected")){
             leaveRequest.setStatus(LeaveStatus.REJECTED);
         }
         else{
             return ResponseEntity.badRequest().body("Invalid action,Use APPROVED or REJECTED");
         }
         leaveRequest.setManager(manager.getName());
         if(actionDTO.getRemarks()!=null){
             leaveRequest.setRemarks(actionDTO.getRemarks());
         }
         return ResponseEntity.ok(leaveRequestRepo.save(leaveRequest));
    }

    public ResponseEntity<?> cancelLeaveRequest(Authentication authentication,long leaveId){
        var employee=employeeRepo.findByEmail(authentication.getName()).orElseThrow(
                ()->new EmployeeNotFound("Employee with email:"+authentication.getName()+" not found")
        );
        var leaveRequest=leaveRequestRepo.findById(leaveId).orElseThrow(
                ()->new LeaveRequestNotFoundException("Leave request with id:"+leaveId+" is not found")
        );
        if(employee.getEmployeeId()!=leaveRequest.getEmployee().getEmployeeId()){
            return ResponseEntity.badRequest().body("You cannot cancel others Leave request..");
        }
        String timezone= employee.getTimezone();
        ZoneId zoneId=ZoneId.of(timezone != null && !timezone.isBlank()
                ? timezone
                : "UTC");
        LocalDate today = LocalDate.now(zoneId);
        if(leaveRequest.getStartDate().isEqual(today)||leaveRequest.getStartDate().isBefore(today)){
            return ResponseEntity.badRequest().body("You cannot cancel leave request after the leave has started");
        }
        leaveRequest.setStatus(LeaveStatus.CANCELLED);
        leaveRequestRepo.save(leaveRequest);
        return ResponseEntity.ok("Leave Request with id:"+ leaveId+" Successfully cancelled");
    }

    private LeaveResponseDTO convertToDTO(LeaveRequest request){
        var responseDTO=new LeaveResponseDTO();
        responseDTO.setLeaveRequestId(request.getId());
        responseDTO.setEmployeeId(request.getEmployee().getEmployeeId());
        responseDTO.setLeaveType(request.getLeaveType());
        responseDTO.setStartDate(request.getStartDate());
        responseDTO.setEndDate(request.getEndDate());
        responseDTO.setReason(request.getReason());
        responseDTO.setStatus(request.getStatus());
        return responseDTO;
    }

    public ResponseEntity<?> getLeaveRequestsByEmployee(Authentication authentication) {
        log.info("Fetching leave requests for employee: {}", authentication.getName());

        Employee employee = employeeRepo.findByEmail(authentication.getName())
                .orElseThrow(() -> new EmployeeNotFound("Employee not found: " +authentication.getName()));

        List<LeaveRequest> requests = leaveRequestRepo.findByEmployeeOrderByStartDateDesc(employee);
        List<LeaveResponseDTO> response = requests.stream()
                .map(this::convertToDTO)
                .collect(Collectors.toList());

        return ResponseEntity.ok(Map.of(
                "employeeName", authentication.getName(),
```

```
                "totalRequests", response.size(),
                "requests", response
        ));
    }
}

```

84 Current: src\main\java\com\example\EmployeeManagementSystem\Service\MealSlotService.java

22 lines

```java
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.Enum.MealSlot;
import org.springframework.stereotype.Service;

import java.time.LocalTime;
import java.util.Map;

@Service
 public class MealSlotService {

     private final Map<MealSlot, LocalTime> slotLocalTimeMap = Map.of(
             MealSlot.BREAKFAST, LocalTime.of(8, 0),
             MealSlot.LUNCH, LocalTime.of(13, 0),
             MealSlot.DINNER, LocalTime.of(21, 0)
     );

     public LocalTime getTime(MealSlot slot) {
         return slotLocalTimeMap.get(slot);
     }
 }

```

85 Current: src\main\java\com\example\EmployeeManagementSystem\Service\RefreshTokenService.java

43 lines

```java
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.RefreshToken;
import com.example.EmployeeManagementSystem.Entity.Vendor;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
import com.example.EmployeeManagementSystem.Repository.VendorRepo;
import org.springframework.stereotype.Service;

 import java.time.Instant;
 import java.time.temporal.ChronoUnit;
 import java.util.UUID;

 @Service
 public class RefreshTokenService {
     private final RefreshTokenRepository refreshTokenRepository;
     private final EmployeeRepo employeeRepo;
     private final VendorRepo vendorRepo;

     public RefreshTokenService(RefreshTokenRepository refreshTokenRepository, EmployeeRepo employeeRepo, VendorRepo vendorRepo) {
         this.refreshTokenRepository = refreshTokenRepository;
         this.employeeRepo = employeeRepo;
         this.vendorRepo= vendorRepo;
     }

     public RefreshToken createForEmployee(Employee employee) {
         RefreshToken refreshToken = new RefreshToken();
         refreshToken.setToken(UUID.randomUUID().toString());
         refreshToken.setExpiryTime(Instant.now().plus(7, ChronoUnit.DAYS));
         refreshToken.setEmployee(employee);
         return refreshToken;
     }

     public RefreshToken createForVendor(Vendor vendor) {
         RefreshToken refreshToken = new RefreshToken();
         refreshToken.setToken(UUID.randomUUID().toString());
         refreshToken.setExpiryTime(Instant.now().plus(7, ChronoUnit.DAYS));
         refreshToken.setVendor(vendor);
         return refreshToken;
     }
 }

```

```java
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.DTO.RestaurantDTO;
import com.example.EmployeeManagementSystem.DTO.RestaurantRequest;
import com.example.EmployeeManagementSystem.Entity.Restaurant;
import com.example.EmployeeManagementSystem.Entity.Vendor;
import com.example.EmployeeManagementSystem.Enum.MealSlot;
import com.example.EmployeeManagementSystem.Exception.RestaurantNotFoundException;
import com.example.EmployeeManagementSystem.Exception.UnauthorizedAccessException;
import com.example.EmployeeManagementSystem.Exception.VendorNotFoundException;
import com.example.EmployeeManagementSystem.Repository.RestaurantRepository;
import com.example.EmployeeManagementSystem.Repository.VendorRepo;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.security.core.Authentication;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.stream.Collectors;

@Service
public class RestaurantService {

    private static final Logger log = LoggerFactory.getLogger(RestaurantService.class);

    private final RestaurantRepository restaurantRepository;
    private final VendorRepo vendorRepo;

    public RestaurantService(RestaurantRepository restaurantRepository, VendorRepo vendorRepo) {
        this.restaurantRepository = restaurantRepository;
        this.vendorRepo = vendorRepo;
    }

    /**
     * Create a new restaurant.
     * Only the owning vendor (matched by vendorId in the request) may do this.
     */
    public RestaurantDTO createRestaurant(RestaurantRequest request,Authentication authentication) {
        Vendor vendor=vendorRepo.findByEmail(authentication.getName())
                .orElseThrow(() -> new VendorNotFoundException(
                        "Vendor with email " + authentication.getName() + " not found"));

        Restaurant restaurant = new Restaurant();
        restaurant.setName(request.getName());
        restaurant.setAddress(request.getAddress());
        restaurant.setDescription(request.getDescription());
        restaurant.setSupportedMealSlots(request.getSupportedMealSlots());
        restaurant.setVendor(vendor);
        restaurant.setActive(true);

        Restaurant saved = restaurantRepository.save(restaurant);
        log.info("Restaurant created: {} by vendor {}", saved.getName(), vendor.getId());
        return toDTO(saved);
    }

    /**
     * Update restaurant details.
     * Vendor must own this restaurant — otherwise 403.
     */
    public RestaurantDTO updateRestaurant(Long restaurantId, RestaurantRequest request,Authentication
authentication) {
        Restaurant restaurant = findOrThrow(restaurantId);
        Vendor vendor=vendorRepo.findByEmail(authentication.getName()).orElseThrow(
                ()->new VendorNotFoundException("Vendor Not found")
        );
        assertOwnership(restaurantId, vendor.getEmail());

        if (request.getName() != null) restaurant.setName(request.getName());
        if (request.getAddress() != null) restaurant.setAddress(request.getAddress());
        if (request.getDescription() != null) restaurant.setDescription(request.getDescription());
        if (request.getSupportedMealSlots() != null && !request.getSupportedMealSlots().isEmpty()) {
```

```
            restaurant.setSupportedMealSlots(request.getSupportedMealSlots());
        }
        return toDTO(restaurantRepository.save(restaurant));
    }

    /**
     * Deactivate a restaurant (soft delete).
     * Only the owning vendor may do this.
     * Active subscriptions to this restaurant will no longer receive new deliveries
     * — the Subscription status should be handled separately.
     */
    public void deactivateRestaurant(Long restaurantId, Authentication authentication) {
        Restaurant restaurant = findOrThrow(restaurantId);
        Vendor vendor=vendorRepo.findByEmail(authentication.getName()).orElseThrow(
                ()->new VendorNotFoundException("Vendor Not found")
        );
        assertOwnership(restaurantId, vendor.getEmail());
        restaurant.setActive(false);
        restaurantRepository.save(restaurant);
        log.info("Restaurant deactivated: id={}", restaurantId);
    }

    /**
     * Get all restaurants owned by a vendor.
     * VENDOR-only endpoint.
     */
    public List<RestaurantDTO> getRestaurantsByVendor(Authentication authentication) {
        vendorRepo.findByEmail(authentication.getName())
                .orElseThrow(() -> new VendorNotFoundException("Vendor " + authentication.getName() + " not found"));
        return restaurantRepository.findByVendor_EmailAndActiveTrue(authentication.getName())
                .stream().map(this::toDTO).collect(Collectors.toList());
    }

    /**
     * Get all active restaurants.
     * Available to EMPLOYEE so they can browse and choose.
     */
    public List<RestaurantDTO> getAllActiveRestaurants() {
        return restaurantRepository.findByActiveTrue()
                .stream().map(this::toDTO).collect(Collectors.toList());
    }

    /**
     * Get active restaurants that serve a specific meal slot.
     * Employees call this when subscribing, to see valid restaurant options for a slot.
     */
    public List<RestaurantDTO> getRestaurantsByMealSlot(MealSlot mealSlot) {
        return restaurantRepository.findActiveByMealSlot(mealSlot)
                .stream().map(this::toDTO).collect(Collectors.toList());
    }

    public RestaurantDTO getRestaurant(Long id) {
        return toDTO(findOrThrow(id));
    }

    //  helpers

    private Restaurant findOrThrow(Long id) {
        return restaurantRepository.findById(id)
                .orElseThrow(() -> new RestaurantNotFoundException(
                        "Restaurant with id " + id + " not found"));
    }

    /**
     * Throws UnauthorizedAccessException if the restaurant does not belong to the vendor.
     * This is the core access-control guard for all mutating vendor operations.
     */
    private void assertOwnership(Long restaurantId, String email) {
        if (!restaurantRepository.existsByIdAndVendor_Email(restaurantId, email)) {
            throw new UnauthorizedAccessException(
                    "Vendor " + email + " does not own restaurant " + restaurantId);
        }
    }

    public RestaurantDTO toDTO(Restaurant r) {
```

```
        RestaurantDTO dto = new RestaurantDTO();
        dto.setId(r.getId());
        dto.setName(r.getName());
        dto.setAddress(r.getAddress());
        dto.setDescription(r.getDescription());
        dto.setSupportedMealSlots(r.getSupportedMealSlots());
        dto.setActive(r.isActive());
        dto.setVendorId(r.getVendor().getId());
        dto.setVendorName(r.getVendor().getName());
        return dto;
    }
}
```

87 Current: src\main\java\com\example\EmployeeManagementSystem\Service\SubscriptionService.java

225 lines

```java
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.DTO.SubscriptionDTO;
import com.example.EmployeeManagementSystem.DTO.SubscriptionRequest;
import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.Restaurant;
import com.example.EmployeeManagementSystem.Entity.Subscription;
import com.example.EmployeeManagementSystem.Enum.MealSlot;
import com.example.EmployeeManagementSystem.Enum.ScheduleType;
import com.example.EmployeeManagementSystem.Enum.SubscriptionStatus;
import com.example.EmployeeManagementSystem.Exception.EmployeeNotFound;
import com.example.EmployeeManagementSystem.Exception.RestaurantNotFoundException;
import com.example.EmployeeManagementSystem.Exception.SubscriptionAlreadyExists;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.RestaurantRepository;
import com.example.EmployeeManagementSystem.Repository.SubscriptionRepository;
import org.springframework.security.core.Authentication;
import org.springframework.stereotype.Service;

import java.time.ZoneId;
import java.time.ZonedDateTime;
import java.util.ArrayList;
import java.util.List;

@Service
public class SubscriptionService {

    private final SubscriptionRepository subscriptionRepository;
    private final EmployeeRepo employeeRepo;
    private final DeliveryTimeService deliveryTimeService;
    private final RestaurantRepository restaurantRepository;

    public SubscriptionService(SubscriptionRepository subscriptionRepository,
                               EmployeeRepo employeeRepo,
                               DeliveryTimeService deliveryTimeService,
                               RestaurantRepository restaurantRepository) {
        this.subscriptionRepository = subscriptionRepository;
        this.employeeRepo = employeeRepo;
        this.deliveryTimeService = deliveryTimeService;
        this.restaurantRepository = restaurantRepository;
    }

    /**
     * Add subscriptions for an employee.
     *
     * Each SlotOrder specifies a meal slot AND the restaurant for that slot,
     * so an employee can have breakfast from Restaurant A and lunch from Restaurant B.
     *
     * Validations:
     * - Employee must exist and be ACTIVE
     * - Restaurant must exist and be active
     * - Restaurant must support the requested meal slot
     * - No duplicate active subscription for the same employee + slot combination
     */
    public void addSubscription(SubscriptionRequest request,Authentication authentication) {
        Employee employee = employeeRepo.findByEmail(authentication.getName())
                .orElseThrow(() -> new EmployeeNotFound(
                        "Employee with id " + authentication.getName() + " not found"));

        if (request.getSlotOrders() == null || request.getSlotOrders().isEmpty()) {
            throw new IllegalArgumentException("slotOrders cannot be null or empty");
        }

        if (request.getScheduleType() == ScheduleType.DAILY) {
            request.setDayOfWeek(null);
        } else if (request.getScheduleType() == ScheduleType.WEEKLY) {
            if (request.getDayOfWeek() == null) {
                throw new IllegalArgumentException("dayOfWeek is required for WEEKLY schedule");
            }
        }

```

```java
        for (SubscriptionRequest.SlotOrder slotOrder : request.getSlotOrders()) {
            MealSlot slot = slotOrder.getMealSlot();
            Long restaurantId = slotOrder.getRestaurantId();

            // 1. Validate restaurant exists and is active
            Restaurant restaurant = restaurantRepository.findById(restaurantId)
                    .orElseThrow(() -> new RestaurantNotFoundException(
                            "Restaurant with id " + restaurantId + " not found"));
            if (!restaurant.isActive()) {
                throw new IllegalArgumentException(
                        "Restaurant '" + restaurant.getName() + "' is not currently active");
            }

            // 2. Validate the restaurant actually serves this meal slot
            if (!restaurant.getSupportedMealSlots().contains(slot)) {
                throw new IllegalArgumentException(
                        "Restaurant '" + restaurant.getName() + "' does not serve " + slot +
                                ". Supported slots: " + restaurant.getSupportedMealSlots());
            }

            // 3. Check for duplicate active subscription (same employee + slot)
            if (subscriptionRepository.checkSubscriptionExists(
                    employee.getEmployeeId(), slot.name(), SubscriptionStatus.ACTIVE.name()) > 0) {
                throw new SubscriptionAlreadyExists(
                        "Employee " + employee.getEmployeeId() +
                                " already has an active " + slot + " subscription. " +
                                "Cancel the existing one before subscribing to a new restaurant.");
            }

            // 4. Create subscription
            Subscription subscription = new Subscription();
            subscription.setEmployee(employee);
            subscription.setSlot(slot);
            subscription.setRestaurant(restaurant);
            subscription.setScheduleType(request.getScheduleType());
            subscription.setDayOfWeek(request.getDayOfWeek());
            subscription.setStatus(SubscriptionStatus.ACTIVE);
            subscription.setNextDeliveryTime(
                    deliveryTimeService.calculateNextDelivery(request, slot, employee));

            subscriptionRepository.save(subscription);
        }
    }

    public List<SubscriptionDTO> getAllSubscriptions() {
        List<Subscription> subscriptions = subscriptionRepository.findAll();
        List<SubscriptionDTO> dtoList = new ArrayList<>();
        for (Subscription s : subscriptions) {
            dtoList.add(convertToDTO(s));
        }
        return dtoList;
    }

    public Subscription updateSubscription(Long id, SubscriptionRequest request) {
        Subscription subscription = subscriptionRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("Subscription not found"));
        Employee employee = subscription.getEmployee();

        if (request.getScheduleType() == ScheduleType.DAILY) {
            subscription.setDayOfWeek(null);
        } else if (request.getScheduleType() == ScheduleType.WEEKLY) {
            if (request.getDayOfWeek() == null) {
                throw new IllegalArgumentException("dayOfWeek is required for WEEKLY schedule");
            }
            subscription.setDayOfWeek(request.getDayOfWeek());
        }
        subscription.setScheduleType(request.getScheduleType());

        // Update slot & restaurant if a single slotOrder is provided
        if (request.getSlotOrders() != null && !request.getSlotOrders().isEmpty()) {
            if (request.getSlotOrders().size() > 1) {
                throw new IllegalArgumentException(
                        "Update supports one slot at a time. Use separate subscriptions for multiple slots.");
            }
            SubscriptionRequest.SlotOrder slotOrder = request.getSlotOrders().get(0);
```

```
            MealSlot newSlot = slotOrder.getMealSlot();
            Restaurant restaurant = restaurantRepository.findById(slotOrder.getRestaurantId())
                    .orElseThrow(() -> new RestaurantNotFoundException(
                            "Restaurant " + slotOrder.getRestaurantId() + " not found"));
            if (!restaurant.isActive()) {
                throw new IllegalArgumentException("Restaurant '" + restaurant.getName() + "' is not active");
            }
            if (!restaurant.getSupportedMealSlots().contains(newSlot)) {
                throw new IllegalArgumentException(
                        "Restaurant '" + restaurant.getName() + "' does not serve " + newSlot);
            }
            subscription.setSlot(newSlot);
            subscription.setRestaurant(restaurant);
        }

        subscription.setNextDeliveryTime(
                deliveryTimeService.calculateNextDelivery(request, subscription.getSlot(), employee));

        return subscriptionRepository.save(subscription);
    }

    public Subscription pauseSubscription(Long id) {
        Subscription subscription = subscriptionRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("Subscription not found"));
        subscription.setStatus(SubscriptionStatus.PAUSED);
        subscription.setNextDeliveryTime(null);
        return subscriptionRepository.save(subscription);
    }

    public Subscription resumeSubscription(Long id) {
        Subscription subscription = subscriptionRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("Subscription not found"));
        subscription.setStatus(SubscriptionStatus.ACTIVE);
        subscription.setNextDeliveryTime(deliveryTimeService.getNextDeliveryTime(subscription));
        return subscriptionRepository.save(subscription);
    }

    public Subscription expireSubscription(Long id) {
        Subscription subscription = subscriptionRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("Subscription not found"));
        subscription.setStatus(SubscriptionStatus.EXPIRED);
        subscription.setNextDeliveryTime(null);
        return subscriptionRepository.save(subscription);
    }

    public List<SubscriptionDTO> getSubscriptionOfUser(Authentication authentication) {
        List<Subscription> subscriptions = subscriptionRepository.findByEmployee_Email(authentication.getName());
        List<SubscriptionDTO> dtoList = new ArrayList<>();
        for (Subscription s : subscriptions) {
            dtoList.add(convertToDTO(s));
        }
        return dtoList;
    }

    //  helpers

    private SubscriptionDTO convertToDTO(Subscription s) {
        SubscriptionDTO dto = new SubscriptionDTO();
        dto.setSubscriptionId(s.getId());
        dto.setUserId(s.getEmployee().getEmployeeId());
        dto.setMealSlot(s.getSlot());
        dto.setScheduleType(s.getScheduleType());
        if (s.getScheduleType() == ScheduleType.WEEKLY) {
            dto.setDayOfWeek(s.getDayOfWeek());
        }
        dto.setStatus(s.getStatus());
        if (s.getNextDeliveryTime() != null) {
            ZoneId userZone = ZoneId.of(s.getEmployee().getTimezone());
            dto.setNextDeliveryTime(s.getNextDeliveryTime().atZone(userZone));
        }
        // Restaurant info
        if (s.getRestaurant() != null) {
            dto.setRestaurantId(s.getRestaurant().getId());
            dto.setRestaurantName(s.getRestaurant().getName());
```

```
            dto.setRestaurantAddress(s.getRestaurant().getAddress());
        }
        return dto;
    }
}
```

88 Current: src\main\java\com\example\EmployeeManagementSystem\Service\TotpService.java

40 lines

```java
package com.example.EmployeeManagementSystem.Service;

import dev.samstevens.totp.code.*;
import dev.samstevens.totp.qr.*;
import dev.samstevens.totp.secret.DefaultSecretGenerator;
import dev.samstevens.totp.secret.SecretGenerator;
import dev.samstevens.totp.time.SystemTimeProvider;
import dev.samstevens.totp.time.TimeProvider;
import org.springframework.stereotype.Service;

@Service
public class TotpService {

    private final SecretGenerator secretGenerator = new DefaultSecretGenerator();
    private final TimeProvider timeProvider = new SystemTimeProvider();
    private final CodeGenerator codeGenerator = new DefaultCodeGenerator();
    private final CodeVerifier codeVerifier = new DefaultCodeVerifier(codeGenerator, timeProvider);

    /** Generate a new Base32 secret for a user */
    public String generateSecret() {
        return secretGenerator.generate();
    }

    /** Generate a QR code URI — user scans this with Google Authenticator */
    public String getQrCodeUri(String secret, String username) {
        return String.format(
                "otpauth://totp/EmployeeMS:%s?secret=%s&issuer=EmployeeManagementSystem",
                username, secret
        );
    }

    /** Verify the 6-digit code the user typed */
    public boolean verifyCode(String secret, String code) {
        try {
            return codeVerifier.isValidCode(secret, code);
        } catch (Exception e) {
            return false;
        }
    }
}
```

```java
package com.example.EmployeeManagementSystem.Util;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import jakarta.annotation.PostConstruct;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Component;

 import javax.crypto.SecretKey;
 import java.util.Date;

 @Component
 public class JWTUtil {

     // FIX 1: Was 1000*60*60 (1 hour) — caused token expiry mid-session.
     // Extended to 24 hours to match the session timeout in application.yaml.
     private final long Expiration_time = 1000L * 60 * 60 * 24;

     @Value("${jwt.secret}")
     private String secret;
     private SecretKey key;

     @PostConstruct
     public void init() {
         key = Keys.hmacShaKeyFor(secret.getBytes());
     }

     public String generateToken(String username) {
         return Jwts.builder()
                 .setSubject(username)
                 .setIssuedAt(new Date())
                 .setExpiration(new Date(System.currentTimeMillis() + Expiration_time))
                 .signWith(key)
                 .compact();
     }

     public String extractUsernameFromToken(String token) {
         Claims body = extractClaims(token);
         return body.getSubject();
     }

     private Claims extractClaims(String token) {
         return Jwts.parser()
                 .verifyWith(key)
                 .build()
                 .parseSignedClaims(token)
                 .getPayload();
     }

     public boolean validateToken(String username, UserDetails userDetails, String token) {
         return username.equals(userDetails.getUsername()) && !isTokenExpired(token);
     }

     private boolean isTokenExpired(String token) {
         return extractClaims(token).getExpiration().before(new Date());
     }
 }
```

90 Current: src\main\resources\application.yaml

66 lines

```yaml
spring:
  application:
    name: EmployeeManagementSystem

  datasource :
    url : jdbc:mysql://localhost:3306/employee_management_db
    username : ${USER_NAME}
    password : ${PASSWORD}

  jpa:
    hibernate :
      ddl-auto: update
    show-sql: true

  springdoc:
    api-docs:
      path: /v3/api-docs
      enabled: true
    swagger-ui:
      path: /swagger-ui.html
      enabled: true
      operations-sorter: method
      tags-sorter: alpha
      try-it-out-enabled: true
      filter: true
    show-actuator: true
  mvc:
    static-path-pattern: /**
    session:
      tracking-modes: COOKIE
  web:
    resources:
      static-locations: classpath:/static/
      add-mappings: true
  session:
    timeout: 86400s  # 24 hours in seconds (was 1440m which is incorrect format)
    store-type: none  # Use default cookie-based session

  servlet:
    session:
      cookie:
        http-only: true
        same-site: strict
        secure: false     # set true in production with HTTPS

  security:
    debug: true
    oauth2:
      client:
        registration:
          google:
            client-id: ${client_id}
            client-secret: ${client_secret}

logging:
  level:
    org.springframework.security: DEBUG
    com.example.EmployeeManagementSystem: DEBUG
jwt:
  secret: ${JWT_SECRET}
# Add this to your application.yaml
api:
  key:
    enabled: true
    header-name: X-API-Key

```

91 Current: src\main\resources\static\dashboard.html 660 lines

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>EMS - Complete Employee Management System</title>
    <link href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&family=DM+Mono:wght@400;500&display=swap" rel="stylesheet">
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        :root {
            --bg: #0d0f14; --bg2: #131720; --bg3: #1a2030;
            --panel: #1e2436; --panel2: #252c40;
            --border: rgba(255,255,255,0.07); --border2: rgba(255,255,255,0.12);
            --text: #e0e0e0; --text2: #8b92a8; --text3: #5557fa;
            --accent: #4f8cff; --accent-dim: rgba(79,140,255,0.12);
            --green: #36d399; --green-dim: rgba(54,211,153,0.12);
            --amber: #f9c74f; --amber-dim: rgba(249,199,79,0.12);
            --red: #f87171; --red-dim: rgba(248,113,113,0.12);
            --violet: #a78bfa; --violet-dim: rgba(167,139,250,0.12);
            --font: 'DM Sans', sans-serif; --mono: 'DM Mono', monospace;
        }
        body { font-family: var(--font); background: var(--bg); color: var(--text); font-size: 14px; line-height: 1.5; overflow: hidden; }
        .app { display: flex; height: 100vh; }
        .sidebar { width: 260px; background: var(--bg2); border-right: 1px solid var(--border); display: flex; flex-direction: column; overflow-y: auto; }
        .logo { padding: 20px; border-bottom: 1px solid var(--border); }
        .logo h2 { font-size: 20px; color: var(--text); }
        .logo span { color: var(--accent); }
        .logo p { font-size: 11px; color: var(--text3); margin-top: 4px; }
        .nav-item { padding: 12px 20px; margin: 4px 12px; border-radius: 8px; cursor: pointer; transition: all 0.2s; color: var(--text2); display: flex; align-items: center; gap: 10px; }
        .nav-item:hover, .nav-item.active { background: var(--accent-dim); color: var(--accent); }
        .main-content { flex: 1; display: flex; flex-direction: column; overflow: hidden; }
        .topbar { background: var(--bg2); padding: 16px 24px; display: flex; justify-content: space-between; align-items: center; border-bottom: 1px solid var(--border); }
        .page-title { font-size: 20px; font-weight: 600; }
        .user-info { display: flex; align-items: center; gap: 16px; }
        .logout-btn { padding: 8px 16px; background: var(--red-dim); border: 1px solid rgba(248,113,113,0.3); border-radius: 6px; color: var(--red); cursor: pointer; }
        .content { flex: 1; overflow-y: auto; padding: 24px; }
        .view { display: none; }
        .view.active { display: block; }
        .stats-grid { display: grid; grid-template-columns: repeat(4, 1fr); gap: 16px; margin-bottom: 24px; }
        .stat-card { background: var(--panel); padding: 20px; border-radius: 12px; border: 1px solid var(--border); }
        .stat-label { font-size: 11px; color: var(--text3); text-transform: uppercase; margin-bottom: 8px; }
        .stat-value { font-size: 28px; font-weight: 600; }
        .form-card { background: var(--panel); border-radius: 12px; border: 1px solid var(--border); padding: 24px; max-width: 600px; }
        .form-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; }
        .form-group { display: flex; flex-direction: column; gap: 6px; }
        .form-group.full { grid-column: 1 / -1; }
        label { font-size: 11px; font-weight: 600; color: var(--text3); text-transform: uppercase; }
        input, select, textarea { padding: 10px 12px; background: var(--bg3); border: 1px solid var(--border2); border-radius: 8px; color: var(--text); font-family: var(--font); font-size: 13px; }
        input:focus, select:focus, textarea:focus { outline: none; border-color: var(--accent); }
        .table-wrap { background: var(--panel); border-radius: 12px; border: 1px solid var(--border); overflow-x: auto; }
        table { width: 100%; border-collapse: collapse; }
        th { text-align: left; padding: 12px 16px; background: var(--bg3); color: var(--text3); font-size: 11px; font-weight: 600; text-transform: uppercase; }
        td { padding: 12px 16px; border-top: 1px solid var(--border); }
        .chip { padding: 4px 10px; border-radius: 20px; font-size: 11px; font-weight: 600; display: inline-block; }
        .chip.active, .chip.approved, .chip.delivered { background: var(--green-dim); color: var(--green); }
        .chip.pending, .chip.scheduled { background: var(--amber-dim); color: var(--amber); }
        .chip.inactive, .chip.rejected, .chip.cancelled, .chip.expired { background: var(--red-dim); color: var(--red); }
```

```
        .chip.on_leave { background: var(--violet-dim); color: var(--violet); }
        .chip.employee { background: var(--accent-dim); color: var(--accent); }
        .chip.manager { background: var(--violet-dim); color: var(--violet); }
        .chip.vendor { background: var(--red-dim); color: var(--red); }
        .btn { padding: 8px 16px; border-radius: 6px; border: none; font-family: var(--font); font-size: 13px; font-weight: 500; cursor: pointer; transition: all 0.2s; }
        .btn-primary { background: var(--accent); color: white; }
        .btn-primary:hover { background: #6ba0ff; }
        .btn-secondary { background: var(--bg3); color: var(--text2); border: 1px solid var(--border2); }
        .btn-danger { background: var(--red-dim); color: var(--red); border: 1px solid rgba(248,113,113,0.3); }
        .btn-sm { padding: 4px 10px; font-size: 11px; margin: 2px; }
        .section-head { display: flex; justify-content: space-between; align-items: center; margin-bottom: 16px; }
        .section-title { font-size: 16px; font-weight: 600; }
        .empty-state { text-align: center; padding: 40px; color: var(--text3); }
        .modal-overlay { display: none; position: fixed; top: 0; left: 0; right: 0; bottom: 0; background: rgba(0,0,0,0.7); z-index: 1000; align-items: center; justify-content: center; }
        .modal-overlay.open { display: flex; }
        .modal { background: var(--panel); border-radius: 12px; padding: 24px; width: 450px; max-width: 90%; }
        .modal-title { font-size: 18px; font-weight: 600; margin-bottom: 16px; }
        .modal-actions { display: flex; gap: 12px; margin-top: 20px; justify-content: flex-end; }
        .alert { position: fixed; top: 20px; right: 20px; padding: 12px 20px; border-radius: 8px; z-index: 1100; animation: slideIn 0.3s ease; }
        .alert-success { background: var(--green-dim); color: var(--green); border: 1px solid var(--green); }
        .alert-error { background: var(--red-dim); color: var(--red); border: 1px solid var(--red); }
        @keyframes slideIn { from { transform: translateX(100%); opacity: 0; } to { transform: translateX(0); opacity: 1; } }
        .action-row { display: flex; gap: 6px; flex-wrap: wrap; }
    </style>
</head>
<body>
<div class="app">
    <!-- Sidebar -->
    <div class="sidebar">
        <div class="logo">
            <h2>E<span>M</span>S</h2>
            <p>Employee Management System</p>
        </div>
        <div style="flex: 1;">
            <div class="nav-item active" onclick="showView('dashboard', event)">&#9632; Dashboard</div>
            <div class="nav-item" onclick="showView('employees', event)">&#9632; Employees</div>
            <div class="nav-item" onclick="showView('apply-leave', event)">&#9632; Apply Leave</div>
            <div class="nav-item" onclick="showView('my-leaves', event)">&#9632; My Leaves</div>
            <div class="nav-item" onclick="showView('pending-leaves', event)">&#9632; Pending Approvals</div>
            <div class="nav-item" onclick="showView('restaurants', event)">&#9632; Restaurants</div>
            <div class="nav-item" onclick="showView('subscriptions', event)">&#9632; My Subscriptions</div>
            <div class="nav-item" onclick="showView('add-subscription', event)">&#9632; Add Subscription</div>
            <div class="nav-item" onclick="showView('deliveries', event)">&#9632; Deliveries</div>
            <div class="nav-item" onclick="showView('vendors', event)">&#9632; Vendors</div>
        </div>
    </div>

    <!-- Main Content -->
    <div class="main-content">
        <div class="topbar">
            <div class="page-title" id="pageTitle">Dashboard</div>
            <div class="user-info">
                <span id="username-display"></span>
                <button class="logout-btn" onclick="logout()">Logout</button>
            </div>
        </div>
        <div class="content" id="content">

            <!-- DASHBOARD VIEW -->
            <div id="dashboard-view" class="view active">
                <div class="stats-grid">
                    <div class="stat-card"><div class="stat-label">Total Employees</div><div class="stat-value" id="total-employees">--</div></div>
                    <div class="stat-card"><div class="stat-label">Active</div><div class="stat-value" id="active-employees">--</div></div>
                    <div class="stat-card"><div class="stat-label">On Leave</div><div class="stat-value"
```

```
id="onleave-employees">--</div></div>
                    <div class="stat-card"><div class="stat-label">Pending Leaves</div><div class="stat-value" id="pending-leaves">--</div></div>
                </div>
                <div class="section-head"><div class="section-title">Recent Leave Requests</div><button class="btn btn-secondary btn-sm" onclick="loadDashboard()">Refresh</button></div>
                <div class="table-wrap"><table><thead><tr><th>Employee</th><th>Type</th><th>Start</th><th>End</th><th>Status</th></tr></thead><tbody id="recent-leaves"></tbody></table></div>
            </div>

            <!-- EMPLOYEES VIEW -->
            <div id="employees-view" class="view">
                <div class="section-head"><div class="section-title">All Employees</div><button class="btn btn-primary btn-sm" onclick="loadEmployees()">Refresh</button></div>
                <div class="table-wrap"><table><thead><tr><th>ID</th><th>Name</th><th>Email</th><th>Department</th><th>Role</th><th>Status</th><th>Timezone</th></tr></thead><tbody id="employees-lis...
            </div>

            <!-- APPLY LEAVE VIEW -->
            <div id="apply-leave-view" class="view">
                <div class="section-title" style="margin-bottom: 20px;">Apply for Leave</div>
                <div class="form-card">
                    <div class="form-grid">
                        <div class="form-group full"><label>Employee Email (authenticated)</label><input type="email" id="leave-email" placeholder="your.email@company.com"></div>
                        <div class="form-group"><label>Leave Type</label><select id="leave-type"><option value="SICK">Sick Leave</option><option value="CASUAL">Casual Leave</option><option value="E...
                        <div class="form-group"><label>Start Date</label><input type="date" id="leave-start"></div>
                        <div class="form-group"><label>End Date</label><input type="date" id="leave-end"></div>
                        <div class="form-group full"><label>Reason</label><textarea id="leave-reason" rows="3" placeholder="Please provide reason for leave..."></textarea></div>
                    </div>
                    <div class="modal-actions" style="margin-top: 20px;"><button class="btn btn-primary" onclick="applyLeave()">Submit Leave Request</button><button class="btn btn-secondary" onclic...
                </div>
            </div>

            <!-- MY LEAVES VIEW -->
            <div id="my-leaves-view" class="view">
                <div class="section-head"><div class="section-title">My Leave Requests</div><button class="btn btn-primary btn-sm" onclick="loadMyLeaves()">Refresh</button></div>
                <div class="table-wrap"><table><thead><tr><th>ID</th><th>Type</th><th>Start Date</th><th>End Date</th><th>Status</th><th>Reason</th><th>Action</th></tr></thead><tbody id="my-leaves-...
            </div>

            <!-- PENDING LEAVES VIEW -->
            <div id="pending-leaves-view" class="view">
                <div class="section-head"><div class="section-title">Pending Leave Approvals</div><button class="btn btn-primary btn-sm" onclick="loadPendingLeaves()">Refresh</button></div>
                <div class="table-wrap"><table><thead><tr><th>ID</th><th>Employee</th><th>Type</th><th>Start</th><th>End</th><th>Reason</th><th>Actions</th></tr></thead><tbody id="pending-leaves-li...
            </div>

            <!-- RESTAURANTS VIEW -->
            <div id="restaurants-view" class="view">
                <div class="section-head"><div class="section-title">Restaurants</div><div><select id="slot-filter" onchange="loadRestaurants()"><option value="">All Slots</option><option value="BR...
                <div class="table-wrap"><table><thead><tr><th>Name</th><th>Address</th><th>Meal Slots</th><th>Status</th><th>Vendor</th></tr></thead><tbody id="restaurants-list"></tbody></table></div>
            </div>

            <!-- MY SUBSCRIPTIONS VIEW -->
            <div id="subscriptions-view" class="view">
                <div class="section-head"><div class="section-title">My Meal Subscriptions</div><button class="btn btn-primary btn-sm" onclick="loadSubscriptions()">Refresh</button></div>
                <div class="table-wrap"><table><thead><tr><th>ID</th><th>Restaurant</th><th>Meal Slot</th><th>Schedule</th><th>Status</th><th>Next Delivery</th><th>Actions</th></tr></thead><tbody i...
            </div>

            <!-- ADD SUBSCRIPTION VIEW -->
            <div id="add-subscription-view" class="view">
```

```
                <div class="section-title" style="margin-bottom: 20px;">Create Meal Subscription</div>
                <div class="form-card">
                    <div class="form-grid">
                        <div class="form-group full"><label>Employee Email</label><input type="email" id="sub-email" placeholder="employee@company.com"></div>
                        <div class="form-group"><label>Restaurant ID</label><input type="number" id="sub-restaurant-id" placeholder="Restaurant ID"></div>
                        <div class="form-group"><label>Meal Slot</label><select id="sub-slot"><option value="BREAKFAST">Breakfast</option><option value="LUNCH">Lunch</option><option value="DINNER">...
                        <div class="form-group"><label>Schedule Type</label><select id="sub-schedule"><option value="DAILY">Daily</option><option value="WEEKLY">Weekly</option></select></div>
                        <div class="form-group" id="day-of-week-group" style="display:none;"><label>Day of Week</label><select id="sub-day"><option value="MONDAY">Monday</option><option value="TUES...
                    </div>
                    <div class="modal-actions" style="margin-top: 20px;"><button class="btn btn-primary" onclick="addSubscription()">Create Subscription</button><button class="btn btn-secondary" on...
                </div>
            </div>

            <!-- DELIVERIES VIEW -->
            <div id="deliveries-view" class="view">
                <div class="section-head"><div class="section-title">My Deliveries</div><button class="btn btn-primary btn-sm" onclick="loadDeliveries()">Refresh</button></div>
                <div class="table-wrap"><table><thead><tr><th>ID</th><th>Subscription ID</th><th>Meal Slot</th><th>Scheduled Time</th><th>Status</th></tr></thead><tbody id="deliveries-list"></tbody...
            </div>

            <!-- VENDORS VIEW -->
            <div id="vendors-view" class="view">
                <div class="section-head"><div class="section-title">Vendors</div><button class="btn btn-primary btn-sm" onclick="loadVendors()">Refresh</button></div>
                <div class="table-wrap"><table><thead><tr><th>ID</th><th>Name</th><th>Email</th><th>Phone</th><th>Registered</th></tr></thead><tbody id="vendors-list"></tbody></table></div>
            </div>

        </div>
    </div>
</div>

<!-- Approve/Reject Modal -->
<div id="action-modal" class="modal-overlay">
    <div class="modal">
        <div class="modal-title" id="modal-title">Approve Leave Request</div>
        <input type="hidden" id="action-leave-id">
        <input type="hidden" id="action-type">
        <div class="form-group"><label>Manager Email</label><input type="email" id="manager-email" placeholder="manager@company.com"></div>
        <div class="form-group"><label>Remarks</label><textarea id="action-remarks" rows="3" placeholder="Add remarks..."></textarea></div>
        <div class="modal-actions"><button class="btn btn-primary" id="action-btn" onclick="submitApproval()">Approve</button><button class="btn btn-secondary" onclick="closeModal()">Cancel</button...
    </div>
</div>

<!-- Cancel Leave Modal -->
<div id="cancel-modal" class="modal-overlay">
    <div class="modal">
        <div class="modal-title">Cancel Leave Request</div>
        <input type="hidden" id="cancel-leave-id">
        <div class="form-group"><label>Employee Email</label><input type="email" id="cancel-email" placeholder="employee@company.com"></div>
        <div class="modal-actions"><button class="btn btn-danger" onclick="confirmCancelLeave()">Cancel Leave</button><button class="btn btn-secondary" onclick="closeCancelModal()">Close</button></...
    </div>
</div>

<script>
    const API = 'http://localhost:8080';
    let currentUser = null;

    // ============ FIX 2: AUTH GUARD — redirect to login if not authenticated ============
    // Previously the page had no guard: it called loadDashboard() immediately,
    // which triggered a 401 on every API call for unauthenticated users.
    (function checkAuth() {
```

```
    const token = localStorage.getItem('jwt_token');
    const authType = localStorage.getItem('auth_type');

    // TEMP DEBUG - remove later
    console.log('=== AUTH DEBUG ===');
    console.log('jwt_token:', token);
    console.log('auth_type:', authType);
    console.log('token length:', token?.length);
    console.log('token starts with eyJ:', token?.startsWith('eyJ'));

    if (!token && authType !== 'session') {
        window.location.href = '/login.html';
    }
})();

    // ============ FIX 3: AUTH HEADER — session users send no JWT, rely on cookie ============
    // Previously getAuthHeader() always tried to send JWT. Session-logged-in users
    // had no jwt_token, so every API call went out without auth and got 401.
    function getAuthHeader() {
        const token = localStorage.getItem('jwt_token');
        return token ? { 'Authorization': `Bearer ${token}` } : {};
        // Session users rely on credentials:'include' (JSESSIONID cookie) — no header needed.
    }

    async function apiFetch(path, opts = {}) {
        try {
            const response = await fetch(`${API}${path}`, {
                ...opts,
                headers: {
                    'Content-Type': 'application/json',
                    ...getAuthHeader(),
                    ...opts.headers
                },
                credentials: 'include'
            });

            if (response.status === 401) {
                // FIX: Clear ALL auth state, not just jwt_token
                localStorage.removeItem('jwt_token');
                localStorage.removeItem('auth_type');
                showAlert('Session expired. Please login again.', 'error');
                setTimeout(() => window.location.href = '/login.html', 1500);
                throw new Error('Unauthorized');
            }

            if (!response.ok) {
                const error = await response.text();
                throw new Error(error || 'Request failed');
            }

            const text = await response.text();
            return text ? JSON.parse(text) : null;
        } catch (e) {
            if (!e.message.includes('Unauthorized')) {
                showAlert(e.message, 'error');
            }
            throw e;
        }
    }

    function showAlert(message, type) {
        const alert = document.createElement('div');
        alert.className = `alert alert-${type}`;
        alert.textContent = message;
        document.body.appendChild(alert);
        setTimeout(() => alert.remove(), 3000);
    }

    function showView(viewName, event) {
        document.querySelectorAll('.view').forEach(v => v.classList.remove('active'));
        document.getElementById(`${viewName}-view`).classList.add('active');
        document.querySelectorAll('.nav-item').forEach(n => n.classList.remove('active'));
        if (event && event.target) event.target.classList.add('active');
        document.getElementById('pageTitle').textContent = viewName.split('-').map(w =>
            w.charAt(0).toUpperCase() + w.slice(1)).join(' ');
        const loaders = {
```

```
            'dashboard': loadDashboard,
            'employees': loadEmployees,
            'my-leaves': loadMyLeaves,
            'pending-leaves': loadPendingLeaves,
            'restaurants': loadRestaurants,
            'subscriptions': loadSubscriptions,
            'deliveries': loadDeliveries,
            'vendors': loadVendors
        };
        if (loaders[viewName]) loaders[viewName]();
    }

    // ============ DASHBOARD ============
    async function loadDashboard() {
        try {
            const [employees, leaves] = await Promise.all([
                apiFetch('/employee'),
                apiFetch('/leave_requests')
            ]);
            document.getElementById('total-employees').textContent = employees.length;
            document.getElementById('active-employees').textContent = employees.filter(e => e.status === 'ACTIVE').length;
            document.getElementById('onleave-employees').textContent = employees.filter(e => e.status === 'ON_LEAVE').length;
            document.getElementById('pending-leaves').textContent = leaves.filter(l => l.status === 'PENDING').length;
            const recent = leaves.slice(-5).reverse();
            const tbody = document.getElementById('recent-leaves');
            if (!recent.length) {
                tbody.innerHTML = '<tr><td colspan="5" class="empty-state">No leave requests</td></tr>';
            } else {
                tbody.innerHTML = recent.map(l => `<tr>
                    <td>${l.employeeId || 'N/A'}</td>
                    <td><span class="chip">${l.leaveType || 'N/A'}</span></td>
                    <td>${l.startDate || 'N/A'}</td>
                    <td>${l.endDate || 'N/A'}</td>
                    <td><span class="chip ${l.status?.toLowerCase()}">${l.status || 'N/A'}</span></td>
                </tr>`).join('');
            }
        } catch (e) { console.error(e); }
    }

    // ============ EMPLOYEES ============
    async function loadEmployees() {
        try {
            const employees = await apiFetch('/employee');
            const tbody = document.getElementById('employees-list');
            if (!employees.length) {
                tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No employees found</td></tr>';
            } else {
                tbody.innerHTML = employees.map(e => `<tr>
                    <td>${e.employeeId}</td>
                    <td>${e.name || 'N/A'}</td>
                    <td>${e.email || 'N/A'}</td>
                    <td>${e.dept || 'N/A'}</td>
                    <td><span class="chip ${e.role?.toLowerCase()}">${e.role || 'N/A'}</span></td>
                    <td><span class="chip ${e.status?.toLowerCase()}">${e.status || 'N/A'}</span></td>
                    <td>${e.timezone || 'N/A'}</td>
                </tr>`).join('');
            }
        } catch (e) { console.error(e); }
    }

    // ============ LEAVE REQUESTS ============
    async function applyLeave() {
        const email = document.getElementById('leave-email').value.trim();
        const leaveType = document.getElementById('leave-type').value;
        const startDate = document.getElementById('leave-start').value;
        const endDate = document.getElementById('leave-end').value;
        const reason = document.getElementById('leave-reason').value.trim();
        if (!email || !startDate || !endDate) {
            showAlert('Please fill all required fields', 'error');
            return;
        }
        try {
            await apiFetch('/leave_requests', {
```

```
                method: 'POST',
                body: JSON.stringify({ leaveType, startDate, endDate, reason })
            });
            showAlert('Leave request submitted successfully!', 'success');
            clearLeaveForm();
            loadMyLeaves();
        } catch (e) { showAlert('Failed to submit: ' + e.message, 'error'); }
    }

    async function loadMyLeaves() {
    try {
        const data = await apiFetch('/leave_requests/employee');
        const leaves = data.requests || []; //  extract the array from wrapper
        const tbody = document.getElementById('my-leaves-list');
        if (!leaves.length) {
            tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No leave requests found</td></tr>';
        } else {
            tbody.innerHTML = leaves.map(l => `<tr>
                <td>${l.leaveRequestId}</td>
                <td><span class="chip">${l.leaveType}</span></td>
                <td>${l.startDate}</td>
                <td>${l.endDate}</td>
                <td><span class="chip ${l.status?.toLowerCase()}">${l.status}</span></td>
                <td style="max-width:200px;overflow:hidden;text-overflow:ellipsis;">${l.reason || '-'}</td>
                <td>${l.status === 'PENDING' ? `<button class="btn btn-danger btn-sm" onclick="openCancelModal(${l.leaveRequestId})">Cancel</button>` : '-'}</td>
            </tr>`).join('');
        }
    } catch (e) { console.error(e); }
}

    async function loadPendingLeaves() {
        try {
            const leaves = await apiFetch('/leave_requests/pending');
            const tbody = document.getElementById('pending-leaves-list');
            if (!leaves.length) {
                tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No pending leave requests</td></tr>';
            } else {
                tbody.innerHTML = leaves.map(l => `<tr>
                    <td>${l.leaveRequestId}</td>
                    <td>${l.employeeId}</td>
                    <td><span class="chip">${l.leaveType}</span></td>
                    <td>${l.startDate}</td>
                    <td>${l.endDate}</td>
                    <td style="max-width:150px;overflow:hidden;text-overflow:ellipsis;">${l.reason || '-'}</td>
                    <td class="action-row">
                        <button class="btn btn-primary btn-sm" onclick="openActionModal(${l.leaveRequestId}, 'APPROVED')">Approve</button>
                        <button class="btn btn-danger btn-sm" onclick="openActionModal(${l.leaveRequestId}, 'REJECTED')">Reject</button>
                    </td>
                </tr>`).join('');
            }
        } catch (e) { console.error(e); }
    }

    function openActionModal(leaveId, action) {
        document.getElementById('action-leave-id').value = leaveId;
        document.getElementById('action-type').value = action;
        document.getElementById('modal-title').textContent = `${action} Leave Request #${leaveId}`;
        document.getElementById('action-btn').textContent = action;
        document.getElementById('action-btn').className = action === 'APPROVED' ? 'btn btn-primary' : 'btn btn-danger';
        document.getElementById('action-modal').classList.add('open');
    }

    async function submitApproval() {
        const leaveRequestId = parseInt(document.getElementById('action-leave-id').value);
        const action = document.getElementById('action-type').value;
        const remarks = document.getElementById('action-remarks').value;
        const managerEmail = document.getElementById('manager-email').value.trim();
        if (!managerEmail) { showAlert('Please enter manager email', 'error'); return; }
        try {
            await apiFetch('/leave_requests/approval', {
```

```
                method: 'PUT',
                body: JSON.stringify({ leaveRequestId, action, remarks })
            });
            showAlert(`Leave request ${action.toLowerCase()} successfully!`, 'success');
            closeModal();
            loadPendingLeaves();
        } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
    }

    function openCancelModal(leaveId) {
        document.getElementById('cancel-leave-id').value = leaveId;
        document.getElementById('cancel-modal').classList.add('open');
    }

    async function confirmCancelLeave() {
        const leaveId = document.getElementById('cancel-leave-id').value;
        const email = document.getElementById('cancel-email').value.trim();
        if (!email) { showAlert('Please enter employee email', 'error'); return; }
        try {
            await apiFetch(`/leave_requests/cancel/${leaveId}?email=${encodeURIComponent(email)}`, { method: 'PUT' });
            showAlert('Leave cancelled successfully!', 'success');
            closeCancelModal();
            loadMyLeaves();
        } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
    }

    function clearLeaveForm() {
        document.getElementById('leave-email').value = '';
        document.getElementById('leave-start').value = '';
        document.getElementById('leave-end').value = '';
        document.getElementById('leave-reason').value = '';
    }

    // ============ RESTAURANTS ============
    async function loadRestaurants() {
        try {
            const slot = document.getElementById('slot-filter').value;
            const path = slot ? `/restaurants/by-slot?slot=${slot}` : '/restaurants';
            const restaurants = await apiFetch(path);
            const tbody = document.getElementById('restaurants-list');
            if (!restaurants.length) {
                tbody.innerHTML = '<tr><td colspan="5" class="empty-state">No restaurants found</td></tr>';
            } else {
                tbody.innerHTML = restaurants.map(r => `<tr>
                    <td><strong>${r.name}</strong></td>
                    <td>${r.address || 'N/A'}</td>
                    <td>${r.supportedMealSlots ? r.supportedMealSlots.map(s => `<span class="chip">${s}</span>`).join(' ') : 'N/A'}</td>
                    <td><span class="chip ${r.active ? 'active' : 'inactive'}">${r.active ? 'ACTIVE' : 'INACTIVE'}</span></td>
                    <td>${r.vendorName || r.vendorId || 'N/A'}</td>
                </tr>`).join('');
            }
        } catch (e) { console.error(e); }
    }

    // ============ SUBSCRIPTIONS ============
    async function loadSubscriptions() {
        try {
            const subscriptions = await apiFetch('/subscription/user');
            const tbody = document.getElementById('subscriptions-list');
            if (!subscriptions.length) {
                tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No subscriptions found</td></tr>';
            } else {
                tbody.innerHTML = subscriptions.map(s => `<tr>
                    <td>${s.subscriptionId}</td>
                    <td>${s.restaurantName || 'N/A'}</td>
                    <td><span class="chip">${s.mealSlot}</span></td>
                    <td>${s.scheduleType}</td>
                    <td><span class="chip ${s.status?.toLowerCase()}">${s.status}</span></td>
                    <td>${s.nextDeliveryTime ? new Date(s.nextDeliveryTime).toLocaleDateString() : 'Not scheduled'}</td>
                    <td class="action-row">
                        ${s.status === 'ACTIVE' ? `<button class="btn btn-secondary btn-sm"
```

```
onclick="pauseSubscription(${s.subscriptionId})">Pause</button>` : ''}
                            ${s.status === 'PAUSED' ? `<button class="btn btn-primary btn-sm" onclick="resumeSubscription(${s.subscriptionId})">Resume</button>` : ''}
                            <button class="btn btn-danger btn-sm" onclick="expireSubscription(${s.subscriptionId})">Expire</button>
                        </td>
                    </tr>`).join('');
                }
            } catch (e) { console.error(e); }
        }

        async function addSubscription() {
            const email = document.getElementById('sub-email').value.trim();
            const restaurantId = parseInt(document.getElementById('sub-restaurant-id').value);
            const mealSlot = document.getElementById('sub-slot').value;
            const scheduleType = document.getElementById('sub-schedule').value;
            const dayOfWeek = document.getElementById('sub-day').value;
            if (!email || !restaurantId) { showAlert('Please fill email and restaurant ID', 'error'); return; }
            const body = { restaurantId, mealSlot, scheduleType };
            if (scheduleType === 'WEEKLY') body.dayOfWeek = dayOfWeek;
            try {
                await apiFetch('/subscription', { method: 'POST', body: JSON.stringify(body) });
                showAlert('Subscription created successfully!', 'success');
                clearSubscriptionForm();
                loadSubscriptions();
            } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
        }

        async function pauseSubscription(id) {
            try { await apiFetch(`/subscription/${id}/pause`, { method: 'PUT' }); showAlert('Subscription paused', 'success'); loadSubscriptions(); }
            catch (e) { showAlert('Failed: ' + e.message, 'error'); }
        }

        async function resumeSubscription(id) {
            try { await apiFetch(`/subscription/${id}/resume`, { method: 'PUT' }); showAlert('Subscription resumed', 'success'); loadSubscriptions(); }
            catch (e) { showAlert('Failed: ' + e.message, 'error'); }
        }

        async function expireSubscription(id) {
            try { await apiFetch(`/subscription/${id}/expire`, { method: 'PUT' }); showAlert('Subscription expired', 'success'); loadSubscriptions(); }
            catch (e) { showAlert('Failed: ' + e.message, 'error'); }
        }

        function clearSubscriptionForm() {
            document.getElementById('sub-email').value = '';
            document.getElementById('sub-restaurant-id').value = '';
            document.getElementById('sub-slot').value = 'BREAKFAST';
            document.getElementById('sub-schedule').value = 'DAILY';
        }

        document.getElementById('sub-schedule')?.addEventListener('change', function () {
            document.getElementById('day-of-week-group').style.display = this.value === 'WEEKLY' ? 'block' : 'none';
        });

        // ============ DELIVERIES ============
        async function loadDeliveries() {
            try {
                const subscriptions = await apiFetch('/subscription/user');
                let allDeliveries = [];
                for (const sub of subscriptions) {
                    try {
                        const deliveries = await apiFetch(`/delivery/subscription/${sub.subscriptionId}`);
                        allDeliveries.push(...deliveries);
                    } catch (e) {}
                }
                const tbody = document.getElementById('deliveries-list');
                if (!allDeliveries.length) {
                    tbody.innerHTML = '<tr><td colspan="5" class="empty-state">No deliveries found</td></tr>';
                } else {
                    tbody.innerHTML = allDeliveries.map(d => `<tr>
                        <td>${d.id}</td>
                        <td>${d.subscription?.id || 'N/A'}</td>
```

```
                    <td><span class="chip">${d.mealSlot || 'N/A'}</span></td>
                    <td>${d.scheduledDeliveryTime ? new Date(d.scheduledDeliveryTime).toLocaleString() : 'N/A'}</td>
                    <td><span class="chip ${d.status?.toLowerCase()}">${d.status || 'SCHEDULED'}</span></td>
                </tr>`).join('');
            }
        } catch (e) { console.error(e); }
    }

    // ============ VENDORS ============
    async function loadVendors() {
        try {
            const vendors = await apiFetch('/vendors');
            const tbody = document.getElementById('vendors-list');
            if (!vendors.length) {
                tbody.innerHTML = '<tr><td colspan="5" class="empty-state">No vendors found</td></tr>';
            } else {
                tbody.innerHTML = vendors.map(v => `<tr>
                    <td>${v.id}</td>
                    <td><strong>${v.name}</strong></td>
                    <td>${v.email}</td>
                    <td>${v.phone || 'N/A'}</td>
                    <td>${v.registeredAt || 'N/A'}</td>
                </tr>`).join('');
            }
        } catch (e) { console.error(e); }
    }

    // ============ MODAL HELPERS ============
    function closeModal() {
        document.getElementById('action-modal').classList.remove('open');
        document.getElementById('action-remarks').value = '';
        document.getElementById('manager-email').value = '';
    }

    function closeCancelModal() {
        document.getElementById('cancel-modal').classList.remove('open');
        document.getElementById('cancel-email').value = '';
    }

    // ============ FIX 5: LOGOUT — also invalidates server-side session for session users ============
    // Previously logout() only cleared localStorage, leaving the JSESSIONID cookie active.
    async function logout() {
        const authType = localStorage.getItem('auth_type');
        if (authType === 'session') {
            try {
                await fetch(`${API}/session/logout`, { method: 'POST', credentials: 'include' });
            } catch (e) { /* ignore network errors during logout */ }
        }
        localStorage.removeItem('jwt_token');
        localStorage.removeItem('username');
        localStorage.removeItem('auth_type');
        showAlert('Logged out successfully', 'success');
        setTimeout(() => window.location.href = '/google.html', 500);
    }

    // Set today's date as default for leave form
    document.getElementById('leave-start').valueAsDate = new Date();
    document.getElementById('leave-end').valueAsDate = new Date();

    // Display username
    const username = localStorage.getItem('username');
    if (username) document.getElementById('username-display').textContent = ` ${username}`;

    // Initial load
    loadDashboard();
</script>
</body>
</html>
```

92 Current: src\main\resources\static\employee-dashboard.html 654 lines

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>EMS — Employee Portal</title>
    <link
href="https://fonts.googleapis.com/css2?family=Sora:wght@300;400;500;600;700&family=JetBrains+Mono:wght@400;500&
display=swap" rel="stylesheet">
    <style>
        *{margin:0;padding:0;box-sizing:border-box}
         :root{
             --bg:#080c10;--bg2:#0d1117;--bg3:#111820;
             --panel:#141c26;--panel2:#1a2332;
             --border:rgba(99,179,237,0.08);--border2:rgba(99,179,237,0.15);
             --text:#d4e6f1;--text2:#7a9bb5;--text3:#4a6d87;
             --accent:#38bdf8;--accent-dim:rgba(56,189,248,0.1);
             --green:#34d399;--green-dim:rgba(52,211,153,0.1);
             --amber:#fbbf24;--amber-dim:rgba(251,191,36,0.1);
             --red:#f87171;--red-dim:rgba(248,113,113,0.1);
             --font:'Sora',sans-serif;--mono:'JetBrains Mono',monospace;
             --sidebar:240px;
         }
         body{font-family:var(--font);background:var(--bg);color:var(--text);font-
size:14px;overflow:hidden;height:100vh}
         .app{display:flex;height:100vh}

         /* SIDEBAR */
         .sidebar{width:var(--sidebar);background:var(--bg2);border-right:1px solid var(--
border);display:flex;flex-direction:column;position:relative;flex-shrink:0}
         .sidebar::before{content:'';position:absolute;top:0;left:0;right:0;height:1px;background:linear-
gradient(90deg,transparent,var(--accent),transparent)}
         .logo{padding:24px 20px;border-bottom:1px solid var(--border)}
         .logo-mark{display:flex;align-items:center;gap:10px;margin-bottom:6px}
         .logo-icon{width:32px;height:32px;background:linear-gradient(135deg,var(--accent),#0ea5e9);border-
radius:8px;display:flex;align-items:center;justify-content:center;font-weight:700;font-size...
         .logo h2{font-size:16px;font-weight:600;color:var(--text);letter-spacing:0.5px}
         .logo p{font-size:11px;color:var(--text3);font-family:var(--mono);margin-top:2px}
         .role-badge{display:inline-flex;align-items:center;gap:5px;padding:4px 10px;background:var(--accent-
dim);border:1px solid rgba(56,189,248,0.25);border-radius:20px;font-size:10px;font-weight...
         .role-badge::before{content:'';width:6px;height:6px;background:var(--accent);border-
radius:50%;animation:pulse 2s infinite}
         @keyframes pulse{0%,100%{opacity:1}50%{opacity:0.4}}
         .nav{flex:1;padding:12px 0;overflow-y:auto}
         .nav-section{padding:4px 20px 8px;font-size:10px;font-weight:600;color:var(--text3);letter-
spacing:1.5px;text-transform:uppercase;margin-top:8px}
         .nav-item{display:flex;align-items:center;gap:10px;padding:10px 20px;margin:2px 8px;border-
radius:8px;cursor:pointer;transition:all 0.2s;color:var(--text2);font-size:13px;font-weight:500}
         .nav-item:hover{background:var(--accent-dim);color:var(--accent)}
         .nav-item.active{background:var(--accent-dim);color:var(--accent);border:1px solid
rgba(56,189,248,0.2)}
         .nav-item .icon{width:20px;text-align:center;font-size:14px;flex-shrink:0}
         .sidebar-footer{padding:16px 20px;border-top:1px solid var(--border)}
         .user-card{display:flex;align-items:center;gap:10px}
         .avatar{width:32px;height:32px;background:linear-gradient(135deg,#0ea5e9,var(--accent));border-
radius:50%;display:flex;align-items:center;justify-content:center;font-size:13px;font-weight:6...
         .user-info-side .name{font-size:12px;font-weight:600;color:var(--text)}
         .user-info-side .email{font-size:10px;color:var(--text3);font-family:var(--mono);white-
space:nowrap;overflow:hidden;text-overflow:ellipsis;max-width:140px}

         /* MAIN */
         .main{flex:1;display:flex;flex-direction:column;overflow:hidden}
         .topbar{background:var(--bg2);padding:14px 24px;display:flex;justify-content:space-between;align-
items:center;border-bottom:1px solid var(--border);flex-shrink:0}
         .page-title{font-size:16px;font-weight:600;color:var(--text)}
         .topbar-right{display:flex;align-items:center;gap:12px}
         .logout-btn{padding:7px 16px;background:transparent;border:1px solid var(--border2);border-
radius:6px;color:var(--text2);font-family:var(--font);font-size:12px;cursor:pointer;transition:all...
         .logout-btn:hover{background:var(--red-dim);border-color:var(--red);color:var(--red)}

         /* CONTENT */
         .content{flex:1;overflow-y:auto;padding:24px}
```

```
58          .view{display:none}
59          .view.active{display:block;animation:fadeIn 0.3s ease}
60          @keyframes fadeIn{from{opacity:0;transform:translateY(8px)}to{opacity:1;transform:translateY(0)}}
61
62          /* CARDS */
63          .stats-grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(160px,1fr));gap:16px;margin-
bottom:24px}
64          .stat-card{background:var(--panel);border:1px solid var(--border);border-
radius:12px;padding:20px;position:relative;overflow:hidden;transition:border-color 0.2s}
65          .stat-card:hover{border-color:var(--border2)}
66          .stat-card::after{content:'';position:absolute;top:0;left:0;right:0;height:2px}
67          .stat-card.blue::after{background:var(--accent)}
68          .stat-card.green::after{background:var(--green)}
69          .stat-card.amber::after{background:var(--amber)}
70          .stat-card.red::after{background:var(--red)}
71          .stat-label{font-size:11px;color:var(--text3);text-transform:uppercase;letter-spacing:1px;margin-
bottom:10px;font-weight:500}
72          .stat-value{font-size:28px;font-weight:700;color:var(--text);font-family:var(--mono)}
73          .stat-sub{font-size:11px;color:var(--text3);margin-top:4px}
74
75          /* SECTION */
76          .section{background:var(--panel);border:1px solid var(--border);border-radius:12px;margin-
bottom:20px;overflow:hidden}
77          .section-head{padding:16px 20px;border-bottom:1px solid var(--border);display:flex;justify-
content:space-between;align-items:center}
78          .section-title{font-size:14px;font-weight:600;color:var(--text)}
79          .section-sub{font-size:12px;color:var(--text3)}
80
81          /* TABLE */
82          .tablewrap{overflow-x:auto}
83          table{width:100%;border-collapse:collapse}
84          th{padding:10px 16px;text-align:left;font-size:11px;color:var(--text3);font-weight:600;letter-
spacing:0.8px;text-transform:uppercase;background:var(--bg3);border-bottom:1px solid var(--bord...
85          td{padding:12px 16px;border-bottom:1px solid var(--border);font-size:13px;color:var(--text2)}
86          tr:last-child td{border-bottom:none}
87          tr:hover td{background:rgba(56,189,248,0.03);color:var(--text)}
88
89          /* CHIPS */
90          .chip{display:inline-flex;align-items:center;padding:3px 10px;border-radius:20px;font-
size:11px;font-weight:600;letter-spacing:0.5px}
91          .chip.pending{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(251,191,36,0.2)}
92          .chip.approved{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.2)}
93          .chip.rejected{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
94          .chip.active{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.2)}
95          .chip.sick{background:var(--red-dim);color:var(--red)}
96          .chip.casual{background:var(--amber-dim);color:var(--amber)}
97          .chip.earned{background:var(--accent-dim);color:var(--accent)}
98
99          /* FORM */
100          .form-card{padding:24px}
101          .form-grid{display:grid;grid-template-columns:1fr 1fr;gap:16px}
102          .form-group{display:flex;flex-direction:column;gap:6px}
103          .form-group.full{grid-column:1/-1}
104          label{font-size:11px;font-weight:600;color:var(--text3);text-transform:uppercase;letter-
spacing:0.8px}
105          input,select,textarea{background:var(--bg3);border:1px solid var(--border);border-
radius:8px;padding:10px 14px;color:var(--text);font-family:var(--font);font-size:13px;transition:border-col...
106          input:focus,select:focus,textarea:focus{outline:none;border-color:var(--accent);background:var(--
bg2)}
107          input::placeholder,textarea::placeholder{color:var(--text3)}
108          select option{background:var(--bg2)}
109
110          /* BUTTONS */
111          .btn{padding:9px 18px;border-radius:8px;border:none;font-family:var(--font);font-size:13px;font-
weight:600;cursor:pointer;transition:all 0.2s}
112          .btn-primary{background:var(--accent);color:#fff}
113          .btn-primary:hover{background:#0ea5e9;transform:translateY(-1px);box-shadow:0 4px 12px
rgba(56,189,248,0.3)}
114          .btn-secondary{background:var(--panel2);color:var(--text2);border:1px solid var(--border)}
115          .btn-secondary:hover{border-color:var(--border2);color:var(--text)}
116          .btn-sm{padding:6px 12px;font-size:12px}
117          .btn-danger{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
118          .btn-danger:hover{background:var(--red);color:#fff}
119          .form-actions{display:flex;gap:10px;margin-top:20px}
120
121          /* LEAVE BALANCE */
```

```
        .balance-grid{display:grid;grid-template-columns:repeat(3,1fr);gap:12px;padding:20px}
        .balance-item{background:var(--bg3);border:1px solid var(--border);border-radius:10px;padding:16px;text-align:center}
        .balance-days{font-size:32px;font-weight:700;color:var(--text);font-family:var(--mono)}
        .balance-type{font-size:11px;color:var(--text3);text-transform:uppercase;letter-spacing:0.8px;margin-top:4px}

        /* ALERT */
        .alert{position:fixed;top:20px;right:20px;padding:12px 20px;border-radius:10px;z-index:1100;animation:slideIn 0.3s ease;font-size:13px;font-weight:500}
        .alert-success{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.3)}
        .alert-error{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.3)}
        @keyframes slideIn{from{transform:translateX(120%);opacity:0}to{transform:translateX(0);opacity:1}}

        .empty-state{text-align:center;color:var(--text3);font-style:italic;padding:30px}
        .profile-grid{display:grid;grid-template-columns:1fr 1fr;gap:16px;padding:24px}
        .profile-field{display:flex;flex-direction:column;gap:4px}
        .profile-field .field-label{font-size:10px;color:var(--text3);text-transform:uppercase;letter-spacing:0.8px;font-weight:600}
        .profile-field .field-value{font-size:14px;color:var(--text);padding:10px 14px;background:var(--bg3);border-radius:8px;border:1px solid var(--border);font-family:var(--mono)}

        .subscription-card{display:grid;grid-template-columns:repeat(auto-fill,minmax(200px,1fr));gap:12px;padding:20px}
        .sub-item{background:var(--bg3);border:1px solid var(--border);border-radius:10px;padding:16px;transition:border-color 0.2s}
        .sub-item:hover{border-color:var(--border2)}
        .sub-item .meal-name{font-weight:600;color:var(--text);margin-bottom:6px}
        .sub-item .meal-meta{font-size:11px;color:var(--text3)}

        .chip.paused{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(251,191,36,0.2)}
        .chip.expired{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
    </style>
</head>
<body>
<div class="app">
    <!-- SIDEBAR -->
    <div class="sidebar">
        <div class="logo">
            <div class="logo-mark">
                <div class="logo-icon">E</div>
                <h2>EMS Portal</h2>
            </div>
            <p>Employee Management</p>
            <div class="role-badge">Employee</div>
        </div>
        <nav class="nav">
            <div class="nav-section">Overview</div>
            <div class="nav-item active" onclick="showView('dashboard',this)"><span class="icon"></span> Dashboard</div>
            <div class="nav-section">Leave</div>
            <div class="nav-item" onclick="showView('apply-leave',this)"><span class="icon"></span> Apply Leave</div>
            <div class="nav-item" onclick="showView('my-leaves',this)"><span class="icon"></span> My Leave History</div>
            <div class="nav-section">Meals</div>
            <div class="nav-item" onclick="showView('restaurants',this)"><span class="icon"></span> Restaurants</div>
            <div class="nav-item" onclick="showView('subscriptions',this)"><span class="icon"></span> My Subscriptions</div>
            <div class="nav-item" onclick="showView('add-subscription',this)"><span class="icon"></span> Add Subscription</div>
            <div class="nav-section">Account</div>
            <div class="nav-item" onclick="showView('profile',this)"><span class="icon"></span> My Profile</div>
        </nav>
        <div class="sidebar-footer">
            <div class="user-card">
                <div class="avatar" id="avatar-initials">?</div>
                <div class="user-info-side">
                    <div class="name" id="sidebar-name">Loading...</div>
                    <div class="email" id="sidebar-email">—</div>
                </div>
            </div>
        </div>
```

```
    </div>

    <!-- MAIN -->
    <div class="main">
        <div class="topbar">
            <div class="page-title" id="pageTitle">Dashboard</div>
            <div class="topbar-right">
                <span style="font-size:12px;color:var(--text3);font-family:var(--mono)" id="topbar-email"></span>
                <button class="logout-btn" onclick="logout()">Sign Out</button>
            </div>
        </div>
        <div class="content">

            <!-- DASHBOARD -->
            <div id="dashboard-view" class="view active">
                <div class="stats-grid">
                    <div class="stat-card blue">
                        <div class="stat-label">Leave Balance</div>
                        <div class="stat-value" id="stat-balance">--</div>
                        <div class="stat-sub">days remaining</div>
                    </div>
                    <div class="stat-card amber">
                        <div class="stat-label">Pending Leaves</div>
                        <div class="stat-value" id="stat-pending">--</div>
                        <div class="stat-sub">awaiting approval</div>
                    </div>
                    <div class="stat-card green">
                        <div class="stat-label">Approved Leaves</div>
                        <div class="stat-value" id="stat-approved">--</div>
                        <div class="stat-sub">this year</div>
                    </div>
                    <div class="stat-card red">
                        <div class="stat-label">Rejected Leaves</div>
                        <div class="stat-value" id="stat-rejected">--</div>
                        <div class="stat-sub">this year</div>
                    </div>
                </div>
                <div class="section">
                    <div class="section-head">
                        <div>
                            <div class="section-title">Recent Leave Requests</div>
                            <div class="section-sub">Your last 5 requests</div>
                        </div>
                        <button class="btn btn-secondary btn-sm" onclick="loadDashboard()">Refresh</button>
                    </div>
                    <div class="tablewrap">
                        <table>
<thead><tr><th>Type</th><th>From</th><th>To</th><th>Reason</th><th>Status</th></tr></thead>
                            <tbody id="recent-leaves"><tr><td colspan="5" class="empty-state">Loading...</td></tr></tbody>
                        </table>
                    </div>
                </div>
                <div class="section">
                    <div class="section-head">
                        <div class="section-title">Active Subscriptions</div>
                    </div>
                    <div id="dashboard-subs" class="subscription-card"></div>
                </div>
            </div>

            <!-- APPLY LEAVE -->
            <div id="apply-leave-view" class="view">
                <div class="section">
                    <div class="section-head"><div class="section-title">Apply for Leave</div></div>
                    <div class="form-card">
                        <div class="form-grid">
                            <div class="form-group">
                                <label>Leave Type</label>
                                <select id="leave-type">
                                    <option value="SICK">Sick Leave</option>
                                    <option value="CASUAL">Casual Leave</option>
                                </select>
```

```
                            </div>
                            <div class="form-group">
                                <label>Start Date</label>
                                <input type="date" id="leave-start">
                            </div>
                            <div class="form-group">
                                <label>End Date</label>
                                <input type="date" id="leave-end">
                            </div>
                            <div class="form-group full">
                                <label>Reason</label>
                                <textarea id="leave-reason" rows="3" placeholder="Please describe the reason for your leave..."></textarea>
                            </div>
                        </div>
                        <div class="form-actions">
                            <button class="btn btn-primary" onclick="applyLeave()">Submit Request</button>
                            <button class="btn btn-secondary" onclick="clearLeaveForm()">Clear</button>
                        </div>
                    </div>
                </div>
            </div>

            <!-- MY LEAVES -->
            <div id="my-leaves-view" class="view">
                <div class="section">
                    <div class="section-head">
                        <div><div class="section-title">My Leave History</div><div class="section-sub">All your leave requests</div></div>
                        <button class="btn btn-secondary btn-sm" onclick="loadMyLeaves()"> Refresh</button>
                    </div>
                    <div class="tablewrap">
                        <table>
<thead><tr><th>Type</th><th>From</th><th>To</th><th>Days</th><th>Reason</th><th>Status</th><th>Action</th></tr></thead>
                            <tbody id="my-leaves-list"><tr><td colspan="7" class="empty-state">Loading...</td></tr></tbody>
                        </table>
                    </div>
                </div>
            </div>

            <!-- RESTAURANTS -->
            <div id="restaurants-view" class="view">
                <div class="section">
                    <div class="section-head">
                        <div class="section-title">Available Restaurants</div>
                        <button class="btn btn-secondary btn-sm" onclick="loadRestaurants()">Refresh</button>
                    </div>
                    <div class="tablewrap">
                        <table>
                            <thead><tr><th>Name</th><th>Meal Slot</th><th>Status</th><th>Action</th></tr></thead>
                            <tbody id="restaurants-list"><tr><td colspan="4" class="empty-state">Loading...</td></tr></tbody>
                        </table>
                    </div>
                </div>
            </div>

            <!-- SUBSCRIPTIONS -->
            <div id="subscriptions-view" class="view">
                <div class="section">
                    <div class="section-head">
                        <div class="section-title">My Meal Subscriptions</div>
                        <button class="btn btn-secondary btn-sm" onclick="loadSubscriptions()">Refresh</button>
                    </div>
                    <div class="tablewrap">
                        <table>
                            <thead>
                            <tr>
                                <th>Restaurant</th>
                                <th>Meal Slot</th>
```

```
                                <th>Schedule</th>
                                <th>Next Delivery</th>
                                <th>Status</th>
                                <th>Action</th>
                            </tr>
                            </thead>
                            <tbody id="subscriptions-list"><tr><td colspan="6" class="empty-state">Loading...</td></tr></tbody>
                        </table>
                    </div>
                </div>
            </div>

            <!-- ADD SUBSCRIPTION -->
            <div id="add-subscription-view" class="view">
                <div class="section">
                    <div class="section-head"><div class="section-title">Subscribe to Meal Plan</div></div>
                    <div class="form-card">
                        <div class="form-grid">
                            <div class="form-group">
                                <label>Meal Slot</label>
                                <select id="sub-meal-slot">
                                    <option value="BREAKFAST">Breakfast</option>
                                    <option value="LUNCH">Lunch</option>
                                    <option value="DINNER">Dinner</option>
                                </select>
                            </div>
                            <div class="form-group">
                                <label>Restaurant ID</label>
                                <input type="number" id="sub-restaurant-id" placeholder="Restaurant ID">
                            </div>
                            <div class="form-group">
                                <label>Schedule Type</label>
                                <select id="sub-schedule-type">
                                    <option value="DAILY">Daily</option>
                                    <option value="WEEKLY">Weekly</option>
                                </select>
                            </div>
                            <div class="form-group" id="day-of-week-group" style="display:none">
                                <label>Day of Week</label>
                                <select id="sub-day-of-week">
                                    <option value="MONDAY">Monday</option>
                                    <option value="TUESDAY">Tuesday</option>
                                    <option value="WEDNESDAY">Wednesday</option>
                                    <option value="THURSDAY">Thursday</option>
                                    <option value="FRIDAY">Friday</option>
                                    <option value="SATURDAY">Saturday</option>
                                    <option value="SUNDAY">Sunday</option>
                                </select>
                            </div>
                        </div>
                        <div class="form-actions">
                            <button class="btn btn-primary" onclick="addSubscription()">Subscribe</button>
                        </div>
                    </div>
                </div>
            </div>

            <!-- PROFILE -->
            <div id="profile-view" class="view">
                <div class="section">
                    <div class="section-head"><div class="section-title">My Profile</div></div>
                    <div class="profile-grid" id="profile-data">
                        <div class="profile-field"><div class="field-label">Name</div><div class="field-value" id="p-name">—</div></div>
                        <div class="profile-field"><div class="field-label">Email</div><div class="field-value" id="p-email">—</div></div>
                        <div class="profile-field"><div class="field-label">Department</div><div class="field-value" id="p-dept">—</div></div>
                        <div class="profile-field"><div class="field-label">Role</div><div class="field-value" id="p-role">—</div></div>
                        <div class="profile-field"><div class="field-label">Status</div><div class="field-value" id="p-status">—</div></div>
                        <div class="profile-field"><div class="field-label">Timezone</div><div class="field-value" id="p-tz">—</div></div>
                    </div>
```

```
                </div>
            </div>

        </div>
    </div>
</div>

<script>
    const API = 'http://localhost:8080';

    // AUTH GUARD
    (function(){
        const token = localStorage.getItem('jwt_token');
        const authType = localStorage.getItem('auth_type');
        if(!token && authType !== 'session') { window.location.href = '/login.html'; return; }
        // Decode JWT to check role
        if(token) {
            try {
                const payload = JSON.parse(atob(token.split('.')[1]));
                const roles = payload.roles || payload.authorities || '';
                if(roles.includes('ROLE_MANAGER')) { window.location.href = '/manager-dashboard.html'; return; }
                if(roles.includes('ROLE_VENDOR')) { window.location.href = '/vendor-dashboard.html'; return; }
            } catch(e) { console.warn('Could not decode JWT for role check'); }
        }
        initUser();
    })();

    function getAuthHeader(){
        const token = localStorage.getItem('jwt_token');
        return token ? {'Authorization':`Bearer ${token}`} : {};
    }

    async function apiFetch(path, opts={}){
    const res = await fetch(`${API}${path}`,{
        ...opts,
        headers:{'Content-Type':'application/json',...getAuthHeader(),...(opts.headers||{})},
        credentials:'include'
    });
    if(res.status===401){
        localStorage.removeItem('jwt_token'); localStorage.removeItem('auth_type');
        showAlert('Session expired. Please login again.','error');
        setTimeout(()=>window.location.href='/login.html',1500);
        throw new Error('Unauthorized');
    }
    if(res.status===403) throw new Error('ACCESS_DENIED');
    if(!res.ok){
        const errText = await res.text();
        throw new Error(errText || 'Request failed');
    }
    const text = await res.text();
    try { return text ? JSON.parse(text) : null; }
    catch { return text; } // plain string response
    }

    async function initUser(){
        try {
            // Try /session/me first, fall back to JWT decode
            let email='', name='';
            const token = localStorage.getItem('jwt_token');
            if(token){
                try{
                    const p=JSON.parse(atob(token.split('.')[1]));
                    email=p.sub||p.email||'';
                }catch(e){}
            }
            // Try to fetch profile
            try{
                const me = await apiFetch('/employee/me');
                if(me){
                    name=me.name||email; email=me.email||email;
                    document.getElementById('p-name').textContent=me.name||'—';
                    document.getElementById('p-email').textContent=me.email||'—';
                    document.getElementById('p-dept').textContent=me.dept||'—';
                    document.getElementById('p-role').textContent=me.role||'EMPLOYEE';
```

```
                    document.getElementById('p-status').textContent=me.status||'—';
                    document.getElementById('p-tz').textContent=me.timezone||'—';
                }
            }catch(e){ console.warn('Could not load profile'); }
            const initials=(name||email||'?').charAt(0).toUpperCase();
            document.getElementById('avatar-initials').textContent=initials;
            document.getElementById('sidebar-name').textContent=name||email||'Employee';
            document.getElementById('sidebar-email').textContent=email;
            document.getElementById('topbar-email').textContent=email;
        }catch(e){console.error(e);}
        loadDashboard();
    }

    function showView(name, el){
        document.querySelectorAll('.view').forEach(v=>v.classList.remove('active'));
        document.getElementById(`${name}-view`).classList.add('active');
        document.querySelectorAll('.nav-item').forEach(n=>n.classList.remove('active'));
        if(el) el.classList.add('active');
        document.getElementById('pageTitle').textContent=name.split('-').map(w=>w.charAt(0).toUpperCase()+w.slice(1)).join(' ');
        const loaders={dashboard:loadDashboard,'my-leaves':loadMyLeaves,'apply-leave':()=>{},'restaurants':loadRestaurants,'subscriptions':loadSubscriptions,'add-subscription':()=>{},'profile':()=>...
        if(loaders[name]) loaders[name]();
    }

    async function loadDashboard(){
        try{
            const leaves = await apiFetch('/leave_requests/employee');
            const arr = leaves?.requests || leaves || [];
            const pending=arr.filter(l=>l.status==='PENDING').length;
            const approved=arr.filter(l=>l.status==='APPROVED').length;
            const rejected=arr.filter(l=>l.status==='REJECTED').length;
            document.getElementById('stat-pending').textContent=pending;
            document.getElementById('stat-approved').textContent=approved;
            document.getElementById('stat-rejected').textContent=rejected;
            document.getElementById('stat-balance').textContent=Math.max(0,20-approved);
            const recent=arr.slice(-5).reverse();
            document.getElementById('recent-leaves').innerHTML=recent.length
                ? recent.map(l=>`<tr><td><span class="chip ${(l.leaveType||'').toLowerCase()}">${l.leaveType||'N/A'}</span></td><td>${l.startDate||'—'}</td><td>${l.endDate||'—'}</td><td>${(l.reason...
                : '<tr><td colspan="5" class="empty-state">No leave requests yet</td></tr>';
        }catch(e){document.getElementById('recent-leaves').innerHTML='<tr><td colspan="5" class="empty-state">Could not load leave data</td></tr>';}
        try{
    const subs = await apiFetch('/subscription/user'); //  fix endpoint
    const arr = subs || [];
    document.getElementById('dashboard-subs').innerHTML = arr.length
        ? arr.filter(s => s.status === 'ACTIVE').slice(0,4).map(s => `
            <div class="sub-item">
                <div class="meal-name">${s.restaurantName || 'Restaurant'}</div>
                <div class="meal-meta">
                    ${s.mealSlot || ''} · ${s.scheduleType || ''}
                    ${s.dayOfWeek ? ' · ' + s.dayOfWeek : ''}
                </div>
                <div class="meal-meta" style="margin-top:4px;color:var(--accent)">
                    Next: ${s.nextDeliveryTime ? new Date(s.nextDeliveryTime).toLocaleString() : 'N/A'}
                </div>
            </div>`).join('')
        : '<div style="padding:16px;color:var(--text3);font-style:italic">No active subscriptions</div>';
      } catch(e) {
     document.getElementById('dashboard-subs').innerHTML =
        '<div style="padding:16px;color:var(--text3)">Could not load subscriptions</div>';
        }
    }

    async function loadMyLeaves(){
        try{
            const data=await apiFetch('/leave_requests/employee');
            const leaves=data?.requests||data||[];
            document.getElementById('my-leaves-list').innerHTML=leaves.length
                ? leaves.map(l=>`<tr><td><span class="chip ${(l.leaveType||'').toLowerCase()}">${l.leaveType||'N/A'}</span></td><td>${l.startDate||'—'}</td><td>${l.endDate||'—'}</td><td>${l.numberO...
                : '<tr><td colspan="7" class="empty-state">No leave requests found</td></tr>';
```

```
        }catch(e){document.getElementById('my-leaves-list').innerHTML='<tr><td colspan="7" class="empty-state">Could not load leave requests</td></tr>';}
    }

    async function applyLeave(){
        const leaveType=document.getElementById('leave-type').value;
        const startDate=document.getElementById('leave-start').value;
        const endDate=document.getElementById('leave-end').value;
        const reason=document.getElementById('leave-reason').value.trim();
        if(!startDate||!endDate){showAlert('Please fill all required fields','error');return;}
        try{
            await apiFetch('/leave_requests',{method:'POST',body:JSON.stringify({leaveType,startDate,endDate,reason})});
            showAlert('Leave request submitted!','success');
            clearLeaveForm();
        }catch(e){showAlert('Failed to submit: '+e.message,'error');}
    }

    async function cancelLeave(id){
    if(!confirm('Cancel this leave request?')) return;
    try{
        await apiFetch(`/leave_requests/cancel/${id}`, {method:'PUT'});
        showAlert('Request cancelled successfully','success');
        loadMyLeaves();
    }
    catch(e){
        if(e.message === 'ACCESS_DENIED') showAlert('You do not have permission to cancel this leave','error');
        else showAlert('Failed to cancel: ' + e.message, 'error');
    }
   }

    function clearLeaveForm(){
        ['leave-start','leave-end','leave-reason'].forEach(id=>{document.getElementById(id).value='';});
    }

    async function loadRestaurants(){
    try{
        const data=await apiFetch('/restaurants');
        document.getElementById('restaurants-list').innerHTML=data?.length
            ? data.map(r=>`<tr>
                <td>${r.name||'—'}</td>
                <td>${r.supportedMealSlots?.join(', ') || '—'}</td>
                <td><span class="chip active">${r.active?'Active':'Inactive'}</span></td>
                <td><button class="btn btn-secondary btn-sm" onclick="document.getElementById('sub-restaurant-id').value='${r.id}';showView('add-subscription',null)">Subscribe</button></td>
              </tr>`).join('')
            : '<tr><td colspan="4" class="empty-state">No restaurants available</td></tr>';
    }catch(e){document.getElementById('restaurants-list').innerHTML='<tr><td colspan="4" class="empty-state">Could not load restaurants</td></tr>';}
   }

    async function loadSubscriptions(){
    try{
        const data = await apiFetch('/subscription/user');
        const arr = data || [];
        document.getElementById('subscriptions-list').innerHTML = arr.length
            ? arr.map(s => `<tr>
                <td>${s.restaurantName || s.restaurantId || '—'}</td>
                <td>${s.mealSlot || '—'}</td>
                <td>${s.scheduleType || '—'}${s.dayOfWeek ? ' · ' + s.dayOfWeek : ''}</td>
                <td>${s.nextDeliveryTime ? new Date(s.nextDeliveryTime).toLocaleString() : '—'}</td>
                <td><span class="chip ${(s.status||'').toLowerCase()}">${s.status || 'ACTIVE'}</span></td>
                <td><button class="btn btn-danger btn-sm" onclick="cancelSub(${s.subscriptionId})">Cancel</button></td>
              </tr>`).join('')
            : '<tr><td colspan="6" class="empty-state">No subscriptions found</td></tr>';
    } catch(e) {
        document.getElementById('subscriptions-list').innerHTML =
            '<tr><td colspan="6" class="empty-state">Could not load subscriptions</td></tr>';
    }
}

    document.getElementById('sub-schedule-type').addEventListener('change', function(){
    document.getElementById('day-of-week-group').style.display =
        this.value === 'WEEKLY' ? 'block' : 'none';
```

```
    });

   async function addSubscription(){
    const restaurantId = parseInt(document.getElementById('sub-restaurant-id').value);
    const mealSlot = document.getElementById('sub-meal-slot').value;
    const scheduleType = document.getElementById('sub-schedule-type').value;
    const dayOfWeek = document.getElementById('sub-day-of-week')?.value;

    if(!restaurantId || !mealSlot){
        showAlert('Please fill all fields','error'); return;
    }

    const body = {
        scheduleType,
        mealSlots: [mealSlot],
        slotOrders: [{ mealSlot, restaurantId }]
    };
    if(scheduleType === 'WEEKLY' && dayOfWeek) body.dayOfWeek = dayOfWeek;

    try{
        await apiFetch('/subscription', {method:'POST', body:JSON.stringify(body)});
        showAlert('Subscribed successfully!','success');
    }catch(e){
        showAlert('Subscription failed: '+e.message,'error');
    }
   }

   async function cancelSub(id){
    if(!confirm('Cancel this subscription?')) return;
    try{
        await apiFetch(`/subscription/${id}/expire`, {method:'PUT'});
        showAlert('Subscription cancelled','success');
        loadSubscriptions();
    }
    catch(e){
        showAlert('Failed to cancel: ' + e.message,'error');
    }
}

    function logout(){
        localStorage.removeItem('jwt_token'); localStorage.removeItem('auth_type');
fetch(`${API}/session/logout`,{method:'POST',credentials:'include'}).finally(()=>window.location.href='/google.html');
    }

    function showAlert(msg,type){
        const el=document.createElement('div');
        el.className=`alert alert-${type}`;el.textContent=msg;
        document.body.appendChild(el);setTimeout(()=>el.remove(),3000);
    }
</script>
</body>
</html>
```

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>EMS — Sign In</title>
    <link
href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;500;600;700&family=JetBrains+Mono:wght@400;50
0&display=swap" rel="stylesheet">
    <style>
        *{margin:0;padding:0;box-sizing:border-box}
         :root{
             --bg:#070910;
             --panel:#0e1118;
             --panel2:#131821;
             --border:rgba(255,255,255,0.07);
             --border-lit:rgba(255,255,255,0.14);
             --text:#dde4f0;--text2:#6b7591;--text3:#3d4560;
             --blue:#4f8cff;--blue-dim:rgba(79,140,255,0.12);
             --violet:#a78bfa;--violet-dim:rgba(167,139,250,0.12);
             --red:#f87171;--red-dim:rgba(248,113,113,0.12);
             --green:#34d399;--green-dim:rgba(52,211,153,0.12);
             --font:'Outfit',sans-serif;--mono:'JetBrains Mono',monospace;
         }

         body{
             font-family:var(--font);
             background:var(--bg);
             color:var(--text);
             min-height:100vh;
             display:flex;
             overflow:hidden;
         }

         /* ---- BACKGROUND ---- */
         .bg-layer{
             position:fixed;inset:0;z-index:0;
             background:
                     radial-gradient(ellipse 60% 50% at 20% 50%, rgba(79,140,255,0.06) 0%, transparent 70%),
                     radial-gradient(ellipse 40% 60% at 80% 20%, rgba(167,139,250,0.05) 0%, transparent 60%),
                     radial-gradient(ellipse 50% 40% at 70% 80%, rgba(52,211,153,0.04) 0%, transparent 60%);
         }
         .grid-overlay{
             position:fixed;inset:0;z-index:0;
             background-image:
                     linear-gradient(rgba(255,255,255,0.025) 1px, transparent 1px),
                     linear-gradient(90deg, rgba(255,255,255,0.025) 1px, transparent 1px);
             background-size:40px 40px;
             mask-image:radial-gradient(ellipse 80% 80% at center, black 20%, transparent 80%);
         }

         /* ---- SPLIT LAYOUT ---- */
         .left-panel{
             flex:1;display:flex;flex-direction:column;justify-content:center;
             padding:60px;position:relative;z-index:1;
             animation:fadeLeft 0.8s ease both;
         }
         @keyframes fadeLeft{from{opacity:0;transform:translateX(-
24px)}to{opacity:1;transform:translateX(0)}}
         .brand{margin-bottom:48px}
         .brand-logo{display:flex;align-items:center;gap:12px;margin-bottom:20px}
         .brand-icon{
             width:44px;height:44px;background:linear-gradient(135deg,#4f8cff,#6366f1);
             border-radius:12px;display:flex;align-items:center;justify-content:center;
             font-weight:700;font-size:18px;color:#fff;
             box-shadow:0 0 30px rgba(79,140,255,0.35);
             flex-shrink:0;
         }
         .brand-name{font-size:22px;font-weight:700;color:var(--text);letter-spacing:-0.3px}
         .brand-tagline{font-size:13px;color:var(--text2)}
         .hero-text h1{
             font-size:clamp(36px,4vw,52px);font-weight:700;
```

```
            line-height:1.1;letter-spacing:-1px;color:var(--text);margin-bottom:16px;
        }
        .hero-text h1 .hl-blue{color:var(--blue)}
        .hero-text h1 .hl-violet{color:var(--violet)}
        .hero-text h1 .hl-green{color:var(--green)}
        .hero-text p{font-size:15px;color:var(--text2);line-height:1.6;max-width:380px}
        .feature-pills{display:flex;flex-wrap:wrap;gap:8px;margin-top:32px}
        .pill{
            display:inline-flex;align-items:center;gap:6px;
            padding:6px 14px;border-radius:20px;font-size:12px;font-weight:500;
            border:1px solid var(--border);background:var(--panel);color:var(--text2);
        }
        .pill .dot{width:6px;height:6px;border-radius:50%;flex-shrink:0}
        .pill.blue .dot{background:var(--blue)} .pill.blue{border-color:rgba(79,140,255,0.2);color:rgba(79,140,255,0.8)}
        .pill.violet .dot{background:var(--violet)} .pill.violet{border-color:rgba(167,139,250,0.2);color:rgba(167,139,250,0.8)}
        .pill.green .dot{background:var(--green)} .pill.green{border-color:rgba(52,211,153,0.2);color:rgba(52,211,153,0.8)}

        #oauth-loading {
      display: none;
      position: fixed; inset: 0;
      background: linear-gradient(135deg, #0d0f14 0%, #131720 100%);
      z-index: 9999;
      justify-content: center;
      align-items: center;
      flex-direction: column;
      gap: 20px;
  }
  #oauth-loading.visible { display: flex; }
  .spinner {
      width: 48px; height: 48px;
       border: 3px solid #2a2f3f;
       border-top-color: #4f8cff;
       border-radius: 50%;
       animation: spin 0.9s linear infinite;
   }
   @keyframes spin { to { transform: rotate(360deg); } }

        /* ---- RIGHT PANEL (FORM) ---- */
        .right-panel{
            width:460px;flex-shrink:0;
            display:flex;align-items:center;justify-content:center;
            padding:32px 40px;position:relative;z-index:1;
            border-left:1px solid var(--border);
            background:rgba(14,17,24,0.8);
            backdrop-filter:blur(20px);
            animation:fadeRight 0.8s ease both;
        }
        @keyframes fadeRight{from{opacity:0;transform:translateX(24px)}to{opacity:1;transform:translateX(0)}}
        .form-box{width:100%}

        /* ---- ROLE SELECTOR ---- */
        .role-label{font-size:10px;font-weight:600;color:var(--text3);letter-spacing:1.5px;text-transform:uppercase;font-family:var(--mono);margin-bottom:10px}
        .role-tabs{display:grid;grid-template-columns:1fr 1fr 1fr;gap:6px;margin-bottom:28px}
        .role-tab{
            padding:10px 8px;border-radius:10px;border:1.5px solid var(--border);
            background:var(--bg);color:var(--text2);font-family:var(--font);
            font-size:12px;font-weight:600;cursor:pointer;transition:all 0.2s;
            display:flex;flex-direction:column;align-items:center;gap:4px;
        }
        .role-tab .role-icon{font-size:16px}
        .role-tab:hover{border-color:var(--border-lit);color:var(--text)}
        .role-tab.active-emp{border-color:var(--blue);background:var(--blue-dim);color:var(--blue)}
        .role-tab.active-mgr{border-color:var(--violet);background:var(--violet-dim);color:var(--violet)}
        .role-tab.active-vnd{border-color:var(--red);background:var(--red-dim);color:var(--red)}

        /* ---- AUTH TABS ---- */
        .auth-tabs{display:flex;background:var(--bg);border-radius:10px;padding:4px;gap:4px;margin-bottom:24px;border:1px solid var(--border)}
        .auth-tab{
            flex:1;padding:9px;border:none;background:transparent;
            color:var(--text2);font-family:var(--font);font-weight:600;
```

```
            font-size:13px;cursor:pointer;border-radius:7px;transition:all 0.2s;
        }
        .auth-tab.active{background:var(--panel2);color:var(--text);box-shadow:0 1px 4px rgba(0,0,0,0.3)}

        /* ---- FORM ELEMENTS ---- */
        .form-group{margin-bottom:16px}
        .form-label{display:block;font-size:10px;font-weight:600;color:var(--text2);text-transform:uppercase;letter-spacing:0.8px;font-family:var(--mono);margin-bottom:7px}
        .form-input{
            width:100%;padding:11px 14px;
            background:var(--bg);border:1px solid var(--border);
            border-radius:9px;color:var(--text);
            font-family:var(--font);font-size:14px;
            transition:all 0.2s;
        }
        .form-input:focus{outline:none;border-color:var(--blue);box-shadow:0 0 0 3px rgba(79,140,255,0.08)}
        .form-input::placeholder{color:var(--text3)}
        .form-input.focus-violet:focus{border-color:var(--violet);box-shadow:0 0 0 3px rgba(167,139,250,0.08)}
        .form-input.focus-red:focus{border-color:var(--red);box-shadow:0 0 0 3px rgba(248,113,113,0.08)}

        /* ---- BUTTONS ---- */
        .btn-login{
            width:100%;padding:12px;border:none;border-radius:9px;
            font-family:var(--font);font-size:14px;font-weight:600;
            cursor:pointer;transition:all 0.2s;margin-top:4px;
            position:relative;overflow:hidden;
        }
        .btn-login.blue{background:var(--blue);color:#fff}
        .btn-login.blue:hover{background:#6ba0ff;box-shadow:0 6px 20px rgba(79,140,255,0.35);transform:translateY(-1px)}
        .btn-login.violet{background:var(--violet);color:#fff}
        .btn-login.violet:hover{background:#b8a0fc;box-shadow:0 6px 20px rgba(167,139,250,0.35);transform:translateY(-1px)}
        .btn-login.red{background:var(--red);color:#fff}
        .btn-login.red:hover{background:#f98a8a;box-shadow:0 6px 20px rgba(248,113,113,0.35);transform:translateY(-1px)}
        .btn-login:active{transform:translateY(0)}
        .btn-login.loading{opacity:0.7;pointer-events:none}

        .divider{display:flex;align-items:center;gap:12px;margin:20px 0;color:var(--text3);font-size:11px;font-family:var(--mono)}
        .divider::before,.divider::after{content:'';flex:1;height:1px;background:var(--border)}

        .btn-google{
            width:100%;padding:11px;background:transparent;
            border:1px solid var(--border-lit);border-radius:9px;
            color:var(--text2);font-family:var(--font);font-size:13px;font-weight:500;
            cursor:pointer;transition:all 0.2s;
            display:flex;align-items:center;justify-content:center;gap:10px;
        }
        .btn-google:hover{background:rgba(255,255,255,0.04);border-color:rgba(255,255,255,0.2);color:var(--text)}

        /* ---- GOOGLE ROLE SELECT ---- */
        .google-role-row{display:flex;gap:8px;margin-bottom:10px;align-items:center}
        .google-role-label{font-size:11px;color:var(--text3);font-family:var(--mono);white-space:nowrap}
        .google-role-select{
            flex:1;padding:8px 10px;background:var(--bg);
            border:1px solid var(--border);border-radius:8px;
            color:var(--text);font-family:var(--font);font-size:13px;cursor:pointer;
        }
        .google-role-select:focus{outline:none;border-color:var(--border-lit)}
        .google-role-select option{background:var(--panel)}

        /* ---- ERROR / STATUS ---- */
        .status-box{
            border-radius:9px;padding:10px 14px;margin-top:14px;
            font-size:13px;font-weight:500;display:none;
            animation:fadeIn 0.3s ease;
        }
        @keyframes fadeIn{from{opacity:0;transform:translateY(-4px)}to{opacity:1;transform:translateY(0)}}
        .status-box.error{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
        .status-box.success{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.2)}
```

```
        .status-box.loading{background:var(--blue-dim);color:var(--blue);border:1px solid rgba(79,140,255,0.2)}

        /* ---- FOOTER ---- */
        .form-footer{margin-top:20px;padding-top:18px;border-top:1px solid var(--border)}
        .tz-row{display:flex;align-items:center;gap:8px;margin-bottom:8px}
        .tz-label{font-size:10px;color:var(--text3);font-family:var(--mono);flex-shrink:0}
        .tz-select{
            flex:1;padding:6px 8px;background:var(--bg);border:1px solid var(--border);
            border-radius:7px;color:var(--text2);font-family:var(--mono);font-size:11px;cursor:pointer;
        }
        .tz-select:focus{outline:none;border-color:var(--border-lit)}
        .tz-select option{background:var(--panel)}
        .tz-time{font-size:10px;color:var(--blue);font-family:var(--mono)}
        .demo-box{background:rgba(79,140,255,0.05);border:1px solid rgba(79,140,255,0.1);border-radius:8px;padding:10px 12px;margin-top:8px}
        .demo-title{font-size:9px;font-weight:700;color:var(--text3);letter-spacing:1.5px;text-transform:uppercase;font-family:var(--mono);margin-bottom:6px}
        .demo-row{font-size:11px;color:var(--text3);font-family:var(--mono);margin-bottom:3px;display:flex;justify-content:space-between}
        .demo-row code{color:var(--blue)}

        /* ---- RESPONSIVE ---- */
        @media(max-width:900px){
            .left-panel{display:none}
            .right-panel{width:100%;border-left:none;background:var(--bg)}
        }
    </style>
</head>
<body>
<div id="oauth-loading">
    <div class="spinner"></div>
    <div style="text-align: center;">
        <p style="color: #e0e0e0; font-size: 16px; font-weight: 600; margin: 0 0 6px; font-family: 'DM Sans', sans-serif;">Signing you in...</p>
        <p style="color: #8b92a8; font-size: 13px; margin: 0; font-family: 'DM Sans', sans-serif;">Verifying your Google account</p>
    </div>
</div>
<div class="bg-layer"></div>
<div class="grid-overlay"></div>

<!-- LEFT HERO -->
<div class="left-panel">
    <div class="brand">
        <div class="brand-logo">
            <div class="brand-icon">E</div>
            <div>
                <div class="brand-name">EMS Portal</div>
                <div class="brand-tagline">Employee Management System</div>
            </div>
        </div>
    </div>
    <div class="hero-text">
        <h1>Your workspace,<br><span class="hl-blue">one login</span><br>away.</h1>
        <p>Employees, Managers, and Vendors each get their own tailored experience. Sign in to access your personalized dashboard.</p>
    </div>
    <div class="feature-pills">
        <div class="pill blue"><span class="dot"></span>Employee Portal</div>
        <div class="pill violet"><span class="dot"></span>Manager Console</div>
        <div class="pill green"><span class="dot"></span>Vendor Hub</div>
        <div class="pill blue"><span class="dot"></span>Leave Management</div>
        <div class="pill violet"><span class="dot"></span>Meal Subscriptions</div>
        <div class="pill green"><span class="dot"></span>Delivery Tracking</div>
    </div>
</div>

<!-- RIGHT FORM -->
<div class="right-panel">
    <div class="form-box">

        <!-- ROLE SELECTOR -->
        <div class="role-label">Sign in as</div>
        <div class="role-tabs">
            <button class="role-tab active-emp" id="role-emp" onclick="selectRole('EMPLOYEE')">
```

```
                <span class="role-icon"></span>Employee
            </button>
            <button class="role-tab" id="role-mgr" onclick="selectRole('MANAGER')">
                <span class="role-icon"></span>Manager
            </button>
            <button class="role-tab" id="role-vnd" onclick="selectRole('VENDOR')">
                <span class="role-icon"></span>Vendor
            </button>
        </div>

        <!-- AUTH METHOD TABS -->
        <div class="auth-tabs">
            <button class="auth-tab active" id="tab-jwt" onclick="switchTab('jwt')">JWT Login</button>
            <button class="auth-tab" id="tab-session" onclick="switchTab('session')">Session Login</button>
        </div>

        <!-- JWT FORM -->
        <div id="jwt-form">
            <div class="form-group">
                <label class="form-label">Username / Email</label>
                <input class="form-input" id="jwt-username" type="text" placeholder="your.email@company.com" autocomplete="off">
            </div>
            <div class="form-group">
                <label class="form-label">Password</label>
                <input class="form-input" id="jwt-password" type="password" placeholder="••••••••" autocomplete="current-password">
            </div>
            <button class="btn-login blue" id="jwt-btn" onclick="jwtLogin()">Sign In</button>
        </div>

        <!-- SESSION FORM -->
        <div id="session-form" style="display:none">
            <div class="form-group">
                <label class="form-label">Username / Email</label>
                <input class="form-input" id="session-username" type="text" placeholder="your.email@company.com">
            </div>
            <div class="form-group">
                <label class="form-label">Password</label>
                <input class="form-input" id="session-password" type="password" placeholder="••••••••">
            </div>
            <button class="btn-login blue" id="session-btn" onclick="sessionLogin()">Sign In with Session</button>
        </div>

        <!-- OAUTH -->
        <div class="divider">or continue with</div>
        <div class="google-role-row">
            <span class="google-role-label">Sign in as</span>
            <select class="google-role-select" id="oauth-role">
                <option value="EMPLOYEE">Employee</option>
                <option value="MANAGER">Manager</option>
                <option value="VENDOR">Vendor</option>
            </select>
        </div>
        <button class="btn-google" onclick="googleLogin()">
            <svg width="18" height="18" viewBox="0 0 24 24">
                <path fill="#4285F4" d="M22.56 12.25c0-.78-.07-1.53-.2-2.25H12v4.26h5.92c-.26 1.37-1.04 2.53-2.21 3.31v2.77h3.57c2.08-1.92 3.28-4.74 3.28-8.09z"/>
                <path fill="#34A853" d="M12 23c2.97 0 5.46-.98 7.28-2.66l-3.57-2.77c-.98.66-2.23 1.06-3.71 1.06-2.86 0-5.29-1.93-6.16-4.53H2.18v2.84C3.99 20.53 7.7 23 12 23z"/>
                <path fill="#FBBC05" d="M5.84 14.09c-.22-.66-.35-1.36-.35-2.09s.13-1.43.35-2.09V7.07H2.18C1.43 8.55 1 10.22 1 12s.43 3.45 1.18 4.93l2.85-2.22.81-.62z"/>
                <path fill="#EA4335" d="M12 5.38c1.62 0 3.06.56 4.21 1.64l3.15-3.15C17.45 2.09 14.97 1 12 1 7.7 1 3.99 3.47 2.18 7.07l3.66 2.84c.87-2.6 3.3-4.53 6.16-4.53z"/>
            </svg>
            Continue with Google
        </button>

        <div class="status-box" id="status-box"></div>

        <!-- FOOTER -->
        <div class="form-footer">
            <div class="tz-row">
                <span class="tz-label">TZ</span>
```

```
                <select class="tz-select" id="tz-select" onchange="updateTz()">
                    <option value="Asia/Kolkata">India — IST UTC+5:30</option>
                    <option value="UTC">UTC</option>
                    <option value="America/New_York">New York — EST UTC-5</option>
                    <option value="America/Los_Angeles">Los Angeles — PST UTC-8</option>
                    <option value="Europe/London">London — GMT UTC+0</option>
                    <option value="Europe/Paris">Paris — CET UTC+1</option>
                    <option value="Asia/Tokyo">Tokyo — JST UTC+9</option>
                    <option value="Asia/Dubai">Dubai — GST UTC+4</option>
                    <option value="Australia/Sydney">Sydney — AEDT UTC+11</option>
                    <option value="Asia/Singapore">Singapore — SGT UTC+8</option>
                </select>
                <span class="tz-time" id="tz-time">--:--:--</span>
            </div>
            <div class="demo-box">
                <div class="demo-title">Demo Credentials</div>
                <div class="demo-row"><span>Employee</span><code>john.doe / password123</code></div>
                <div class="demo-row"><span>Manager</span><code>jane.manager / manager123</code></div>
                <div class="demo-row"><span>Vendor</span><code>vendor@email.com / vendor123</code></div>
            </div>
        </div>

    </div>
</div>

<script>
    const API = 'http://localhost:8080';
    const GOOGLE_CLIENT_ID = '525324511556-s9eugd1o761n6cjje2nr60cv8ho6joq2.apps.googleusercontent.com';
    const REDIRECT_URI = 'http://localhost:8080/google.html';

    // Role  dashboard mapping
    const DASHBOARDS = {
        EMPLOYEE: '/employee-dashboard.html',
        MANAGER:  '/manager-dashboard.html',
        VENDOR:   '/vendor-dashboard.html'
    };

    let currentRole = 'EMPLOYEE';

    // ---- Auth guard: already logged in? ----
    (function(){
        const token = localStorage.getItem('jwt_token');
        const authType = localStorage.getItem('auth_type');
        if (token || authType === 'session') {
            redirectByRole(token);
        }
    })();

    // ---- Decode JWT and route ----
    function redirectByRole(token) {
        let role = localStorage.getItem('last_role') || 'EMPLOYEE';

        if (token) {
            try {
                const payload = JSON.parse(atob(token.split('.')[1]));
                const authorities =
                    payload.roles ||
                    payload.authorities ||
                    payload.scope ||
                    payload.role ||
                    '';

                const authText = Array.isArray(authorities)
                    ? authorities.join(',')
                    : String(authorities);

                if (authText.includes('MANAGER') || authText.includes('ROLE_MANAGER')) role = 'MANAGER';
                else if (authText.includes('VENDOR') || authText.includes('ROLE_VENDOR')) role = 'VENDOR';
                else if (authText.includes('EMPLOYEE') || authText.includes('ROLE_EMPLOYEE')) role =
'EMPLOYEE';
            } catch (e) {
                console.warn('JWT decode failed, using saved role');
            }
        }

        window.location.href = DASHBOARDS[role] || DASHBOARDS.EMPLOYEE;
```

```
    }


    // ---- Role selector ----
    function selectRole(role) {
        currentRole = role;
        // Reset all tabs
        document.getElementById('role-emp').className = 'role-tab';
        document.getElementById('role-mgr').className = 'role-tab';
        document.getElementById('role-vnd').className = 'role-tab';
        // Activate selected
        const classMap = { EMPLOYEE: 'active-emp', MANAGER: 'active-mgr', VENDOR: 'active-vnd' };
        const idMap = { EMPLOYEE: 'role-emp', MANAGER: 'role-mgr', VENDOR: 'role-vnd' };
        document.getElementById(idMap[role]).classList.add(classMap[role]);
        // Sync google role select
        document.getElementById('oauth-role').value = role;
        // Update button accent color
        const colorMap = { EMPLOYEE: 'blue', MANAGER: 'violet', VENDOR: 'red' };
        const col = colorMap[role];
        document.getElementById('jwt-btn').className = `btn-login ${col}`;
        document.getElementById('session-btn').className = `btn-login ${col}`;
        // Update input focus color
        document.querySelectorAll('.form-input').forEach(inp => {
            inp.className = `form-input focus-${col === 'blue' ? '' : col}`.trim();
        });
    }

    // ---- Auth tab switching ----
    function switchTab(type) {
        document.getElementById('jwt-form').style.display = type === 'jwt' ? 'block' : 'none';
        document.getElementById('session-form').style.display = type === 'session' ? 'block' : 'none';
        document.getElementById('tab-jwt').classList.toggle('active', type === 'jwt');
        document.getElementById('tab-session').classList.toggle('active', type === 'session');
        hideStatus();
    }

    // ---- Status display ----
    function showStatus(type, msg) {
        const box = document.getElementById('status-box');
        box.className = `status-box ${type}`;
        box.textContent = msg;
        box.style.display = 'block';
    }
    function hideStatus() {
        document.getElementById('status-box').style.display = 'none';
    }

    // ---- JWT Login ----
    async function jwtLogin() {
        const username = document.getElementById('jwt-username').value.trim();
        const password = document.getElementById('jwt-password').value;
        if (!username || !password) { showStatus('error', 'Please enter your username and password');
return; }
        const btn = document.getElementById('jwt-btn');
        btn.classList.add('loading'); btn.textContent = 'Signing in...';
        showStatus('loading', 'Authenticating...');
        try {
            const res = await fetch(`${API}/Authenticate`, {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ username, password })
            });
            if (res.ok) {
                const data = await res.json();
                localStorage.setItem('jwt_token', data.accessToken);
                localStorage.setItem('refresh_token', data.refreshToken || '');
                localStorage.setItem('auth_type', 'jwt');
                localStorage.setItem('last_role', currentRole);
                localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
                showStatus('success', 'Login successful! Redirecting...');
                setTimeout(() => redirectByRole(data.accessToken), 600);
            } else {
                const err = await res.text();
                showStatus('error', 'Login failed: ' + (err || 'Invalid credentials'));
                btn.classList.remove('loading');
                btn.textContent = 'Sign In';
```

```
            }
        } catch(e) {
            showStatus('error', 'Connection error: ' + e.message);
            btn.classList.remove('loading'); btn.textContent = 'Sign In';
        }
    }

    // ---- Session Login ----
    async function sessionLogin() {
        const username = document.getElementById('session-username').value.trim();
        const password = document.getElementById('session-password').value;
        if (!username || !password) { showStatus('error', 'Please enter your username and password');
return; }
        const btn = document.getElementById('session-btn');
        btn.classList.add('loading'); btn.textContent = 'Signing in...';
        showStatus('loading', 'Authenticating...');
        try {
            const res = await fetch(`${API}/session/login`, {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ username, password }),
                credentials: 'include'
            });
            if (res.ok) {
                localStorage.setItem('auth_type', 'session');
                localStorage.setItem('last_role', currentRole);
                localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
                showStatus('success', 'Login successful! Redirecting...');
                setTimeout(() => window.location.href = DASHBOARDS[currentRole], 600);
            } else {
                const err = await res.json().catch(() => ({}));
                showStatus('error', 'Login failed: ' + (err.error || 'Invalid credentials'));
                btn.classList.remove('loading'); btn.textContent = 'Sign In with Session';
            }
        } catch(e) {
            showStatus('error', 'Connection error: ' + e.message);
            btn.classList.remove('loading'); btn.textContent = 'Sign In with Session';
        }
    }

    // ---- Google OAuth ----
    function googleLogin() {
        const role = document.getElementById('oauth-role').value;
        localStorage.setItem('oauth_role', role);
        localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
        const scope = 'https://www.googleapis.com/auth/userinfo.email
https://www.googleapis.com/auth/userinfo.profile';
        window.location.href =
            `https://accounts.google.com/o/oauth2/v2/auth` +
            `?client_id=${encodeURIComponent(GOOGLE_CLIENT_ID)}` +
            `&redirect_uri=${encodeURIComponent(REDIRECT_URI)}` +
            `&response_type=code` +
            `&scope=${encodeURIComponent(scope)}`;
    }

    // ---- OAuth callback handler (runs when Google redirects back here) ----
    async function handleOAuthCallback() {
        const params = new URLSearchParams(window.location.search);
        const code = params.get('code');
        if (!code) return;
        // Clear the code from URL immediately
        window.history.replaceState({}, document.title, window.location.pathname);
        showStatus('loading', 'Signing you in with Google...');
        const role = localStorage.getItem('oauth_role') || 'EMPLOYEE';
        const timezone = localStorage.getItem('user_timezone') || 'Asia/Kolkata';
        try {
            const res = await fetch(
    `${API}/auth/google/callback` +
    `?code=${encodeURIComponent(code)}` +
    `&role=${encodeURIComponent(role)}` +
    `&timezone=${encodeURIComponent(timezone)}`  //  ADD
      );

            if (!res.ok) {
                const txt = await res.text();
                showStatus('error', 'Google sign-in failed. Please try again.');
```

```
                console.error('OAuth error:', res.status, txt);
                return;
            }
            const data = await res.json();
            if (data.accessToken) {
                localStorage.setItem('jwt_token', data.accessToken);
                localStorage.setItem('refresh_token', data.refreshToken || '');
                localStorage.setItem('auth_type', 'jwt');
                localStorage.setItem('last_role', role);
                localStorage.removeItem('oauth_role');
                showStatus('success', 'Signed in! Redirecting to your dashboard...');
                setTimeout(() => redirectByRole(data.accessToken), 800);
            } else {
                showStatus('error', 'Sign-in failed: no token received.');
            }
        } catch(e) {
            showStatus('error', 'Server error: ' + e.message);
        }
    }

    // ---- Timezone ----
    function updateTz() {
        const tz = document.getElementById('tz-select').value;
        localStorage.setItem('user_timezone', tz);
        refreshTzClock();
    }
    function refreshTzClock() {
        const tz = document.getElementById('tz-select').value;
        const t = new Date().toLocaleTimeString('en-GB', { timeZone: tz, hour12: false });
        document.getElementById('tz-time').textContent = t;
    }

    // ---- Enter key support ----
    document.addEventListener('keydown', e => {
        if (e.key !== 'Enter') return;
        if (document.getElementById('jwt-form').style.display !== 'none') jwtLogin();
        else sessionLogin();
    });

    // ---- Init ----
    window.onload = () => {
        const savedTz = localStorage.getItem('user_timezone') || 'Asia/Kolkata';
        document.getElementById('tz-select').value = savedTz;
        refreshTzClock();
        setInterval(refreshTzClock, 1000);

    const params = new URLSearchParams(window.location.search);
    const code = params.get('code');

    if (code) {
        // Coming back from Google — show loading overlay, hide form
        document.getElementById('oauth-loading').classList.add('visible');
        document.querySelector('.right-panel').style.display = 'none';
        document.querySelector('.left-panel').style.display = 'none';
        handleOAuthCallback();
    }
   };
</script>
</body>
</html>
```

94 Current: src\main\resources\static\login.html 586 lines

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>EMS — Sign In</title>
    <link
href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;500;600;700&family=JetBrains+Mono:wght@400;50
0&display=swap" rel="stylesheet">
    <style>
        *{margin:0;padding:0;box-sizing:border-box}
        :root{
            --bg:#070910;
            --panel:#0e1118;
            --panel2:#131821;
            --border:rgba(255,255,255,0.07);
            --border-lit:rgba(255,255,255,0.14);
            --text:#dde4f0;--text2:#6b7591;--text3:#3d4560;
            --blue:#4f8cff;--blue-dim:rgba(79,140,255,0.12);
            --violet:#a78bfa;--violet-dim:rgba(167,139,250,0.12);
            --red:#f87171;--red-dim:rgba(248,113,113,0.12);
            --green:#34d399;--green-dim:rgba(52,211,153,0.12);
            --font:'Outfit',sans-serif;--mono:'JetBrains Mono',monospace;
        }

        body{
            font-family:var(--font);
            background:var(--bg);
            color:var(--text);
            min-height:100vh;
            display:flex;
            overflow:hidden;
        }

        /* ---- BACKGROUND ---- */
        .bg-layer{
            position:fixed;inset:0;z-index:0;
            background:
                    radial-gradient(ellipse 60% 50% at 20% 50%, rgba(79,140,255,0.06) 0%, transparent 70%),
                    radial-gradient(ellipse 40% 60% at 80% 20%, rgba(167,139,250,0.05) 0%, transparent 60%),
                    radial-gradient(ellipse 50% 40% at 70% 80%, rgba(52,211,153,0.04) 0%, transparent 60%);
        }
        .grid-overlay{
            position:fixed;inset:0;z-index:0;
            background-image:
                    linear-gradient(rgba(255,255,255,0.025) 1px, transparent 1px),
                    linear-gradient(90deg, rgba(255,255,255,0.025) 1px, transparent 1px);
            background-size:40px 40px;
            mask-image:radial-gradient(ellipse 80% 80% at center, black 20%, transparent 80%);
        }

        /* ---- SPLIT LAYOUT ---- */
        .left-panel{
            flex:1;display:flex;flex-direction:column;justify-content:center;
            padding:60px;position:relative;z-index:1;
            animation:fadeLeft 0.8s ease both;
        }
        @keyframes fadeLeft{from{opacity:0;transform:translateX(-
24px)}to{opacity:1;transform:translateX(0)}}
        .brand{margin-bottom:48px}
        .brand-logo{display:flex;align-items:center;gap:12px;margin-bottom:20px}
        .brand-icon{
            width:44px;height:44px;background:linear-gradient(135deg,#4f8cff,#6366f1);
            border-radius:12px;display:flex;align-items:center;justify-content:center;
            font-weight:700;font-size:18px;color:#fff;
            box-shadow:0 0 30px rgba(79,140,255,0.35);
            flex-shrink:0;
        }
        .brand-name{font-size:22px;font-weight:700;color:var(--text);letter-spacing:-0.3px}
        .brand-tagline{font-size:13px;color:var(--text2)}
        .hero-text h1{
            font-size:clamp(36px,4vw,52px);font-weight:700;
```

```
            line-height:1.1;letter-spacing:-1px;color:var(--text);margin-bottom:16px;
        }
        .hero-text h1 .hl-blue{color:var(--blue)}
        .hero-text h1 .hl-violet{color:var(--violet)}
        .hero-text h1 .hl-green{color:var(--green)}
        .hero-text p{font-size:15px;color:var(--text2);line-height:1.6;max-width:380px}
        .feature-pills{display:flex;flex-wrap:wrap;gap:8px;margin-top:32px}
        .pill{
            display:inline-flex;align-items:center;gap:6px;
            padding:6px 14px;border-radius:20px;font-size:12px;font-weight:500;
            border:1px solid var(--border);background:var(--panel);color:var(--text2);
        }
        .pill .dot{width:6px;height:6px;border-radius:50%;flex-shrink:0}
        .pill.blue .dot{background:var(--blue)} .pill.blue{border-color:rgba(79,140,255,0.2);color:rgba(79,140,255,0.8)}
        .pill.violet .dot{background:var(--violet)} .pill.violet{border-color:rgba(167,139,250,0.2);color:rgba(167,139,250,0.8)}
        .pill.green .dot{background:var(--green)} .pill.green{border-color:rgba(52,211,153,0.2);color:rgba(52,211,153,0.8)}

        /* ---- RIGHT PANEL (FORM) ---- */
        .right-panel{
            width:460px;flex-shrink:0;
            display:flex;align-items:center;justify-content:center;
            padding:32px 40px;position:relative;z-index:1;
            border-left:1px solid var(--border);
            background:rgba(14,17,24,0.8);
            backdrop-filter:blur(20px);
            animation:fadeRight 0.8s ease both;
        }
        @keyframes fadeRight{from{opacity:0;transform:translateX(24px)}to{opacity:1;transform:translateX(0)}}
        .form-box{width:100%}

         /* ---- ROLE SELECTOR ---- */
         .role-label{font-size:10px;font-weight:600;color:var(--text3);letter-spacing:1.5px;text-transform:uppercase;font-family:var(--mono);margin-bottom:10px}
         .role-tabs{display:grid;grid-template-columns:1fr 1fr 1fr;gap:6px;margin-bottom:28px}
         .role-tab{
             padding:10px 8px;border-radius:10px;border:1.5px solid var(--border);
             background:var(--bg);color:var(--text2);font-family:var(--font);
             font-size:12px;font-weight:600;cursor:pointer;transition:all 0.2s;
             display:flex;flex-direction:column;align-items:center;gap:4px;
         }
         .role-tab .role-icon{font-size:16px}
         .role-tab:hover{border-color:var(--border-lit);color:var(--text)}
         .role-tab.active-emp{border-color:var(--blue);background:var(--blue-dim);color:var(--blue)}
         .role-tab.active-mgr{border-color:var(--violet);background:var(--violet-dim);color:var(--violet)}
         .role-tab.active-vnd{border-color:var(--red);background:var(--red-dim);color:var(--red)}

         /* ---- AUTH TABS ---- */
         .auth-tabs{display:flex;background:var(--bg);border-radius:10px;padding:4px;gap:4px;margin-bottom:24px;border:1px solid var(--border)}
         .auth-tab{
             flex:1;padding:9px;border:none;background:transparent;
             color:var(--text2);font-family:var(--font);font-weight:600;
             font-size:13px;cursor:pointer;border-radius:7px;transition:all 0.2s;
         }
         .auth-tab.active{background:var(--panel2);color:var(--text);box-shadow:0 1px 4px rgba(0,0,0,0.3)}

         /* ---- FORM ELEMENTS ---- */
         .form-group{margin-bottom:16px}
         .form-label{display:block;font-size:10px;font-weight:600;color:var(--text2);text-transform:uppercase;letter-spacing:0.8px;font-family:var(--mono);margin-bottom:7px}
         .form-input{
             width:100%;padding:11px 14px;
             background:var(--bg);border:1px solid var(--border);
             border-radius:9px;color:var(--text);
             font-family:var(--font);font-size:14px;
             transition:all 0.2s;
         }
         .form-input:focus{outline:none;border-color:var(--blue);box-shadow:0 0 0 3px rgba(79,140,255,0.08)}
         .form-input::placeholder{color:var(--text3)}
         .form-input.focus-violet:focus{border-color:var(--violet);box-shadow:0 0 0 3px rgba(167,139,250,0.08)}
         .form-input.focus-red:focus{border-color:var(--red);box-shadow:0 0 0 3px rgba(248,113,113,0.08)}
```

```
        /* ---- BUTTONS ---- */
        .btn-login{
            width:100%;padding:12px;border:none;border-radius:9px;
            font-family:var(--font);font-size:14px;font-weight:600;
            cursor:pointer;transition:all 0.2s;margin-top:4px;
            position:relative;overflow:hidden;
        }
        .btn-login.blue{background:var(--blue);color:#fff}
        .btn-login.blue:hover{background:#6ba0ff;box-shadow:0 6px 20px rgba(79,140,255,0.35);transform:translateY(-1px)}
        .btn-login.violet{background:var(--violet);color:#fff}
        .btn-login.violet:hover{background:#b8a0fc;box-shadow:0 6px 20px rgba(167,139,250,0.35);transform:translateY(-1px)}
        .btn-login.red{background:var(--red);color:#fff}
        .btn-login.red:hover{background:#f98a8a;box-shadow:0 6px 20px rgba(248,113,113,0.35);transform:translateY(-1px)}
        .btn-login:active{transform:translateY(0)}
        .btn-login.loading{opacity:0.7;pointer-events:none}

        .divider{display:flex;align-items:center;gap:12px;margin:20px 0;color:var(--text3);font-size:11px;font-family:var(--mono)}
        .divider::before,.divider::after{content:'';flex:1;height:1px;background:var(--border)}

        .btn-google{
            width:100%;padding:11px;background:transparent;
            border:1px solid var(--border-lit);border-radius:9px;
            color:var(--text2);font-family:var(--font);font-size:13px;font-weight:500;
            cursor:pointer;transition:all 0.2s;
            display:flex;align-items:center;justify-content:center;gap:10px;
        }
        .btn-google:hover{background:rgba(255,255,255,0.04);border-color:rgba(255,255,255,0.2);color:var(--text)}

        /* ---- GOOGLE ROLE SELECT ---- */
        .google-role-row{display:flex;gap:8px;margin-bottom:10px;align-items:center}
        .google-role-label{font-size:11px;color:var(--text3);font-family:var(--mono);white-space:nowrap}
        .google-role-select{
            flex:1;padding:8px 10px;background:var(--bg);
            border:1px solid var(--border);border-radius:8px;
            color:var(--text);font-family:var(--font);font-size:13px;cursor:pointer;
        }
        .google-role-select:focus{outline:none;border-color:var(--border-lit)}
        .google-role-select option{background:var(--panel)}

        /* ---- ERROR / STATUS ---- */
        .status-box{
            border-radius:9px;padding:10px 14px;margin-top:14px;
            font-size:13px;font-weight:500;display:none;
            animation:fadeIn 0.3s ease;
        }
        @keyframes fadeIn{from{opacity:0;transform:translateY(-4px)}to{opacity:1;transform:translateY(0)}}
        .status-box.error{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
        .status-box.success{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.2)}
        .status-box.loading{background:var(--blue-dim);color:var(--blue);border:1px solid rgba(79,140,255,0.2)}

        /* ---- FOOTER ---- */
        .form-footer{margin-top:20px;padding-top:18px;border-top:1px solid var(--border)}
        .tz-row{display:flex;align-items:center;gap:8px;margin-bottom:8px}
        .tz-label{font-size:10px;color:var(--text3);font-family:var(--mono);flex-shrink:0}
        .tz-select{
            flex:1;padding:6px 8px;background:var(--bg);border:1px solid var(--border);
            border-radius:7px;color:var(--text2);font-family:var(--mono);font-size:11px;cursor:pointer;
        }
        .tz-select:focus{outline:none;border-color:var(--border-lit)}
        .tz-select option{background:var(--panel)}
        .tz-time{font-size:10px;color:var(--blue);font-family:var(--mono)}
        .demo-box{background:rgba(79,140,255,0.05);border:1px solid rgba(79,140,255,0.1);border-radius:8px;padding:10px 12px;margin-top:8px}
        .demo-title{font-size:9px;font-weight:700;color:var(--text3);letter-spacing:1.5px;text-transform:uppercase;font-family:var(--mono);margin-bottom:6px}
        .demo-row{font-size:11px;color:var(--text3);font-family:var(--mono);margin-bottom:3px;display:flex;justify-content:space-between}
```

```
        .demo-row code{color:var(--blue)}

        /* ---- RESPONSIVE ---- */
        @media(max-width:900px){
            .left-panel{display:none}
            .right-panel{width:100%;border-left:none;background:var(--bg)}
        }
    </style>
</head>
<body>
<div class="bg-layer"></div>
<div class="grid-overlay"></div>

<!-- LEFT HERO -->
<div class="left-panel">
    <div class="brand">
        <div class="brand-logo">
            <div class="brand-icon">E</div>
            <div>
                <div class="brand-name">EMS Portal</div>
                <div class="brand-tagline">Employee Management System</div>
            </div>
        </div>
    </div>
    <div class="hero-text">
        <h1>Your workspace,<br><span class="hl-blue">one login</span><br>away.</h1>
        <p>Employees, Managers, and Vendors each get their own tailored experience. Sign in to access your personalized dashboard.</p>
    </div>
    <div class="feature-pills">
        <div class="pill blue"><span class="dot"></span>Employee Portal</div>
        <div class="pill violet"><span class="dot"></span>Manager Console</div>
        <div class="pill green"><span class="dot"></span>Vendor Hub</div>
        <div class="pill blue"><span class="dot"></span>Leave Management</div>
        <div class="pill violet"><span class="dot"></span>Meal Subscriptions</div>
        <div class="pill green"><span class="dot"></span>Delivery Tracking</div>
    </div>
</div>

<!-- RIGHT FORM -->
<div class="right-panel">
    <div class="form-box">

        <!-- ROLE SELECTOR -->
        <div class="role-label">Sign in as</div>
        <div class="role-tabs">
            <button class="role-tab active-emp" id="role-emp" onclick="selectRole('EMPLOYEE')">
                <span class="role-icon"></span>Employee
            </button>
            <button class="role-tab" id="role-mgr" onclick="selectRole('MANAGER')">
                <span class="role-icon"></span>Manager
            </button>
            <button class="role-tab" id="role-vnd" onclick="selectRole('VENDOR')">
                <span class="role-icon"></span>Vendor
            </button>
        </div>

        <!-- AUTH METHOD TABS -->
        <div class="auth-tabs">
            <button class="auth-tab active" id="tab-jwt" onclick="switchTab('jwt')">JWT Login</button>
            <button class="auth-tab" id="tab-session" onclick="switchTab('session')">Session Login</button>
        </div>

        <!-- JWT FORM -->
        <div id="jwt-form">
            <div class="form-group">
                <label class="form-label">Username / Email</label>
                <input class="form-input" id="jwt-username" type="text" placeholder="your.email@company.com" autocomplete="off">
            </div>
            <div class="form-group">
                <label class="form-label">Password</label>
                <input class="form-input" id="jwt-password" type="password" placeholder="••••••••" autocomplete="current-password">
            </div>
            <button class="btn-login blue" id="jwt-btn" onclick="jwtLogin()">Sign In</button>
```

```
        </div>

        <!-- SESSION FORM -->
        <div id="session-form" style="display:none">
            <div class="form-group">
                <label class="form-label">Username / Email</label>
                <input class="form-input" id="session-username" type="text" placeholder="your.email@company.com">
            </div>
            <div class="form-group">
                <label class="form-label">Password</label>
                <input class="form-input" id="session-password" type="password" placeholder="••••••••">
            </div>
            <button class="btn-login blue" id="session-btn" onclick="sessionLogin()">Sign In with Session</button>
        </div>

        <!-- OAUTH -->
        <div class="divider">or continue with</div>
        <div class="google-role-row">
            <span class="google-role-label">Sign in as</span>
            <select class="google-role-select" id="oauth-role">
                <option value="EMPLOYEE">Employee</option>
                <option value="MANAGER">Manager</option>
                <option value="VENDOR">Vendor</option>
            </select>
        </div>
        <button class="btn-google" onclick="googleLogin()">
            <svg width="18" height="18" viewBox="0 0 24 24">
                <path fill="#4285F4" d="M22.56 12.25c0-.78-.07-1.53-.2-2.25H12v4.26h5.92c-.26 1.37-1.04 2.53-2.21 3.31v2.77h3.57c2.08-1.92 3.28-4.74 3.28-8.09z"/>
                <path fill="#34A853" d="M12 23c2.97 0 5.46-.98 7.28-2.66l-3.57-2.77c-.98.66-2.23 1.06-3.71 1.06-2.86 0-5.29-1.93-6.16-4.53H2.18v2.84C3.99 20.53 7.7 23 12 23z"/>
                <path fill="#FBBC05" d="M5.84 14.09c-.22-.66-.35-1.36-.35-2.09s.13-1.43.35-2.09V7.07H2.18C1.43 8.55 1 10.22 1 12s.43 3.45 1.18 4.93l2.85-2.22.81-.62z"/>
                <path fill="#EA4335" d="M12 5.38c1.62 0 3.06.56 4.21 1.64l3.15-3.15C17.45 2.09 14.97 1 12 1 7.7 1 3.99 3.47 2.18 7.07l3.66 2.84c.87-2.6 3.3-4.53 6.16-4.53z"/>
            </svg>
            Continue with Google
        </button>

        <div class="status-box" id="status-box"></div>

        <!-- FOOTER -->
        <div class="form-footer">
            <div class="tz-row">
                <span class="tz-label">TZ</span>
                <select class="tz-select" id="tz-select" onchange="updateTz()">
                    <option value="Asia/Kolkata">India — IST UTC+5:30</option>
                    <option value="UTC">UTC</option>
                    <option value="America/New_York">New York — EST UTC-5</option>
                    <option value="America/Los_Angeles">Los Angeles — PST UTC-8</option>
                    <option value="Europe/London">London — GMT UTC+0</option>
                    <option value="Europe/Paris">Paris — CET UTC+1</option>
                    <option value="Asia/Tokyo">Tokyo — JST UTC+9</option>
                    <option value="Asia/Dubai">Dubai — GST UTC+4</option>
                    <option value="Australia/Sydney">Sydney — AEDT UTC+11</option>
                    <option value="Asia/Singapore">Singapore — SGT UTC+8</option>
                </select>
                <span class="tz-time" id="tz-time">--:--:--</span>
            </div>
            <div class="demo-box">
                <div class="demo-title">Demo Credentials</div>
                <div class="demo-row"><span>Employee</span><code>john.doe / password123</code></div>
                <div class="demo-row"><span>Manager</span><code>jane.manager / manager123</code></div>
                <div class="demo-row"><span>Vendor</span><code>vendor@email.com / vendor123</code></div>
            </div>
        </div>

    </div>
</div>

<script>
    const API = 'http://localhost:8080';
    const GOOGLE_CLIENT_ID = '525324511556-s9eugd1o761n6cjje2nr60cv8ho6joq2.apps.googleusercontent.com';
    const REDIRECT_URI = 'http://localhost:8080/login.html';
```

```

    // Role  dashboard mapping
    const DASHBOARDS = {
        EMPLOYEE: '/employee-dashboard.html',
        MANAGER:  '/manager-dashboard.html',
        VENDOR:   '/vendor-dashboard.html'
    };

    let currentRole = 'EMPLOYEE';

    // ---- Auth guard: already logged in? ----
    (function(){
        const token = localStorage.getItem('jwt_token');
        const authType = localStorage.getItem('auth_type');
        if (token || authType === 'session') {
            redirectByRole(token);
        }
    })();

    // ---- Decode JWT and route ----
    function redirectByRole(token) {
        let role = localStorage.getItem('last_role') || 'EMPLOYEE';

        if (token) {
            try {
                const payload = JSON.parse(atob(token.split('.')[1]));
                const authorities =
                    payload.roles ||
                    payload.authorities ||
                    payload.scope ||
                    payload.role ||
                    '';

                const authText = Array.isArray(authorities)
                    ? authorities.join(',')
                    : String(authorities);

                if (authText.includes('MANAGER') || authText.includes('ROLE_MANAGER')) role = 'MANAGER';
                else if (authText.includes('VENDOR') || authText.includes('ROLE_VENDOR')) role = 'VENDOR';
                else if (authText.includes('EMPLOYEE') || authText.includes('ROLE_EMPLOYEE')) role =
'EMPLOYEE';
            } catch (e) {
                console.warn('JWT decode failed, using saved role');
            }
        }

        window.location.href = DASHBOARDS[role] || DASHBOARDS.EMPLOYEE;
    }


    // ---- Role selector ----
    function selectRole(role) {
        currentRole = role;
        // Reset all tabs
        document.getElementById('role-emp').className = 'role-tab';
        document.getElementById('role-mgr').className = 'role-tab';
        document.getElementById('role-vnd').className = 'role-tab';
        // Activate selected
        const classMap = { EMPLOYEE: 'active-emp', MANAGER: 'active-mgr', VENDOR: 'active-vnd' };
        const idMap = { EMPLOYEE: 'role-emp', MANAGER: 'role-mgr', VENDOR: 'role-vnd' };
        document.getElementById(idMap[role]).classList.add(classMap[role]);
        // Sync google role select
        document.getElementById('oauth-role').value = role;
        // Update button accent color
        const colorMap = { EMPLOYEE: 'blue', MANAGER: 'violet', VENDOR: 'red' };
        const col = colorMap[role];
        document.getElementById('jwt-btn').className = `btn-login ${col}`;
        document.getElementById('session-btn').className = `btn-login ${col}`;
        // Update input focus color
        document.querySelectorAll('.form-input').forEach(inp => {
            inp.className = `form-input focus-${col === 'blue' ? '' : col}`.trim();
        });
    }

    // ---- Auth tab switching ----
    function switchTab(type) {
```

```
        document.getElementById('jwt-form').style.display = type === 'jwt' ? 'block' : 'none';
        document.getElementById('session-form').style.display = type === 'session' ? 'block' : 'none';
        document.getElementById('tab-jwt').classList.toggle('active', type === 'jwt');
        document.getElementById('tab-session').classList.toggle('active', type === 'session');
        hideStatus();
    }

    // ---- Status display ----
    function showStatus(type, msg) {
        const box = document.getElementById('status-box');
        box.className = `status-box ${type}`;
        box.textContent = msg;
        box.style.display = 'block';
    }
    function hideStatus() {
        document.getElementById('status-box').style.display = 'none';
    }

    // ---- JWT Login ----
    async function jwtLogin() {
        const username = document.getElementById('jwt-username').value.trim();
        const password = document.getElementById('jwt-password').value;
        if (!username || !password) { showStatus('error', 'Please enter your username and password'); return; }
        const btn = document.getElementById('jwt-btn');
        btn.classList.add('loading'); btn.textContent = 'Signing in...';
        showStatus('loading', 'Authenticating...');
        try {
            const res = await fetch(`${API}/Authenticate`, {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ username, password })
            });
            if (res.ok) {
                const data = await res.json();
                localStorage.setItem('jwt_token', data.accessToken);
                localStorage.setItem('refresh_token', data.refreshToken || '');
                localStorage.setItem('auth_type', 'jwt');
                localStorage.setItem('last_role', currentRole);
                localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
                showStatus('success', 'Login successful! Redirecting...');
                setTimeout(() => redirectByRole(data.accessToken), 600);
            } else {
                const err = await res.text();
                showStatus('error', 'Login failed: ' + (err || 'Invalid credentials'));
                btn.classList.remove('loading');
                btn.textContent = 'Sign In';
            }
        } catch(e) {
            showStatus('error', 'Connection error: ' + e.message);
            btn.classList.remove('loading'); btn.textContent = 'Sign In';
        }
    }

    // ---- Session Login ----
    async function sessionLogin() {
        const username = document.getElementById('session-username').value.trim();
        const password = document.getElementById('session-password').value;
        if (!username || !password) { showStatus('error', 'Please enter your username and password'); return; }
        const btn = document.getElementById('session-btn');
        btn.classList.add('loading'); btn.textContent = 'Signing in...';
        showStatus('loading', 'Authenticating...');
        try {
            const res = await fetch(`${API}/session/login`, {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ username, password }),
                credentials: 'include'
            });
            if (res.ok) {
                localStorage.setItem('auth_type', 'session');
                localStorage.setItem('last_role', currentRole);
                localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
                showStatus('success', 'Login successful! Redirecting...');
                setTimeout(() => window.location.href = DASHBOARDS[currentRole], 600);
```

```
            } else {
                const err = await res.json().catch(() => ({}));
                showStatus('error', 'Login failed: ' + (err.error || 'Invalid credentials'));
                btn.classList.remove('loading'); btn.textContent = 'Sign In with Session';
            }
        } catch(e) {
            showStatus('error', 'Connection error: ' + e.message);
            btn.classList.remove('loading'); btn.textContent = 'Sign In with Session';
        }
    }

    // ---- Google OAuth ----
    function googleLogin() {
        const role = document.getElementById('oauth-role').value;
        localStorage.setItem('oauth_role', role);
        localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
        const scope = 'https://www.googleapis.com/auth/userinfo.email https://www.googleapis.com/auth/userinfo.profile';
        window.location.href =
            `https://accounts.google.com/o/oauth2/v2/auth` +
            `?client_id=${encodeURIComponent(GOOGLE_CLIENT_ID)}` +
            `&redirect_uri=${encodeURIComponent(REDIRECT_URI)}` +
            `&response_type=code` +
            `&scope=${encodeURIComponent(scope)}`;
    }

    // ---- OAuth callback handler (runs when Google redirects back here) ----
    async function handleOAuthCallback() {
        const params = new URLSearchParams(window.location.search);
        const code = params.get('code');
        if (!code) return;
        // Clear the code from URL immediately
        window.history.replaceState({}, document.title, window.location.pathname);
        showStatus('loading', 'Signing you in with Google...');
        const role = localStorage.getItem('oauth_role') || 'EMPLOYEE';
        try {
            const res = await fetch(
                `${API}/auth/google/callback?code=${encodeURIComponent(code)}&role=${encodeURIComponent(role)}`
            );
            if (!res.ok) {
                const txt = await res.text();
                showStatus('error', 'Google sign-in failed. Please try again.');
                console.error('OAuth error:', res.status, txt);
                return;
            }
            const data = await res.json();
            if (data.accessToken) {
                localStorage.setItem('jwt_token', data.accessToken);
                localStorage.setItem('refresh_token', data.refreshToken || '');
                localStorage.setItem('auth_type', 'jwt');
                localStorage.removeItem('oauth_role');
                showStatus('success', 'Signed in! Redirecting to your dashboard...');
                setTimeout(() => redirectByRole(data.accessToken), 800);
            } else {
                showStatus('error', 'Sign-in failed: no token received.');
            }
        } catch(e) {
            showStatus('error', 'Server error: ' + e.message);
        }
    }

    // ---- Timezone ----
    function updateTz() {
        const tz = document.getElementById('tz-select').value;
        localStorage.setItem('user_timezone', tz);
        refreshTzClock();
    }
    function refreshTzClock() {
        const tz = document.getElementById('tz-select').value;
        const t = new Date().toLocaleTimeString('en-GB', { timeZone: tz, hour12: false });
        document.getElementById('tz-time').textContent = t;
    }

    // ---- Enter key support ----
    document.addEventListener('keydown', e => {
```

```
        if (e.key !== 'Enter') return;
        if (document.getElementById('jwt-form').style.display !== 'none') jwtLogin();
        else sessionLogin();
    });

    // ---- Init ----
    window.onload = () => {
        // Load saved timezone
        const savedTz = localStorage.getItem('user_timezone') || 'Asia/Kolkata';
        document.getElementById('tz-select').value = savedTz;
        refreshTzClock();
        setInterval(refreshTzClock, 1000);
        // Handle OAuth callback
        handleOAuthCallback();
    };
</script>
</body>
</html>
```

95 Current: src\main\resources\static\manager-dashboard.html 604 lines

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>EMS — Manager Console</title>
    <link href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@300;400;500;600;700&family=IBM+Plex+Mono:wght@400;500&display=swap" rel="stylesheet">
    <style>
        *{margin:0;padding:0;box-sizing:border-box}
        :root{
            --bg:#0a0b0f;--bg2:#0f1117;--bg3:#14171f;
            --panel:#171c28;--panel2:#1d2336;
            --border:rgba(167,139,250,0.08);--border2:rgba(167,139,250,0.18);
            --text:#e8e0f5;--text2:#8b82a8;--text3:#504d68;
            --accent:#a78bfa;--accent-dim:rgba(167,139,250,0.1);
            --green:#34d399;--green-dim:rgba(52,211,153,0.1);
            --amber:#fbbf24;--amber-dim:rgba(251,191,36,0.1);
            --red:#f87171;--red-dim:rgba(248,113,113,0.1);
            --blue:#60a5fa;--blue-dim:rgba(96,165,250,0.1);
            --font:'Plus Jakarta Sans',sans-serif;--mono:'IBM Plex Mono',monospace;
            --sidebar:250px;
        }
        body{font-family:var(--font);background:var(--bg);color:var(--text);font-size:14px;overflow:hidden;height:100vh}
        .app{display:flex;height:100vh}

        /* SIDEBAR */
        .sidebar{width:var(--sidebar);background:var(--bg2);border-right:1px solid var(--border);display:flex;flex-direction:column;flex-shrink:0;position:relative;overflow:hidden}
        .sidebar::after{content:'';position:absolute;bottom:0;left:0;right:0;height:200px;background:linear-gradient(to top,rgba(167,139,250,0.04),transparent);pointer-events:none}
        .logo{padding:22px 20px;border-bottom:1px solid var(--border);position:relative}
        .logo-row{display:flex;align-items:center;gap:10px;margin-bottom:4px}
        .logo-icon{width:34px;height:34px;background:linear-gradient(135deg,var(--accent),#7c3aed);border-radius:10px;display:flex;align-items:center;justify-content:center;font-weight:700;font-siz...
        .logo h2{font-size:16px;font-weight:700;color:var(--text)}
        .logo p{font-size:11px;color:var(--text3);font-family:var(--mono)}
        .role-badge{display:inline-flex;align-items:center;gap:6px;padding:4px 12px;background:linear-gradient(90deg,rgba(167,139,250,0.15),rgba(124,58,237,0.08));border:1px solid rgba(167,139,250,...
        .role-dot{width:6px;height:6px;background:var(--accent);border-radius:50%;animation:glow 2s infinite}
        @keyframes glow{0%,100%{box-shadow:0 0 4px var(--accent)}50%{box-shadow:0 0 10px var(--accent),0 0 20px rgba(167,139,250,0.4)}}
        .nav{flex:1;padding:10px 0;overflow-y:auto}
        .nav-section{padding:12px 20px 6px;font-size:9px;font-weight:700;color:var(--text3);letter-spacing:2px;text-transform:uppercase}
        .nav-item{display:flex;align-items:center;gap:10px;padding:10px 20px;margin:1px 8px;border-radius:8px;cursor:pointer;transition:all 0.2s;color:var(--text2);font-size:13px;font-weight:500}
        .nav-item:hover{background:var(--accent-dim);color:var(--accent)}
        .nav-item.active{background:linear-gradient(90deg,var(--accent-dim),rgba(124,58,237,0.06));color:var(--accent);border:1px solid rgba(167,139,250,0.2)}
        .nav-badge{margin-left:auto;background:var(--red);color:#fff;font-size:10px;font-weight:700;padding:2px 7px;border-radius:10px;min-width:18px;text-align:center}
        .nav-badge.hidden{display:none}
        .sidebar-footer{padding:16px 20px;border-top:1px solid var(--border)}
        .user-card{display:flex;align-items:center;gap:10px}
        .avatar{width:34px;height:34px;background:linear-gradient(135deg,#7c3aed,var(--accent));border-radius:50%;display:flex;align-items:center;justify-content:center;font-size:14px;font-weight:7...
        .user-info-side .name{font-size:12px;font-weight:600;color:var(--text)}
        .user-info-side .role-text{font-size:10px;color:var(--accent);font-family:var(--mono);text-transform:uppercase;letter-spacing:0.8px}

        /* MAIN */
        .main{flex:1;display:flex;flex-direction:column;overflow:hidden}
        .topbar{background:var(--bg2);padding:14px 24px;display:flex;justify-content:space-between;align-items:center;border-bottom:1px solid var(--border);flex-shrink:0}
        .page-title{font-size:16px;font-weight:700;color:var(--text)}
        .topbar-right{display:flex;align-items:center;gap:12px}
        .logout-btn{padding:7px 16px;background:transparent;border:1px solid var(--border2);border-radius:6px;color:var(--text2);font-family:var(--font);font-size:12px;cursor:pointer;transition:all...
```

```
.logout-btn:hover{background:var(--red-dim);border-color:var(--red);color:var(--red)}
.content{flex:1;overflow-y:auto;padding:24px}
.view{display:none}
.view.active{display:block;animation:fadeIn 0.3s ease}
@keyframes fadeIn{from{opacity:0;transform:translateY(8px)}to{opacity:1;transform:translateY(0)}}

/* STATS */
.stats-grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(160px,1fr));gap:16px;margin-bottom:24px}
.stat-card{background:var(--panel);border:1px solid var(--border);border-radius:14px;padding:20px;position:relative;overflow:hidden;transition:all 0.2s;cursor:default}
.stat-card:hover{border-color:var(--border2);transform:translateY(-1px)}
.stat-card::before{content:'';position:absolute;top:0;left:0;right:0;height:3px}
.stat-card.purple::before{background:linear-gradient(90deg,#7c3aed,var(--accent))}
.stat-card.green::before{background:var(--green)}
.stat-card.amber::before{background:var(--amber)}
.stat-card.blue::before{background:var(--blue)}
.stat-label{font-size:11px;color:var(--text3);text-transform:uppercase;letter-spacing:0.8px;margin-bottom:10px;font-weight:600}
.stat-value{font-size:30px;font-weight:700;color:var(--text);font-family:var(--mono);line-height:1}
.stat-sub{font-size:11px;color:var(--text3);margin-top:6px}

/* SECTION */
.section{background:var(--panel);border:1px solid var(--border);border-radius:14px;margin-bottom:20px;overflow:hidden}
.section-head{padding:16px 20px;border-bottom:1px solid var(--border);display:flex;justify-content:space-between;align-items:center}
.section-title{font-size:14px;font-weight:700;color:var(--text)}
.section-sub{font-size:12px;color:var(--text3);margin-top:2px}

/* TABLE */
.tablewrap{overflow-x:auto}
table{width:100%;border-collapse:collapse}
th{padding:10px 16px;text-align:left;font-size:10px;color:var(--text3);font-weight:700;letter-spacing:1px;text-transform:uppercase;background:var(--bg3);border-bottom:1px solid var(--border)}
td{padding:12px 16px;border-bottom:1px solid var(--border);font-size:13px;color:var(--text2)}
tr:last-child td{border-bottom:none}
tr:hover td{background:rgba(167,139,250,0.03);color:var(--text)}

/* CHIPS */
.chip{display:inline-flex;align-items:center;padding:3px 10px;border-radius:20px;font-size:11px;font-weight:600}
.chip.pending{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(251,191,36,0.2)}
.chip.approved{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.2)}
.chip.rejected{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
.chip.active{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.2)}
.chip.employee{background:var(--blue-dim);color:var(--blue)}
.chip.manager{background:var(--accent-dim);color:var(--accent)}

/* BUTTONS */
.btn{padding:8px 16px;border-radius:8px;border:none;font-family:var(--font);font-size:12px;font-weight:600;cursor:pointer;transition:all 0.2s}
.btn-primary{background:var(--accent);color:#fff}
.btn-primary:hover{background:#7c3aed;box-shadow:0 4px 12px rgba(167,139,250,0.3)}
.btn-approve{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.2)}
.btn-approve:hover{background:var(--green);color:#fff}
.btn-reject{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
.btn-reject:hover{background:var(--red);color:#fff}
.btn-secondary{background:var(--panel2);color:var(--text2);border:1px solid var(--border)}
.btn-secondary:hover{border-color:var(--border2);color:var(--text)}
.btn-sm{padding:5px 12px;font-size:11px}
.action-row{display:flex;gap:6px}

/* FORM */
.form-card{padding:24px}
.form-grid{display:grid;grid-template-columns:1fr 1fr;gap:16px}
.form-group{display:flex;flex-direction:column;gap:6px}
.form-group.full{grid-column:1/-1}
label{font-size:10px;font-weight:700;color:var(--text3);text-transform:uppercase;letter-spacing:0.8px}
input,select,textarea{background:var(--bg3);border:1px solid var(--border);border-radius:8px;padding:10px 14px;color:var(--text);font-family:var(--font);font-size:13px;transition:border-col...
input:focus,select:focus,textarea:focus{outline:none;border-color:var(--accent)}
input::placeholder{color:var(--text3)}
select option{background:var(--bg2)}
.form-actions{display:flex;gap:10px;margin-top:20px}
```

```
        /* ALERT */
        .alert{position:fixed;top:20px;right:20px;padding:12px 20px;border-radius:10px;z-index:1100;animation:slideIn 0.3s ease;font-size:13px;font-weight:500}
        .alert-success{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.3)}
        .alert-error{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.3)}
        @keyframes slideIn{from{transform:translateX(120%);opacity:0}to{transform:translateX(0);opacity:1}}
        .empty-state{text-align:center;color:var(--text3);font-style:italic;padding:30px}

        /* GRID DASHBOARD */
        .dash-grid{display:grid;grid-template-columns:1fr 1fr;gap:20px;margin-bottom:0}

        /* SEARCH */
        .search-bar{display:flex;align-items:center;gap:10px;margin-bottom:16px}
        .search-input{flex:1;background:var(--bg3);border:1px solid var(--border);border-radius:8px;padding:9px 14px;color:var(--text);font-family:var(--font);font-size:13px}
        .search-input:focus{outline:none;border-color:var(--accent)}
        .search-input::placeholder{color:var(--text3)}
    </style>
</head>
<body>
<div class="app">
    <!-- SIDEBAR -->
    <div class="sidebar">
        <div class="logo">
            <div class="logo-row">
                <div class="logo-icon">M</div>
                <h2>EMS Console</h2>
            </div>
            <p>Management Portal</p>
            <div class="role-badge"><span class="role-dot"></span>Manager</div>
        </div>
        <nav class="nav">
            <div class="nav-section">Overview</div>
            <div class="nav-item active" onclick="showView('dashboard',this)">
                <span></span> Dashboard
            </div>
            <div class="nav-section">Team</div>
            <div class="nav-item" onclick="showView('employees',this)"><span></span> All Employees</div>
            <div class="nav-item" onclick="showView('add-employee',this)"><span></span> Add Employee</div>
            <div class="nav-section">Leave Management</div>
            <div class="nav-item" onclick="showView('pending-leaves',this)">
                <span></span> Pending Approvals
                <span class="nav-badge" id="pending-badge">0</span>
            </div>
            <div class="nav-item" onclick="showView('all-leaves',this)"><span></span> All Leave Requests</div>
            <div class="nav-section">Meals & Vendors</div>
            <div class="nav-item" onclick="showView('restaurants',this)"><span></span> Restaurants</div>
            <div class="nav-item" onclick="showView('vendors',this)"><span></span> Vendors</div>
            <div class="nav-section">Personal</div>
            <div class="nav-item" onclick="showView('my-leaves',this)"><span></span> My Leaves</div>
        </nav>
        <div class="sidebar-footer">
            <div class="user-card">
                <div class="avatar" id="avatar-initials">M</div>
                <div class="user-info-side">
                    <div class="name" id="sidebar-name">Manager</div>
                    <div class="role-text">Manager</div>
                </div>
            </div>
        </div>
    </div>

    <!-- MAIN -->
    <div class="main">
        <div class="topbar">
            <div class="page-title" id="pageTitle">Dashboard</div>
            <div class="topbar-right">
                <span style="font-size:12px;color:var(--text3);font-family:var(--mono)" id="topbar-email"></span>
                <button class="logout-btn" onclick="logout()">Sign Out</button>
            </div>
        </div>
        <div class="content">
```

```
            <!-- DASHBOARD -->
            <div id="dashboard-view" class="view active">
                <div class="stats-grid">
                    <div class="stat-card purple">
                        <div class="stat-label">Total Employees</div>
                        <div class="stat-value" id="stat-total">--</div>
                        <div class="stat-sub">in your organization</div>
                    </div>
                    <div class="stat-card green">
                        <div class="stat-label">Active Now</div>
                        <div class="stat-value" id="stat-active">--</div>
                        <div class="stat-sub">currently working</div>
                    </div>
                    <div class="stat-card amber">
                        <div class="stat-label">Pending Approvals</div>
                        <div class="stat-value" id="stat-pending">--</div>
                        <div class="stat-sub">need your action</div>
                    </div>
                    <div class="stat-card blue">
                        <div class="stat-label">On Leave Today</div>
                        <div class="stat-value" id="stat-on-leave">--</div>
                        <div class="stat-sub">employees absent</div>
                    </div>
                </div>
                <div class="dash-grid">
                    <div class="section">
                        <div class="section-head">
                            <div><div class="section-title"> Needs Attention</div><div class="section-sub">Pending leave requests</div></div>
                            <button class="btn btn-secondary btn-sm" onclick="showView('pending-leaves',null)">View All </button>
                        </div>
                        <div class="tablewrap">
                            <table>
<thead><tr><th>Employee</th><th>Type</th><th>Dates</th><th>Action</th></tr></thead>
                                <tbody id="dash-pending"><tr><td colspan="4" class="empty-state">Loading...</td></tr></tbody>
                            </table>
                        </div>
                    </div>
                    <div class="section">
                        <div class="section-head">
                            <div><div class="section-title"> Team Overview</div><div class="section-sub">Recent employees</div></div>
                            <button class="btn btn-secondary btn-sm" onclick="showView('employees',null)">View All </button>
                        </div>
                        <div class="tablewrap">
                            <table>
                                <thead><tr><th>Name</th><th>Dept</th><th>Status</th></tr></thead>
                                <tbody id="dash-employees"><tr><td colspan="3" class="empty-state">Loading...</td></tr></tbody>
                            </table>
                        </div>
                    </div>
                </div>
            </div>

            <!-- ALL EMPLOYEES -->
            <div id="employees-view" class="view">
                <div class="section">
                    <div class="section-head">
                        <div><div class="section-title">All Employees</div><div class="section-sub">Manage your team</div></div>
                        <button class="btn btn-secondary btn-sm" onclick="loadEmployees()">Refresh</button>
                    </div>
                    <div style="padding:16px 20px;border-bottom:1px solid var(--border)">
                        <div class="search-bar">
                            <input class="search-input" id="emp-search" placeholder="Search by name, email or department..." oninput="filterEmployees()">
                        </div>
                    </div>
                    <div class="tablewrap">
```

```
                        <table>
<thead><tr><th>ID</th><th>Name</th><th>Email</th><th>Department</th><th>Role</th><th>Status</th><th>Actions</th></tr></thead>
                            <tbody id="employees-list"><tr><td colspan="7" class="empty-state">Loading...</td></tr></tbody>
                        </table>
                    </div>
                </div>
            </div>

            <!-- ADD EMPLOYEE -->
            <div id="add-employee-view" class="view">
                <div class="section">
                    <div class="section-head"><div class="section-title">Add New Employee</div></div>
                    <div class="form-card">
                        <div class="form-grid">
                            <div class="form-group"><label>Full Name</label><input id="emp-name" placeholder="John Doe"></div>
                            <div class="form-group"><label>Email</label><input type="email" id="emp-email" placeholder="john@company.com"></div>
                            <div class="form-group"><label>Password</label><input type="password" id="emp-pass" placeholder="Temporary password"></div>
                            <div class="form-group"><label>Department</label><input id="emp-dept" placeholder="Engineering"></div>
                            <div class="form-group"><label>Role</label>
                                <select id="emp-role"><option value="EMPLOYEE">Employee</option><option value="MANAGER">Manager</option></select>
                            </div>
                            <div class="form-group"><label>Manager ID (optional)</label><input type="number" id="emp-manager" placeholder="Manager employee ID"></div>
                        </div>
                        <div class="form-actions">
                            <button class="btn btn-primary" onclick="addEmployee()">Add Employee</button>
                            <button class="btn btn-secondary" onclick="clearEmpForm()">Clear</button>
                        </div>
                    </div>
                </div>
            </div>

            <!-- PENDING LEAVE APPROVALS -->
            <div id="pending-leaves-view" class="view">
                <div class="section">
                    <div class="section-head">
                        <div><div class="section-title"> Pending Leave Approvals</div><div class="section-sub">Review and take action</div></div>
                        <button class="btn btn-secondary btn-sm" onclick="loadPendingLeaves()">Refresh</button>
                    </div>
                    <div class="tablewrap">
                        <table>
<thead><tr><th>Employee</th><th>Type</th><th>From</th><th>To</th><th>Days</th><th>Reason</th><th>Actions</th></tr></thead>
                            <tbody id="pending-leaves-list"><tr><td colspan="7" class="empty-state">Loading...</td></tr></tbody>
                        </table>
                    </div>
                </div>
            </div>

            <!-- ALL LEAVES -->
            <div id="all-leaves-view" class="view">
                <div class="section">
                    <div class="section-head">
                        <div><div class="section-title">All Leave Requests</div></div>
                        <button class="btn btn-secondary btn-sm" onclick="loadAllLeaves()">Refresh</button>
                    </div>
                    <div class="tablewrap">
                        <table>
<thead><tr><th>Employee</th><th>Type</th><th>From</th><th>To</th><th>Days</th><th>Status</th><th>Action</th></tr></thead>
                            <tbody id="all-leaves-list"><tr><td colspan="7" class="empty-state">Loading...</td></tr></tbody>
```

```
                        </table>
                    </div>
                </div>
            </div>

            <!-- RESTAURANTS -->
            <div id="restaurants-view" class="view">
                <div class="section">
                    <div class="section-head">
                        <div class="section-title">Restaurants</div>
                        <button class="btn btn-secondary btn-sm" onclick="loadRestaurants()">
Refresh</button>
                    </div>
                    <div class="tablewrap">
                        <table>
                            <thead><tr><th>Name</th><th>Meal
Slot</th><th>Vendor</th><th>Status</th></tr></thead>
                            <tbody id="restaurants-list"><tr><td colspan="4" class="empty-
state">Loading...</td></tr></tbody>
                        </table>
                    </div>
                </div>
            </div>

            <!-- VENDORS -->
            <div id="vendors-view" class="view">
                <div class="section">
                    <div class="section-head">
                        <div class="section-title">Vendors</div>
                        <button class="btn btn-secondary btn-sm" onclick="loadVendors()"> Refresh</button>
                    </div>
                    <div class="tablewrap">
                        <table>
                            <thead><tr><th>Name</th><th>Email</th><th>Restaurants</th></tr></thead>
                            <tbody id="vendors-list"><tr><td colspan="3" class="empty-
state">Loading...</td></tr></tbody>
                        </table>
                    </div>
                </div>
            </div>

            <!-- MY LEAVES -->
            <div id="my-leaves-view" class="view">
                <div class="section">
                    <div class="section-head">
                        <div class="section-title">My Leave Requests</div>
                        <button class="btn btn-secondary btn-sm" onclick="loadMyLeaves()"> Refresh</button>
                    </div>
                    <div class="tablewrap">
                        <table>
<thead><tr><th>Type</th><th>From</th><th>To</th><th>Days</th><th>Status</th></tr></thead>
                            <tbody id="my-leaves-list"><tr><td colspan="5" class="empty-
state">Loading...</td></tr></tbody>
                        </table>
                    </div>
                </div>
            </div>

        </div>
    </div>
</div>

<script>
    const API='http://localhost:8080';
    let allEmployeesCache=[];

    (function(){
        const token=localStorage.getItem('jwt_token');
        const authType=localStorage.getItem('auth_type');
        if(!token&&authType!=='session'){window.location.href='/login.html';return;}
        if(token){
            try{
                const p=JSON.parse(atob(token.split('.')[1]));
                const roles=p.roles||p.authorities||'';
                if(roles.includes('ROLE_MANAGER')) { window.location.href = '/manager-dashboard.html';
```

```
return; }
                if(roles.includes('ROLE_EMPLOYEE')){window.location.href='/employee-dashboard.html';return;}
                if(roles.includes('ROLE_VENDOR')){window.location.href='/vendor-dashboard.html';return;}
            }catch(e){}
        }
        initUser();
    })();

    function getAuthHeader(){
        const t=localStorage.getItem('jwt_token');
        return t?{'Authorization':`Bearer ${t}`}:{};
    }
    async function apiFetch(path,opts={}){
        const res=await fetch(`${API}${path}`,{...opts,headers:{'Content-Type':'application/json',...getAuthHeader(),...(opts.headers||{})},credentials:'include'});
if(res.status===401){localStorage.removeItem('jwt_token');localStorage.removeItem('auth_type');showAlert('Session expired','error');setTimeout(()=>window.location.href='/login.html',1500);t...
        const text=await res.text();return text?JSON.parse(text):null;
    }

    async function initUser(){
        const token=localStorage.getItem('jwt_token');
        let email='';
        if(token){try{const p=JSON.parse(atob(token.split('.')[1]));email=p.sub||p.email||'';}catch(e){}}
        document.getElementById('avatar-initials').textContent=(email||'M').charAt(0).toUpperCase();
        document.getElementById('sidebar-name').textContent=email||'Manager';
        document.getElementById('topbar-email').textContent=email;
        loadDashboard();
    }

    function showView(name,el){
        document.querySelectorAll('.view').forEach(v=>v.classList.remove('active'));
        document.getElementById(`${name}-view`).classList.add('active');
        document.querySelectorAll('.nav-item').forEach(n=>n.classList.remove('active'));
        if(el)el.classList.add('active');
        document.getElementById('pageTitle').textContent=name.split('-').map(w=>w.charAt(0).toUpperCase()+w.slice(1)).join(' ');
        const loaders={dashboard:loadDashboard,employees:loadEmployees,'pending-leaves':loadPendingLeaves,'all-leaves':loadAllLeaves,restaurants:loadRestaurants,vendors:loadVendors,'my-leaves':load...
        if(loaders[name])loaders[name]();
    }

    async function loadDashboard(){
        try{
            const [emps,leaves]=await Promise.all([apiFetch('/employee'),apiFetch('/leave_requests')]);
            document.getElementById('stat-total').textContent=emps?.length||0;
            document.getElementById('stat-active').textContent=emps?.filter(e=>e.status==='ACTIVE').length||0;
            document.getElementById('stat-on-leave').textContent=emps?.filter(e=>e.status==='ON_LEAVE').length||0;
            const pending=(leaves||[]).filter(l=>l.status==='PENDING');
            document.getElementById('stat-pending').textContent=pending.length;
            document.getElementById('pending-badge').textContent=pending.length;
            document.getElementById('pending-badge').classList.toggle('hidden',pending.length===0);
            // Dash pending
            document.getElementById('dash-pending').innerHTML=pending.length
                ? pending.slice(0,5).map(l=>`<tr><td>${l.employeeName||l.employeeId||'—'}</td><td><span class="chip pending">${l.leaveType}</span></td><td>${l.startDate}  ${l.endDate}</td><td><div...
                : '<tr><td colspan="4" class="empty-state">No pending requests </td></tr>';
            // Dash employees
            document.getElementById('dash-employees').innerHTML=emps?.length
                ? emps.slice(0,6).map(e=>`<tr><td>${e.name||'—'}</td><td>${e.dept||'—'}</td><td><span class="chip ${(e.status||'').toLowerCase()}">${e.status||'—'}</span></td></tr>`).join('')
                : '<tr><td colspan="3" class="empty-state">No employees</td></tr>';
        }catch(e){console.error(e);}
    }

    async function loadEmployees(){
        try{
            const emps=await apiFetch('/employee');
            allEmployeesCache=emps||[];
            renderEmployees(allEmployeesCache);
        }catch(e){document.getElementById('employees-list').innerHTML='<tr><td colspan="7" class="empty-
```

```
state">Could not load employees</td></tr>';}
    }

    function renderEmployees(emps){
        document.getElementById('employees-list').innerHTML=emps.length
            ? emps.map(e=>`<tr><td style="font-family:var(--mono);font-size:12px">#${e.employeeId}</td><td>${e.name||'—'}</td><td style="font-family:var(--mono);font-size:12px">${e.email||'—'}</td>...
            : '<tr><td colspan="7" class="empty-state">No employees found</td></tr>';
    }

    function filterEmployees(){
        const q=document.getElementById('emp-search').value.toLowerCase();
renderEmployees(allEmployeesCache.filter(e=>(e.name||'').toLowerCase().includes(q)||(e.email||'').toLowerCase().includes(q)||(e.dept||'').toLowerCase().includes(q)));
    }

    async function addEmployee(){
        const name=document.getElementById('emp-name').value.trim();
        const email=document.getElementById('emp-email').value.trim();
        const password=document.getElementById('emp-pass').value;
        const dept=document.getElementById('emp-dept').value.trim();
        const role=document.getElementById('emp-role').value;
        const managerId=document.getElementById('emp-manager').value;
        if(!name||!email||!password||!dept){showAlert('Please fill all required fields','error');return;}
        try{
            await apiFetch('/employee',{method:'POST',body:JSON.stringify({name,email,password,dept,role,managerId:managerId?parseInt(managerId):null})});
            showAlert('Employee added successfully!','success');clearEmpForm();
        }catch(e){showAlert('Failed: '+e.message,'error');}
    }

    function clearEmpForm(){['emp-name','emp-email','emp-pass','emp-dept','emp-manager'].forEach(id=>document.getElementById(id).value='');}

    async function deleteEmployee(id){
        if(!confirm('Remove this employee?'))return;
        try{await apiFetch(`/employee/${id}`,{method:'DELETE'});showAlert('Employee removed','success');loadEmployees();}
        catch(e){showAlert('Failed to remove employee','error');}
    }

    async function loadPendingLeaves(){
        try{
            const leaves = await apiFetch('/leave_requests/pending');
            const pending=(leaves||[]).filter(l=>l.status===='PENDING');
            document.getElementById('pending-leaves-list').innerHTML=pending.length
                ? pending.map(l=>`<tr><td>${l.employeeName||l.employeeId||'—'}</td><td><span class="chip pending">${l.leaveType}</span></td><td>${l.startDate}</td><td>${l.endDate}</td><td>${l.numbe...
                : '<tr><td colspan="7" class="empty-state">No pending approvals </td></tr>';
        }catch(e){document.getElementById('pending-leaves-list').innerHTML='<tr><td colspan="7" class="empty-state">Could not load</td></tr>';}
    }

    async function approveLeave(id) {
        try {
            const res = await fetch(`${API}/leave_requests/approval`, {
                method: 'PUT',
                headers: { 'Content-Type': 'application/json', ...getAuthHeader() },
                credentials: 'include',
                body: JSON.stringify({
                    leaveRequestId: id,
                    action: 'approved',
                    remarks: ''
                })
            });

            const text = await res.text();
            console.log('Status:', res.status, 'Body:', text);

            if (res.ok) {
                showAlert('Approved ', 'success');
                loadPendingLeaves();
                loadDashboard();
```

```
                loadAllLeaves();
            } else {
                showAlert('Error: ' + text, 'error');
            }
        } catch (e) {
            showAlert(e.message, 'error');
        }
    }


    async function rejectLeave(id) {
        try {
            const res = await fetch(`${API}/leave_requests/approval`, {
                method: 'PUT',
                headers: { 'Content-Type': 'application/json', ...getAuthHeader() },
                credentials: 'include',
                body: JSON.stringify({
                    leaveRequestId: id,
                    action: 'rejected',
                    remarks: ''
                })
            });

            const text = await res.text();
            console.log('Status:', res.status, 'Body:', text);

            if (res.ok) {
                showAlert('Leave rejected', 'success');
                loadPendingLeaves();
                loadDashboard();
                loadAllLeaves();
            } else {
                showAlert('Error: ' + text, 'error');
            }
        } catch (e) {
            showAlert('Failed to reject: ' + e.message, 'error');
        }
    }

    async function loadAllLeaves(){
        try{
            const leaves=await apiFetch('/leave_requests');
            document.getElementById('all-leaves-list').innerHTML=(leaves||[]).length
                ? (leaves).map(l=>`<tr><td>${l.employeeName||l.employeeId||'—'}</td><td><span class="chip
${(l.leaveType||'').toLowerCase()}">${l.leaveType}</span></td><td>${l.startDate}</td><td>${...
                : '<tr><td colspan="7" class="empty-state">No leave requests</td></tr>';
        }catch(e){document.getElementById('all-leaves-list').innerHTML='<tr><td colspan="7" class="empty-
state">Could not load</td></tr>';}
    }

    async function loadRestaurants(){
        try{
            const data=await apiFetch('/restaurants');
            document.getElementById('restaurants-list').innerHTML=(data||[]).length
                ?
data.map(r=>`<tr><td>${r.name||'—'}</td><td>${r.mealSlot||'—'}</td><td>${r.vendorName||r.vendorId||'—'}</td><td>
<span class="chip ${r.active?'active':'rejected'}">${r.active?'Acti...
                : '<tr><td colspan="4" class="empty-state">No restaurants</td></tr>';
        }catch(e){document.getElementById('restaurants-list').innerHTML='<tr><td colspan="4" class="empty-
state">Could not load</td></tr>';}
    }

    async function loadVendors(){
        try{
            const data=await apiFetch('/vendors');
            document.getElementById('vendors-list').innerHTML=(data||[]).length
                ? data.map(v=>`<tr><td>${v.name||'—'}</td><td style="font-family:var(--mono);font-
size:12px">${v.email||'—'}</td><td>${v.restaurantCount||'—'}</td></tr>`).join('')
                : '<tr><td colspan="3" class="empty-state">No vendors</td></tr>';
        }catch(e){document.getElementById('vendors-list').innerHTML='<tr><td colspan="3" class="empty-
state">Could not load</td></tr>';}
    }

    async function loadMyLeaves(){
        try{
            const data=await apiFetch('/leave_requests/employee');
```

```
            const leaves=data?.requests||data||[];
            document.getElementById('my-leaves-list').innerHTML=leaves.length
                ? leaves.map(l=>`<tr><td><span class="chip ${(l.leaveType||'').toLowerCase()}">${l.leaveType}</span></td><td>${l.startDate}</td><td>${l.endDate}</td><td>${l.numberOfDays||'—'}</td><...
                : '<tr><td colspan="5" class="empty-state">No leave requests</td></tr>';
        }catch(e){document.getElementById('my-leaves-list').innerHTML='<tr><td colspan="5" class="empty-state">Could not load</td></tr>';}
    }

    function logout(){
        localStorage.removeItem('jwt_token');localStorage.removeItem('auth_type');
fetch(`${API}/session/logout`,{method:'POST',credentials:'include'}).finally(()=>window.location.href='/google.html');
    }

    function showAlert(msg,type){
        const el=document.createElement('div');
        el.className=`alert alert-${type}`;el.textContent=msg;
        document.body.appendChild(el);setTimeout(()=>el.remove(),3000);
    }
</script>
</body>
</html>
```

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>EMS — Vendor Hub</title>
    <link href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@300;400;500;600;700&family=Space+Mono:wght@400;700&display=swap" rel="stylesheet">
    <style>
        *{margin:0;padding:0;box-sizing:border-box}
        :root{
            --bg:#08100a;--bg2:#0c160e;--bg3:#101c12;
            --panel:#131f15;--panel2:#182219;
            --border:rgba(52,211,153,0.08);--border2:rgba(52,211,153,0.18);
            --text:#d4edd9;--text2:#6b9474;--text3:#3d5e44;
            --accent:#34d399;--accent-dim:rgba(52,211,153,0.1);
            --amber:#fbbf24;--amber-dim:rgba(251,191,36,0.1);
            --red:#f87171;--red-dim:rgba(248,113,113,0.1);
            --blue:#60a5fa;--blue-dim:rgba(96,165,250,0.1);
            --font:'DM Sans',sans-serif;--mono:'Space Mono',monospace;
            --sidebar:245px;
        }
        body{font-family:var(--font);background:var(--bg);color:var(--text);font-size:14px;overflow:hidden;height:100vh}
        .app{display:flex;height:100vh}

        /* SIDEBAR */
        .sidebar{width:var(--sidebar);background:var(--bg2);border-right:1px solid var(--border);display:flex;flex-direction:column;flex-shrink:0;position:relative}
        .sidebar::before{content:'';position:absolute;left:0;top:0;bottom:0;width:2px;background:linear-gradient(to bottom,transparent,var(--accent),transparent)}
        .logo{padding:22px 20px;border-bottom:1px solid var(--border)}
        .logo-row{display:flex;align-items:center;gap:10px;margin-bottom:4px}
        .logo-icon{width:34px;height:34px;background:linear-gradient(135deg,#16a34a,var(--accent));border-radius:10px;display:flex;align-items:center;justify-content:center;font-weight:700;font-siz...
        .logo h2{font-size:16px;font-weight:700;color:var(--text)}
        .logo p{font-size:10px;color:var(--text3);font-family:var(--mono)}
        .role-badge{display:inline-flex;align-items:center;gap:6px;padding:4px 12px;background:var(--accent-dim);border:1px solid rgba(52,211,153,0.2);border-radius:4px;font-size:10px;font-weight:7...
        .nav{flex:1;padding:10px 0;overflow-y:auto}
        .nav-section{padding:12px 20px 6px;font-size:9px;font-weight:700;color:var(--text3);letter-spacing:2px;text-transform:uppercase;font-family:var(--mono)}
        .nav-item{display:flex;align-items:center;gap:10px;padding:10px 20px;margin:2px 8px;border-radius:6px;cursor:pointer;transition:all 0.2s;color:var(--text2);font-size:13px;font-weight:500}
        .nav-item:hover{background:var(--accent-dim);color:var(--accent)}
        .nav-item.active{background:var(--accent-dim);color:var(--accent);border-left:2px solid var(--accent);margin-left:6px}
        .sidebar-footer{padding:16px 20px;border-top:1px solid var(--border)}
        .user-card{display:flex;align-items:center;gap:10px}
        .avatar{width:34px;height:34px;background:linear-gradient(135deg,#16a34a,var(--accent));border-radius:6px;display:flex;align-items:center;justify-content:center;font-size:14px;font-weight:7...
        .user-info-side .name{font-size:12px;font-weight:600;color:var(--text)}
        .user-info-side .role-text{font-size:9px;color:var(--accent);font-family:var(--mono);text-transform:uppercase;letter-spacing:1px}

        /* MAIN */
        .main{flex:1;display:flex;flex-direction:column;overflow:hidden}
        .topbar{background:var(--bg2);padding:14px 24px;display:flex;justify-content:space-between;align-items:center;border-bottom:1px solid var(--border);flex-shrink:0}
        .page-title{font-size:16px;font-weight:700;color:var(--text);font-family:var(--mono)}
        .topbar-right{display:flex;align-items:center;gap:12px}
        .logout-btn{padding:7px 16px;background:transparent;border:1px solid var(--border2);border-radius:4px;color:var(--text2);font-family:var(--mono);font-size:11px;cursor:pointer;transition:all...
        .logout-btn:hover{background:var(--red-dim);border-color:var(--red);color:var(--red)}
        .content{flex:1;overflow-y:auto;padding:24px}
        .view{display:none}
        .view.active{display:block;animation:fadeIn 0.25s ease}
        @keyframes fadeIn{from{opacity:0;transform:translateY(6px)}to{opacity:1;transform:translateY(0)}}

        /* STATS */
        .stats-grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(150px,1fr));gap:14px;margin-
```

```
bottom:24px}
        .stat-card{background:var(--panel);border:1px solid var(--border);border-radius:10px;padding:18px;position:relative;transition:border-color 0.2s}
        .stat-card:hover{border-color:var(--border2)}
        .stat-card::before{content:'';position:absolute;top:0;left:0;right:0;height:2px;border-radius:10px 10px 0 0}
        .stat-card.green::before{background:var(--accent)}
        .stat-card.amber::before{background:var(--amber)}
        .stat-card.blue::before{background:var(--blue)}
        .stat-card.red::before{background:var(--red)}
        .stat-label{font-size:10px;color:var(--text3);text-transform:uppercase;letter-spacing:1.2px;margin-bottom:10px;font-weight:600;font-family:var(--mono)}
        .stat-value{font-size:28px;font-weight:700;color:var(--text);font-family:var(--mono)}
        .stat-sub{font-size:11px;color:var(--text3);margin-top:4px}

        /* SECTION */
        .section{background:var(--panel);border:1px solid var(--border);border-radius:10px;margin-bottom:18px;overflow:hidden}
        .section-head{padding:14px 20px;border-bottom:1px solid var(--border);display:flex;justify-content:space-between;align-items:center}
        .section-title{font-size:13px;font-weight:700;color:var(--text);font-family:var(--mono)}
        .section-sub{font-size:11px;color:var(--text3);margin-top:2px}

        /* TABLE */
        .tablewrap{overflow-x:auto}
        table{width:100%;border-collapse:collapse}
        th{padding:10px 16px;text-align:left;font-size:10px;color:var(--text3);font-weight:700;letter-spacing:1px;text-transform:uppercase;background:var(--bg3);border-bottom:1px solid var(--border...
        td{padding:11px 16px;border-bottom:1px solid var(--border);font-size:13px;color:var(--text2)}
        tr:last-child td{border-bottom:none}
        tr:hover td{background:rgba(52,211,153,0.03);color:var(--text)}

        /* CHIPS */
        .chip{display:inline-flex;align-items:center;padding:3px 8px;border-radius:4px;font-size:10px;font-weight:700;font-family:var(--mono);letter-spacing:0.5px}
        .chip.active{background:var(--accent-dim);color:var(--accent);border:1px solid rgba(52,211,153,0.2)}
        .chip.inactive{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
        .chip.pending{background:var(--amber-dim);color:var(--amber)}
        .chip.delivered{background:var(--accent-dim);color:var(--accent)}
        .chip.processing{background:var(--blue-dim);color:var(--blue)}
        .chip.breakfast{background:var(--amber-dim);color:var(--amber)}
        .chip.lunch{background:var(--accent-dim);color:var(--accent)}
        .chip.dinner{background:var(--blue-dim);color:var(--blue)}

        /* BUTTONS */
        .btn{padding:8px 16px;border-radius:6px;border:none;font-family:var(--mono);font-size:11px;font-weight:700;cursor:pointer;transition:all 0.2s;letter-spacing:0.5px;text-transform:uppercase}
        .btn-primary{background:var(--accent);color:#08100a}
        .btn-primary:hover{background:#22c77a;box-shadow:0 4px 12px rgba(52,211,153,0.3)}
        .btn-secondary{background:var(--panel2);color:var(--text2);border:1px solid var(--border)}
         .btn-secondary:hover{border-color:var(--border2);color:var(--text)}
         .btn-danger{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
         .btn-danger:hover{background:var(--red);color:#fff}
         .btn-warning{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(251,191,36,0.2)}
         .btn-warning:hover{background:var(--amber);color:#000}
         .btn-sm{padding:5px 10px;font-size:10px}
         .action-row{display:flex;gap:6px}

         /* FORM */
         .form-card{padding:22px}
         .form-grid{display:grid;grid-template-columns:1fr 1fr;gap:14px}
         .form-group{display:flex;flex-direction:column;gap:5px}
         .form-group.full{grid-column:1/-1}
         label{font-size:10px;font-weight:700;color:var(--text3);text-transform:uppercase;letter-spacing:0.8px;font-family:var(--mono)}
         input,select,textarea{background:var(--bg3);border:1px solid var(--border);border-radius:6px;padding:9px 12px;color:var(--text);font-family:var(--font);font-size:13px;transition:border-colo...
         input:focus,select:focus,textarea:focus{outline:none;border-color:var(--accent)}
         input::placeholder,textarea::placeholder{color:var(--text3)}
         select option{background:var(--bg2)}
         .form-actions{display:flex;gap:10px;margin-top:18px}

         /* ALERT */
         .alert{position:fixed;top:20px;right:20px;padding:12px 20px;border-radius:8px;z-index:1100;animation:slideIn 0.3s ease;font-size:12px;font-weight:700;font-family:var(--mono)}
         .alert-success{background:var(--accent-dim);color:var(--accent);border:1px solid
```

```
rgba(52,211,153,0.3)}
        .alert-error{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.3)}
        @keyframes slideIn{from{transform:translateX(120%);opacity:0}to{transform:translateX(0);opacity:1}}
        .empty-state{text-align:center;color:var(--text3);font-style:italic;padding:28px;font-family:var(--mono);font-size:12px}

        /* RESTAURANT CARDS */
        .restaurant-grid{display:grid;grid-template-columns:repeat(auto-fill,minmax(240px,1fr));gap:14px;padding:20px}
        .restaurant-card{background:var(--bg3);border:1px solid var(--border);border-radius:10px;padding:18px;transition:all 0.2s;position:relative;overflow:hidden}
        .restaurant-card::before{content:'';position:absolute;top:0;left:0;bottom:0;width:3px;background:var(--accent)}
        .restaurant-card:hover{border-color:var(--border2);transform:translateX(2px)}
        .restaurant-card.inactive::before{background:var(--red)}
        .rest-name{font-weight:700;color:var(--text);margin-bottom:6px;font-size:14px}
        .rest-meta{font-size:11px;color:var(--text3);font-family:var(--mono);margin-bottom:12px}
        .rest-actions{display:flex;gap:8px}
    </style>
</head>
<body>
<div class="app">
    <!-- SIDEBAR -->
    <div class="sidebar">
        <div class="logo">
            <div class="logo-row">
                <div class="logo-icon">V</div>
                <h2>Vendor Hub</h2>
            </div>
            <p>EMS Vendor Portal</p>
            <div class="role-badge">// VENDOR</div>
        </div>
        <nav class="nav">
            <div class="nav-section">// overview</div>
            <div class="nav-item active" onclick="showView('dashboard',this)"><span></span> Dashboard</div>
            <div class="nav-section">// restaurants</div>
            <div class="nav-item" onclick="showView('my-restaurants',this)"><span></span> My Restaurants</div>
            <div class="nav-item" onclick="showView('add-restaurant',this)"><span></span> Add Restaurant</div>
            <div class="nav-section">// operations</div>
            <div class="nav-item" onclick="showView('subscriptions',this)"><span></span> Subscriptions</div>
            <div class="nav-item" onclick="showView('deliveries',this)"><span></span> Deliveries</div>
            <div class="nav-section">// account</div>
            <div class="nav-item" onclick="showView('profile',this)"><span></span> My Profile</div>
        </nav>
        <div class="sidebar-footer">
            <div class="user-card">
                <div class="avatar" id="avatar-initials">V</div>
                <div class="user-info-side">
                    <div class="name" id="sidebar-name">Vendor</div>
                    <div class="role-text">// vendor</div>
                </div>
            </div>
        </div>
    </div>

    <!-- MAIN -->
    <div class="main">
        <div class="topbar">
            <div class="page-title" id="pageTitle">// dashboard</div>
            <div class="topbar-right">
                <span style="font-size:11px;color:var(--text3);font-family:var(--mono)" id="topbar-email"></span>
                <button class="logout-btn" onclick="logout()">SIGN OUT</button>
            </div>
        </div>
        <div class="content">

            <!-- DASHBOARD -->
            <div id="dashboard-view" class="view active">
                <div class="stats-grid">
                    <div class="stat-card green">
                        <div class="stat-label">Restaurants</div>
                        <div class="stat-value" id="stat-restaurants">--</div>
```

```
                        <div class="stat-sub">active listings</div>
                    </div>
                    <div class="stat-card amber">
                        <div class="stat-label">Subscriptions</div>
                        <div class="stat-value" id="stat-subs">--</div>
                        <div class="stat-sub">active customers</div>
                    </div>
                    <div class="stat-card blue">
                        <div class="stat-label">Deliveries</div>
                        <div class="stat-value" id="stat-deliveries">--</div>
                        <div class="stat-sub">total today</div>
                    </div>
                    <div class="stat-card red">
                        <div class="stat-label">Pending</div>
                        <div class="stat-value" id="stat-pending">--</div>
                        <div class="stat-sub">deliveries pending</div>
                    </div>
                </div>
                <div style="display:grid;grid-template-columns:1fr 1fr;gap:18px">
                    <div class="section">
                        <div class="section-head">
                            <div><div class="section-title">// my_restaurants</div></div>
                            <button class="btn btn-secondary btn-sm" onclick="showView('my-restaurants',null)">view </button>
                        </div>
                        <div id="dash-restaurants" style="padding:4px 0"></div>
                    </div>
                    <div class="section">
                        <div class="section-head">
                            <div><div class="section-title">// recent_deliveries</div></div>
                            <button class="btn btn-secondary btn-sm" onclick="showView('deliveries',null)">view </button>
                        </div>
                        <div class="tablewrap">
                            <table>
                                <thead><tr><th>Meal</th><th>Slot</th><th>Status</th></tr></thead>
                                <tbody id="dash-deliveries"><tr><td colspan="3" class="empty-state">loading...</td></tr></tbody>
                            </table>
                        </div>
                    </div>
                </div>
            </div>

            <!-- MY RESTAURANTS -->
            <div id="my-restaurants-view" class="view">
                <div class="section">
                    <div class="section-head">
                        <div><div class="section-title">// my_restaurants</div><div class="section-sub">Manage your listings</div></div>
                        <button class="btn btn-secondary btn-sm" onclick="loadMyRestaurants()">refresh</button>
                    </div>
                    <div class="restaurant-grid" id="restaurants-grid">
                        <div class="empty-state">loading...</div>
                    </div>
                </div>
            </div>

            <!-- ADD RESTAURANT -->
            <div id="add-restaurant-view" class="view">
                <div class="section">
                    <div class="section-head"><div class="section-title">// add_restaurant</div></div>
                    <div class="form-card">
                        <div class="form-grid">
                            <div class="form-group full"><label>Restaurant Name</label><input id="rest-name" placeholder="My Restaurant Name"></div>
                            <div class="form-group">
                                <label>Meal Slot</label>
                                <select id="rest-slot">
                                    <option value="BREAKFAST">Breakfast</option>
                                    <option value="LUNCH">Lunch</option>
                                    <option value="DINNER">Dinner</option>
                                </select>
                            </div>
                            <div class="form-group"><label>Vendor ID</label><input type="number" id="rest-
```

```
vendor-id" placeholder="Your vendor ID"></div>
                                <div class="form-group full"><label>Description (optional)</label><textarea
id="rest-desc" rows="2" placeholder="Brief description of your restaurant..."></textarea></div>
                            </div>
                            <div class="form-actions">
                                <button class="btn btn-primary" onclick="addRestaurant()">CREATE
RESTAURANT</button>
                                <button class="btn btn-secondary" onclick="clearRestForm()">CLEAR</button>
                            </div>
                        </div>
                    </div>
                </div>

                <!-- SUBSCRIPTIONS -->
                <div id="subscriptions-view" class="view">
                    <div class="section">
                        <div class="section-head">
                            <div><div class="section-title">// subscriptions</div><div class="section-
sub">Customers subscribed to your restaurants</div></div>
                            <button class="btn btn-secondary btn-sm" onclick="loadSubscriptions()">
refresh</button>
                        </div>
                        <div class="tablewrap">
                            <table>
                                <thead><tr><th>Employee</th><th>Restaurant</th><th>Meal
Slot</th><th>From</th><th>To</th><th>Status</th></tr></thead>
                                <tbody id="subscriptions-list"><tr><td colspan="6" class="empty-
state">loading...</td></tr></tbody>
                            </table>
                        </div>
                    </div>
                </div>

                <!-- DELIVERIES -->
                <div id="deliveries-view" class="view">
                    <div class="section">
                        <div class="section-head">
                            <div><div class="section-title">// deliveries</div></div>
                            <button class="btn btn-secondary btn-sm" onclick="loadDeliveries()">
refresh</button>
                        </div>
                        <div class="tablewrap">
                            <table>
                                <thead><tr><th>ID</th><th>Restaurant</th><th>Meal
Slot</th><th>Date</th><th>Status</th><th>Action</th></tr></thead>
                                <tbody id="deliveries-list"><tr><td colspan="6" class="empty-
state">loading...</td></tr></tbody>
                            </table>
                        </div>
                    </div>
                </div>

                <!-- PROFILE -->
                <div id="profile-view" class="view">
                    <div class="section">
                        <div class="section-head"><div class="section-title">// my_profile</div></div>
                        <div style="padding:24px;display:grid;grid-template-columns:1fr 1fr;gap:16px"
id="vendor-profile">
                            <div><div style="font-size:10px;color:var(--text3);font-family:var(--mono);text-
transform:uppercase;letter-spacing:0.8px;margin-bottom:6px">NAME</div><div style="font-family...
                            <div><div style="font-size:10px;color:var(--text3);font-family:var(--mono);text-
transform:uppercase;letter-spacing:0.8px;margin-bottom:6px">EMAIL</div><div style="font-famil...
                            <div><div style="font-size:10px;color:var(--text3);font-family:var(--mono);text-
transform:uppercase;letter-spacing:0.8px;margin-bottom:6px">VENDOR ID</div><div style="font-f...
                            <div><div style="font-size:10px;color:var(--text3);font-family:var(--mono);text-
transform:uppercase;letter-spacing:0.8px;margin-bottom:6px">ROLE</div><div style="font-family...
                        </div>
                    </div>
                </div>

            </div>
        </div>
    </div>

    <script>
        const API='http://localhost:8080';
```

```
    let vendorId=null;

    (function(){
        const token=localStorage.getItem('jwt_token');
        const authType=localStorage.getItem('auth_type');
        if(!token&&authType!=='session'){window.location.href='/login.html';return;}
        if(token){
            try{
                const p=JSON.parse(atob(token.split('.')[1]));
                const roles=p.roles||p.authorities||'';
                if(roles.includes('ROLE_VENDOR')) { window.location.href = '/vendor-dashboard.html'; return; }
                if(roles.includes('ROLE_EMPLOYEE')){window.location.href='/employee-dashboard.html';return;}
                if(roles.includes('ROLE_MANAGER')){window.location.href='/manager-dashboard.html';return;}
            }catch(e){}
        }
        initUser();
    })();

    function getAuthHeader(){const t=localStorage.getItem('jwt_token');return t?{'Authorization':`Bearer ${t}`}:{};}
    async function apiFetch(path,opts={}){
        const res=await fetch(`${API}${path}`,{...opts,headers:{'Content-Type':'application/json',...getAuthHeader(),...(opts.headers||{})},credentials:'include'});

if(res.status===401){localStorage.removeItem('jwt_token');localStorage.removeItem('auth_type');showAlert('SESSION EXPIRED','error');setTimeout(()=>window.location.href='/login.html',1500);t...
        const text=await res.text();return text?JSON.parse(text):null;
    }

    async function initUser(){
        const token=localStorage.getItem('jwt_token');
        let email='';
        if(token){try{const p=JSON.parse(atob(token.split('.')[1]));email=p.sub||p.email||'';}catch(e){}}
        document.getElementById('avatar-initials').textContent=(email||'V').charAt(0).toUpperCase();
        document.getElementById('sidebar-name').textContent=email.split('@')[0]||'Vendor';
        document.getElementById('topbar-email').textContent=email;
        document.getElementById('p-email').textContent=email;
        // Try load vendor profile
        try{
            const vendors=await apiFetch('/vendors');
            const me=(vendors||[]).find(v=>v.email===email);
            if(me){vendorId=me.id||me.vendorId;document.getElementById('p-name').textContent=me.name||'—';document.getElementById('p-id').textContent=vendorId||'—';document.getElementById('rest-ven...
        }catch(e){}
        loadDashboard();
    }

    function showView(name,el){
        document.querySelectorAll('.view').forEach(v=>v.classList.remove('active'));
        document.getElementById(`${name}-view`).classList.add('active');
        document.querySelectorAll('.nav-item').forEach(n=>n.classList.remove('active'));
        if(el)el.classList.add('active');
        document.getElementById('pageTitle').textContent='// '+name.replace(/-/g,'_');
        const loaders={dashboard:loadDashboard,'my-restaurants':loadMyRestaurants,'add-restaurant':()=>{},'subscriptions':loadSubscriptions,'deliveries':loadDeliveries,'profile':()=>{}};
        if(loaders[name])loaders[name]();
    }

    async function loadDashboard(){
        try{
            const [rests,subs,deliveries]=await Promise.all([
                apiFetch(vendorId?`/restaurants/vendor/${vendorId}`:'/restaurants'),
                apiFetch('/subscriptions').catch(()=>[]),
                apiFetch('/deliveries').catch(()=>[])
            ]);
            const restArr=rests||[];
            const subArr=subs||[];
            const delArr=deliveries||[];
            document.getElementById('stat-restaurants').textContent=restArr.filter(r=>r.active!==false).length;
            document.getElementById('stat-subs').textContent=subArr.length;
            document.getElementById('stat-deliveries').textContent=delArr.length;
            document.getElementById('stat-
```

```
pending').textContent=delArr.filter(d=>d.status===='PENDING').length;
            // Dash restaurants
            document.getElementById('dash-restaurants').innerHTML=restArr.length
                ? restArr.slice(0,4).map(r=>`<div style="padding:12px 20px;border-bottom:1px solid var(--
border);display:flex;justify-content:space-between;align-items:center"><div><div style="font...
                : '<div class="empty-state">no restaurants yet</div>';
            // Dash deliveries
            document.getElementById('dash-deliveries').innerHTML=delArr.length
                ? delArr.slice(0,5).map(d=>`<tr><td>${d.restaurantName||d.restaurantId||'—'}</td><td><span
class="chip ${(d.mealSlot||'').toLowerCase()}">${d.mealSlot||'—'}</span></td><td><span cla...
                : '<tr><td colspan="3" class="empty-state">no deliveries</td></tr>';
        }catch(e){console.error(e);}
    }

    async function loadMyRestaurants(){
        try{
            const rests=await apiFetch(vendorId?`/restaurants/vendor/${vendorId}`:'/restaurants');
            const arr=rests||[];
            document.getElementById('restaurants-grid').innerHTML=arr.length
                ? arr.map(r=>`<div class="restaurant-card ${r.active===false?'inactive':''}">
          <div class="rest-name">${r.name||'—'}</div>
          <div class="rest-meta">${r.mealSlot||'—'} · ID: ${r.id||r.restaurantId||'—'}</div>
          <div class="rest-actions">
            ${r.active!==false
                    ? `<button class="btn btn-warning btn-sm"
onclick="deactivateRestaurant(${r.id||r.restaurantId})">DEACTIVATE</button>`
                    : `<span style="font-size:10px;color:var(--red);font-family:var(--
mono)">INACTIVE</span>`}
          </div>
        </div>`).join('')
                : '<div class="empty-state">no restaurants found. create one!</div>';
        }catch(e){document.getElementById('restaurants-grid').innerHTML='<div class="empty-state">could not
load restaurants</div>';}
    }

    async function addRestaurant(){
        const name=document.getElementById('rest-name').value.trim();
        const mealSlot=document.getElementById('rest-slot').value;
        const vid=document.getElementById('rest-vendor-id').value;
        if(!name||!vid){showAlert('NAME AND VENDOR ID REQUIRED','error');return;}
        try{
            await
apiFetch('/restaurants',{method:'POST',body:JSON.stringify({name,mealSlot,vendorId:parseInt(vid)})});
            showAlert('RESTAURANT CREATED!','success');clearRestForm();loadMyRestaurants();
        }catch(e){showAlert('FAILED: '+e.message,'error');}
    }

    async function deactivateRestaurant(id){
        if(!confirm('Deactivate this restaurant?'))return;
        try{await
apiFetch(`/restaurants/${id}/deactivate`,{method:'PUT'});showAlert('DEACTIVATED','success');loadMyRestaurants();
}
        catch(e){showAlert('FAILED','error');}
    }

    function clearRestForm(){['rest-name','rest-desc'].forEach(id=>document.getElementById(id).value='');}

    async function loadSubscriptions(){
        try{
            const data=await apiFetch('/subscriptions');
            document.getElementById('subscriptions-list').innerHTML=(data||[]).length
                ? data.map(s=>`<tr><td style="font-family:var(--mono);font-
size:12px">${s.employeeEmail||s.employeeId||'—'}</td><td>${s.restaurantName||s.restaurantId||'—'}</td><td><span
class="chi...
                : '<tr><td colspan="6" class="empty-state">no subscriptions found</td></tr>';
        }catch(e){document.getElementById('subscriptions-list').innerHTML='<tr><td colspan="6"
class="empty-state">could not load subscriptions</td></tr>';}
    }

    async function loadDeliveries(){
        try{
            const data=await apiFetch('/deliveries');
            document.getElementById('deliveries-list').innerHTML=(data||[]).length
                ? data.map(d=>`<tr><td style="font-family:var(--mono);font-
size:11px">#${d.id||'—'}</td><td>${d.restaurantName||d.restaurantId||'—'}</td><td><span class="chip
${(d.mealSlot||'').toL...
```

```
                : '<tr><td colspan="6" class="empty-state">no deliveries found</td></tr>';
        }catch(e){document.getElementById('deliveries-list').innerHTML='<tr><td colspan="6" class="empty-state">could not load deliveries</td></tr>';}
    }

    async function markDelivered(id){
        try{await apiFetch(`/deliveries/${id}/delivered`,{method:'PUT'});showAlert('MARKED AS DELIVERED','success');loadDeliveries();}
        catch(e){showAlert('FAILED','error');}
    }

    function logout(){
        localStorage.removeItem('jwt_token');localStorage.removeItem('auth_type');
fetch(`${API}/session/logout`,{method:'POST',credentials:'include'}).finally(()=>window.location.href='/google.html');
    }

    function showAlert(msg,type){
        const el=document.createElement('div');
        el.className=`alert alert-${type}`;el.textContent=msg;
        document.body.appendChild(el);setTimeout(()=>el.remove(),3000);
    }
</script>
</body>
</html>
```

97 Current: src\test\java\com\example\EmployeeManagementSystem\EmployeeManagementSystemApplicationT

```
package com.example.EmployeeManagementSystem;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class EmployeeManagementSystemApplicationTests {

    @Test
     void contextLoads() {
     }

 }

```